Fraud Detection - Methodology Notebook

Arjun Sekhar

2026-08-16

1. Introduction

1.1. Objective

This notebook documents the analytical workflow used to investigate fraud detection using both traditional classification models and a survival-based approach. The aim is to compare the baseline banking-oriented models such as the Logistic Regression against a Random Survival Forest (RSF) framework under extreme class imbalance.

1.2. Research Motivation

Fraud detection is traditionally handled as a binary classification problem. However, fraud may also be understand using a time-to-event approach, where the question is not to detect whether fraud occurs, but also with how the fraud risk evolves over time.

1.3. Analytical Roadmap

This notebook is structured to move from data understanding through to model development, evaluation, and interpretation, with a focus on both performance and explainability. It is thus organised as follows:

1. Data Description: An overview of the dataset and its key characteristics.
2. Data Preparation: Construction of modelling variables, including the proxy time index and handling of class imbalance.
3. Exploratory Data Analysis (EDA): Examination of feature distributions and class imbalance patterns.
4. Modelling Framework: Implementation of four model configurations, including Logistic Regression and Random Survival Forest (RSF), with and without SMOTE.
5. Results: Evaluation of model performance using sensitivity, AUC, and AUPRC across repeated train-test splits.
6. Explainability: Interpretation of model behaviour using global and local explanation techniques, including SHAP and LIME.
7. Discussion and Banking Implications: Analysis of model performance and relevance to real-world fraud detection workflows.
8. Limitations and Conclusion: Summary of findings, methodological limitations, and directions for future work.

2. Libraries

Some of the libraries that we use in this process that are to be loaded here include:

```
library(tidyverse)
library(caret)
```

```
library(pROC)
library(PRROC)
library(randomForest)
library(randomForestSRC)
library(survival)
library(survminer)
```

Additionally to handle the Explainability, we incorporate the following packages specific to this:

```
library(SHAPforxgboost)
library(iml)
library(lime)
```

3. Data Description

3.1. Load Data

To start things off, we will set the seed, ingest the dataset and then omit the missing values. This will make the dataset become analysis-ready.

```
## Step 1: Set seed
set.seed(44940076)

## Step 3: Load Data
fraud_data <- read.csv('creditcard.csv', header = TRUE)

## Step 4: Check for Missing Values
colSums(is.na(fraud_data))
```

```
##   Time     V1     V2     V3     V4     V5     V6     V7     V8     V9    V10
##      0      0      0      0      0      0      0      0      0      0      0
##    V11    V12    V13    V14    V15    V16    V17    V18    V19    V20    V21
##      0      0      0      0      0      0      0      0      0      0      0
##    V22    V23    V24    V25    V26    V27    V28 Amount  Class
##      0      0      0      0      0      0      0      0      0
```

```
## Step 5: Remove Missing Values
fraud_data <- na.omit(fraud_data)
```

3.2. Missing Values Check

We will quickly check whether any of the columns have missing values in them.

```
colSums(is.na(fraud_data))
```

```
##   Time     V1     V2     V3     V4     V5     V6     V7     V8     V9    V10
##      0      0      0      0      0      0      0      0      0      0      0
##    V11    V12    V13    V14    V15    V16    V17    V18    V19    V20    V21
##      0      0      0      0      0      0      0      0      0      0      0
##    V22    V23    V24    V25    V26    V27    V28 Amount  Class
##      0      0      0      0      0      0      0      0      0
```

Good, this means that there is nothing much to worry about from the missingness standpoint.

3.3. Structure and Dimensions

This next step will involve understanding the structure and dimensions, such that we are able to understand the specifics of the data we are dealing with.

```
## [1] 284807     31
```

```
## Rows: 284,807
## Columns: 31
## $ Time   <dbl> 0, 0, 1, 1, 2, 2, 4, 7, 7, 9, 10, 10, 10, 11, 12, 12, 12, 13, 1~
## $ V1     <dbl> -1.3598071, 1.1918571, -1.3583541, -0.9662717, -1.1582331, -0.4~
## $ V2     <dbl> -0.07278117, 0.26615071, -1.34016307, -0.18522601, 0.87773675, ~
## $ V3     <dbl> 2.53634674, 0.16648011, 1.77320934, 1.79299334, 1.54871785, 1.1~
## $ V4     <dbl> 1.37815522, 0.44815408, 0.37977959, -0.86329128, 0.40303393, -0~
## $ V5     <dbl> -0.33832077, 0.06001765, -0.50319813, -0.01030888, -0.40719338,~
## $ V6     <dbl> 0.46238778, -0.08236081, 1.80049938, 1.24720317, 0.09592146, -0~
## $ V7     <dbl> 0.239598554, -0.078802983, 0.791460956, 0.237608940, 0.59294074~
## $ V8     <dbl> 0.098697901, 0.085101655, 0.247675787, 0.377435875, -0.27053267~
```

```
## $ V9     <dbl> 0.3637870, -0.2554251, -1.5146543, -1.3870241, 0.8177393, -0.56~
## $ V10    <dbl> 0.09079417, -0.16697441, 0.20764287, -0.05495192, 0.75307443, -~
## $ V11    <dbl> -0.55159953, 1.61272666, 0.62450146, -0.22648726, -0.82284288, ~
## $ V12    <dbl> -0.61780086, 1.06523531, 0.06608369, 0.17822823, 0.53819555, 0.~
## $ V13    <dbl> -0.99138985, 0.48909502, 0.71729273, 0.50775687, 1.34585159, -0~
## $ V14    <dbl> -0.31116935, -0.14377230, -0.16594592, -0.28792375, -1.11966983~
## $ V15    <dbl> 1.468176972, 0.635558093, 2.345864949, -0.631418118, 0.17512113~
## $ V16    <dbl> -0.47040053, 0.46391704, -2.89008319, -1.05964725, -0.45144918,~
## $ V17    <dbl> 0.207971242, -0.114804663, 1.109969379, -0.684092786, -0.237033~
## $ V18    <dbl> 0.02579058, -0.18336127, -0.12135931, 1.96577500, -0.03819479, ~
## $ V19    <dbl> 0.40399296, -0.14578304, -2.26185710, -1.23262197, 0.80348692, ~
## $ V20    <dbl> 0.25141210, -0.06908314, 0.52497973, -0.20803778, 0.40854236, 0~
## $ V21    <dbl> -0.018306778, -0.225775248, 0.247998153, -0.108300452, -0.00943~
## $ V22    <dbl> 0.277837576, -0.638671953, 0.771679402, 0.005273597, 0.79827849~
## $ V23    <dbl> -0.110473910, 0.101288021, 0.909412262, -0.190320519, -0.137458~
## $ V24    <dbl> 0.06692807, -0.33984648, -0.68928096, -1.17557533, 0.14126698, ~
## $ V25    <dbl> 0.12853936, 0.16717040, -0.32764183, 0.64737603, -0.20600959, -~
## $ V26    <dbl> -0.18911484, 0.12589453, -0.13909657, -0.22192884, 0.50229222, ~
## $ V27    <dbl> 0.133558377, -0.008983099, -0.055352794, 0.062722849, 0.2194222~
## $ V28    <dbl> -0.021053053, 0.014724169, -0.059751841, 0.061457629, 0.2151531~
## $ Amount <dbl> 149.62, 2.69, 378.66, 123.50, 69.99, 3.67, 4.99, 40.80, 93.20, ~
## $ Class  <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
```

```
##       Time              V1                  V2                  V3          
##  Min.   :     0   Min.   :-56.40751   Min.   :-72.71573   Min.   :-48.3256  
##  1st Qu.: 54202   1st Qu.: -0.92037   1st Qu.: -0.59855   1st Qu.: -0.8904  
##  Median : 84692   Median :  0.01811   Median :  0.06549   Median :  0.1798  
##  Mean   : 94814   Mean   :  0.00000   Mean   :  0.00000   Mean   :  0.0000  
##  3rd Qu.:139321   3rd Qu.:  1.31564   3rd Qu.:  0.80372   3rd Qu.:  1.0272  
##  Max.   :172792   Max.   :  2.45493   Max.   : 22.05773   Max.   :  9.3826  
##        V4                 V5                  V6                  V7          
##  Min.   :-5.68317   Min.   :-113.74331   Min.   :-26.1605   Min.   :-43.5572  
##  1st Qu.:-0.84864   1st Qu.:  -0.69160   1st Qu.: -0.7683   1st Qu.: -0.5541  
##  Median :-0.01985   Median :  -0.05434   Median : -0.2742   Median :  0.0401  
##  Mean   : 0.00000   Mean   :   0.00000   Mean   :  0.0000   Mean   :  0.0000  
##  3rd Qu.: 0.74334   3rd Qu.:   0.61193   3rd Qu.:  0.3986   3rd Qu.:  0.5704  
##  Max.   :16.87534   Max.   :  34.80167   Max.   : 73.3016   Max.   :120.5895  
##        V8                 V9                 V10                 V11          
##  Min.   :-73.21672   Min.   :-13.43407   Min.   :-24.58826   Min.   :-4.79747  
##  1st Qu.: -0.20863   1st Qu.: -0.64310   1st Qu.: -0.53543   1st Qu.:-0.76249  
##  Median :  0.02236   Median : -0.05143   Median : -0.09292   Median :-0.03276  
##  Mean   :  0.00000   Mean   :  0.00000   Mean   :  0.00000   Mean   : 0.00000  
##  3rd Qu.:  0.32735   3rd Qu.:  0.59714   3rd Qu.:  0.45392   3rd Qu.: 0.73959  
##  Max.   : 20.00721   Max.   : 15.59499   Max.   : 23.74514   Max.   :12.01891  
##       V12                V13                 V14                 V15          
##  Min.   :-18.6837   Min.   :-5.79188   Min.   :-19.2143   Min.   :-4.49894  
##  1st Qu.: -0.4056   1st Qu.:-0.64854   1st Qu.: -0.4256   1st Qu.:-0.58288  
##  Median :  0.1400   Median :-0.01357   Median :  0.0506   Median : 0.04807  
##  Mean   :  0.0000   Mean   : 0.00000   Mean   :  0.0000   Mean   : 0.00000  
##  3rd Qu.:  0.6182   3rd Qu.: 0.66250   3rd Qu.:  0.4931   3rd Qu.: 0.64882  
##  Max.   :  7.8484   Max.   : 7.12688   Max.   : 10.5268   Max.   : 8.87774  
##       V16                V17                 V18          
##  Min.   :-14.12985   Min.   :-25.16280   Min.   :-9.498746  
##  1st Qu.: -0.46804   1st Qu.: -0.48375   1st Qu.:-0.498850
```

```
##  Median :  0.06641   Median : -0.06568   Median :-0.003636
##  Mean   :  0.00000   Mean   :  0.00000   Mean   : 0.000000
##  3rd Qu.:  0.52330   3rd Qu.:  0.39968   3rd Qu.: 0.500807
##  Max.   : 17.31511   Max.   :  9.25353   Max.   : 5.041069
##       V19                 V20                 V21
##  Min.   :-7.213527   Min.   :-54.49772   Min.   :-34.83038
##  1st Qu.:-0.456299   1st Qu.: -0.21172   1st Qu.: -0.22839
##  Median : 0.003735   Median : -0.06248   Median : -0.02945
##  Mean   : 0.000000   Mean   :  0.00000   Mean   :  0.00000
##  3rd Qu.: 0.458949   3rd Qu.:  0.13304   3rd Qu.:  0.18638
##  Max.   : 5.591971   Max.   : 39.42090   Max.   : 27.20284
##       V22                 V23                 V24
##  Min.   :-10.933144   Min.   :-44.80774   Min.   :-2.83663
##  1st Qu.: -0.542350   1st Qu.: -0.16185   1st Qu.:-0.35459
##  Median :  0.006782   Median : -0.01119   Median : 0.04098
##  Mean   :  0.000000   Mean   :  0.00000   Mean   : 0.00000
##  3rd Qu.:  0.528554   3rd Qu.:  0.14764   3rd Qu.: 0.43953
##  Max.   : 10.503090   Max.   : 22.52841   Max.   : 4.58455
##       V25                 V26                 V27
##  Min.   :-10.29540   Min.   :-2.60455   Min.   :-22.565679
##  1st Qu.: -0.31715   1st Qu.:-0.32698   1st Qu.: -0.070839
##  Median :  0.01659   Median :-0.05214   Median :  0.001342
##  Mean   :  0.00000   Mean   : 0.00000   Mean   :  0.000000
##  3rd Qu.:  0.35072   3rd Qu.: 0.24095   3rd Qu.:  0.091045
##  Max.   :  7.51959   Max.   : 3.51735   Max.   : 31.612198
##       V28                Amount              Class
##  Min.   :-15.43008   Min.   :    0.00   Min.   :0.000000
##  1st Qu.: -0.05296   1st Qu.:    5.60   1st Qu.:0.000000
##  Median :  0.01124   Median :   22.00   Median :0.000000
##  Mean   :  0.00000   Mean   :   88.35   Mean   :0.001727
##  3rd Qu.:  0.07828   3rd Qu.:   77.17   3rd Qu.:0.000000
##  Max.   : 33.84781   Max.   :25691.16   Max.   :1.000000
```

3.4. Class Distribution

Finally, we will see what the breakdown of the data is, based on the 'Class' variable. This will also establish the imbalance that we are dealing with here.

```
table(fraud_data$Class)
```

```
##
##      0      1
## 284315    492
```

```
prop.table(table(fraud_data$Class))
```

```
##
##           0           1
## 0.998272514 0.001727486
```

As shown above, there is a clear imbalance, where less than 1% of the data is recorded as a fraudulent transaction.

4. Initial Notes on the Dataset

The dataset was loaded from the `creditcard.csv` file and was cleaned using the listwise deletion approach, which was through `na.omit()`. Following this step, there were no further missing values remaining in the dataset.

Inspecting the response variable, we can see how the data is highly imbalanced, with fraudulent transactions representing a small proportion of the entire data. This is an important feature of the problem, as this directly impacts the model fitting, prediction quality, and the choice of evaluation metrics.

The predictor variables are largely anonymised, limited the direct substantiative interpretation of the individual features. However, the dataset remains useful for evaluating the model behaviour under extreme class imbalance.

Although the original dataset is structured as a fraud detection problem, this study extends the analysis into a survival framework through the later construction of a proxied time-to-event representation.

5. Data Preparation

In order to prepare for the section on the Exploratory Data Analysis (EDA), we want to ensure that the data has been generated and prepared appropriately, which is what we will focus on this section.

5.1. Create the Survival Variables

Firstly, we will work towards creating our respective survival variables, as this will set the scene to understand the structure of the data better.

```
fraud_data$time <- 1:nrow(fraud_data)      # Proxy time index
fraud_data$status <- fraud_data$Class      # Event indicator: 1 = fraud, 0 = non-fraud
```

5.2. Downsample Majority Class for Computational Efficiency

The original dataset is highly imbalanced and computationally large. To make this more robust, all fraud cases were retained, while a random sample of 50000 non-fraud cases were selected. This preserves the minority class of the substantive interest while reducing the runtime and memory burden of subsequent modelling work. This is what we call **downsampling**.

While this increases the fraud percentage by 0.8%, it also means that it stays well in line with the general percentage, wherein around than 1% of transactions are recorded as a 'fraud'.

5.3 Train/Test Split

The dataset was partitioned into training and testing subsets using a stratified sampling approach. This ensures that the class distribution is preserved across both subsets, which is particularly important given the extreme imbalance in the dataset.

A 70–30 split was adopted, where 70% of the data is used for model training and the remaining 30% is reserved for out-of-sample evaluation. This approach allows for a robust assessment of model performance while ensuring sufficient data is available for training.

To further improve the stability of the results, this partitioning process is repeated multiple times in later sections. This repeated sampling framework helps reduce variance in performance estimates and provides a more reliable comparison between modelling approaches.

5.4. Create RSF Ready Train and Test Data

To enable the use of a survival-based modelling framework, the dataset was restructured to include the necessary components for Random Survival Forest (RSF) modelling. Specifically, the proxy time variable and event indicator (`status`) were used to define the survival object.

Since the `status` variable directly represents the fraud outcome, the original `Class` variable was removed from the modelling dataset to avoid duplication and potential leakage. This ensures that the model is trained using a consistent outcome representation.

The predictor variables were retained in their original form, with the exception of the time variable when required for specific preprocessing steps such as SMOTE. This results in a dataset that is compatible with RSF modelling while maintaining alignment with the original feature space.

This structured preparation ensures that both classification and survival-based models can be applied to a consistent dataset, enabling a fair comparison of their performance in later sections.

5.5. Benchmarking Sample

To support efficient experimentation and reduce computational overhead, a benchmarking sample of the training data is constructed. This subset allows for faster iteration during model development, particularly for computationally intensive procedures such as repeated resampling, SMOTE, and explainability methods.

A fixed-size random sample of the training dataset is taken, with a maximum of 50,000 observations. This ensures that the sample remains sufficiently large to preserve the underlying data structure while enabling more efficient computation.

The sampling process is performed using a fixed random seed to ensure reproducibility. Importantly, this benchmarking sample is used only for intermediate experimentation and validation, while final model evaluation is conducted on the full dataset to ensure robustness of results.

```
set.seed(44940076)

sample_size <- min(50000, nrow(rsf_train))
rsf_train_sample <- rsf_train[sample(nrow(rsf_train), sample_size), ]

dim(rsf_train_sample)
```

```
## [1] 35345    32
```

5.6. Notes on the RSF Data Construction

A proxy time index was created to enable a survival-style modelling framework. In this setup, `status` represents the fraud event indicator. Since `status` duplicates the outcome information contained in `Class`, the original `Class` variable was removed from the RSF-specific modelling data to avoid leakage and duplication.

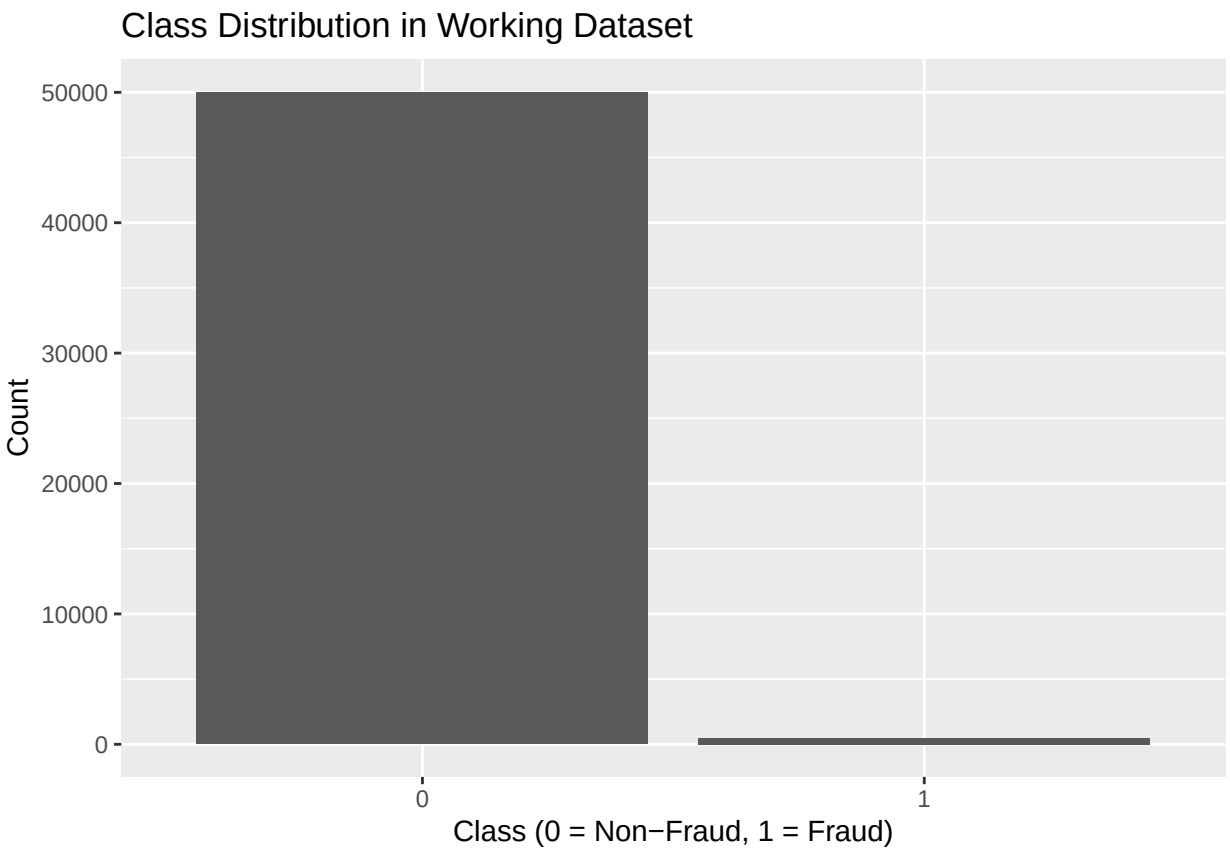
Additionally, an optional benchmarking subset was created to support faster experimentation where required.

6. Exploratory Data Analysis

This stage will involve analysing the working dataset.

6.1. Class Distribution in the Working Dataset

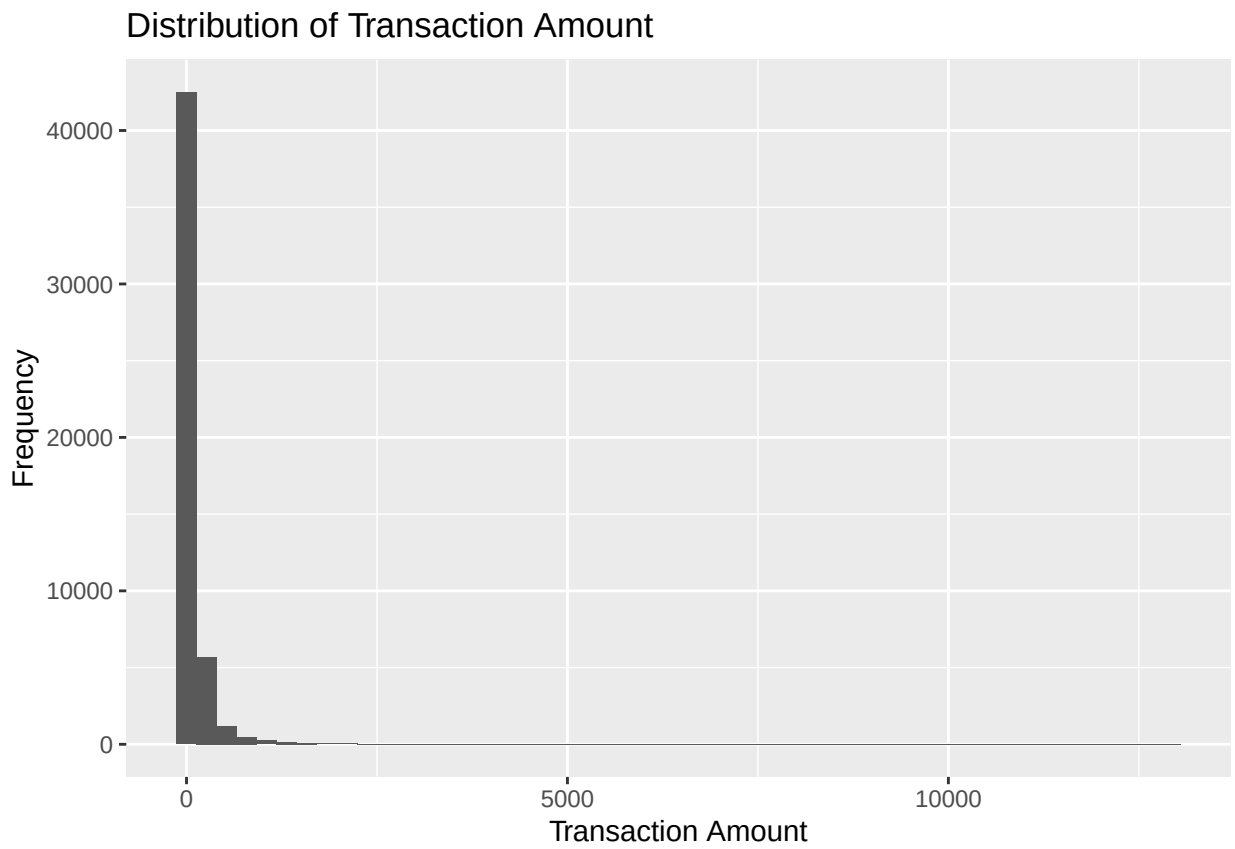
```
fraud_subset %>%
  count(Class) %>%
  ggplot(aes(x = factor(Class), y = n)) +
  geom_col() +
  labs(
    title = "Class Distribution in Working Dataset",
    x = "Class (0 = Non-Fraud, 1 = Fraud)",
    y = "Count"
  )
```



As previously established, there is an extreme class imbalance that exists in the dataset. The nature of this imbalance can be felt in the above visual.

6.2. Distribution of Transaction Amount

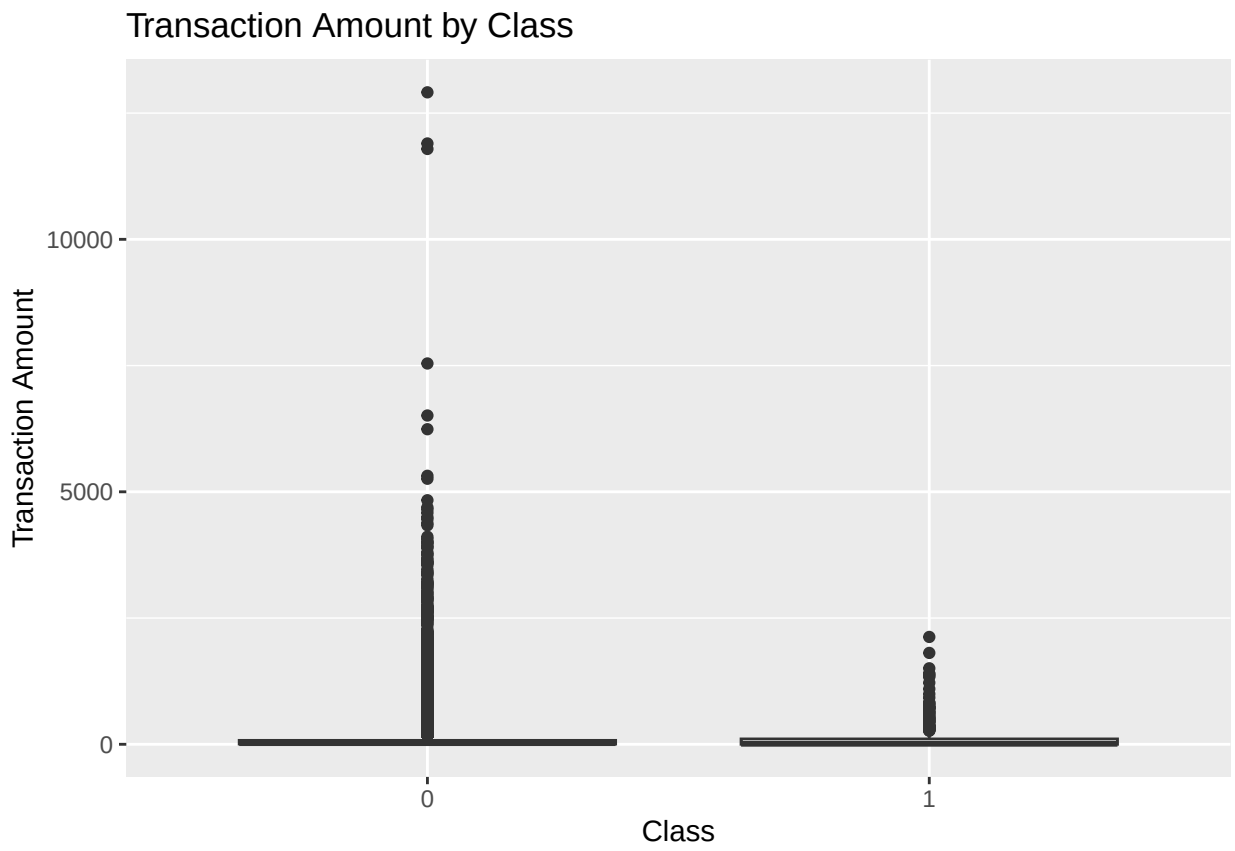
```
ggplot(fraud_subset, aes(x = Amount)) +
  geom_histogram(bins = 50) +
  labs(
    title = "Distribution of Transaction Amount",
    x = "Transaction Amount",
    y = "Frequency"
  )
```



From the above, we can see that the transaction amounts are typically the small amounts. While this is true, it also shows how fraudulent transactions are predominantly low in value.

6.3. Transaction Amount by Class

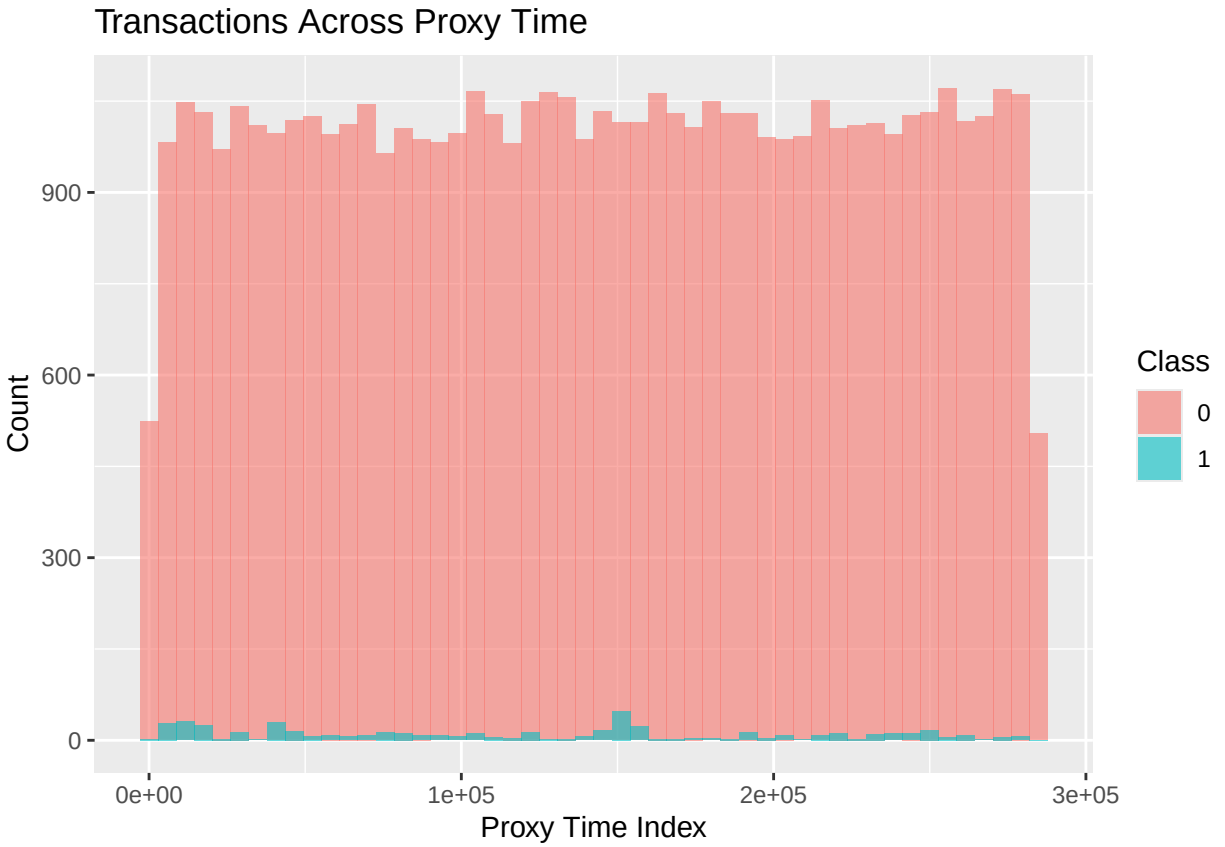
```
ggplot(fraud_subset, aes(x = factor(Class), y = Amount)) +
  geom_boxplot() +
  labs(
    title = "Transaction Amount by Class",
    x = "Class",
    y = "Transaction Amount"
  )
```



The above shows us how the detected amounts are typically smaller, which as mentioned in the previous subsection, shows how the detected amounts are negligible amounts. This also means that people are most impacted when they least expect it.

6.4. Transactions across Proxy Time

```
ggplot(fraud_subset, aes(x = time, fill = factor(Class))) +
  geom_histogram(bins = 50, position = "identity", alpha = 0.6) +
  labs(
    title = "Transactions Across Proxy Time",
    x = "Proxy Time Index",
    y = "Count",
    fill = "Class"
  )
```

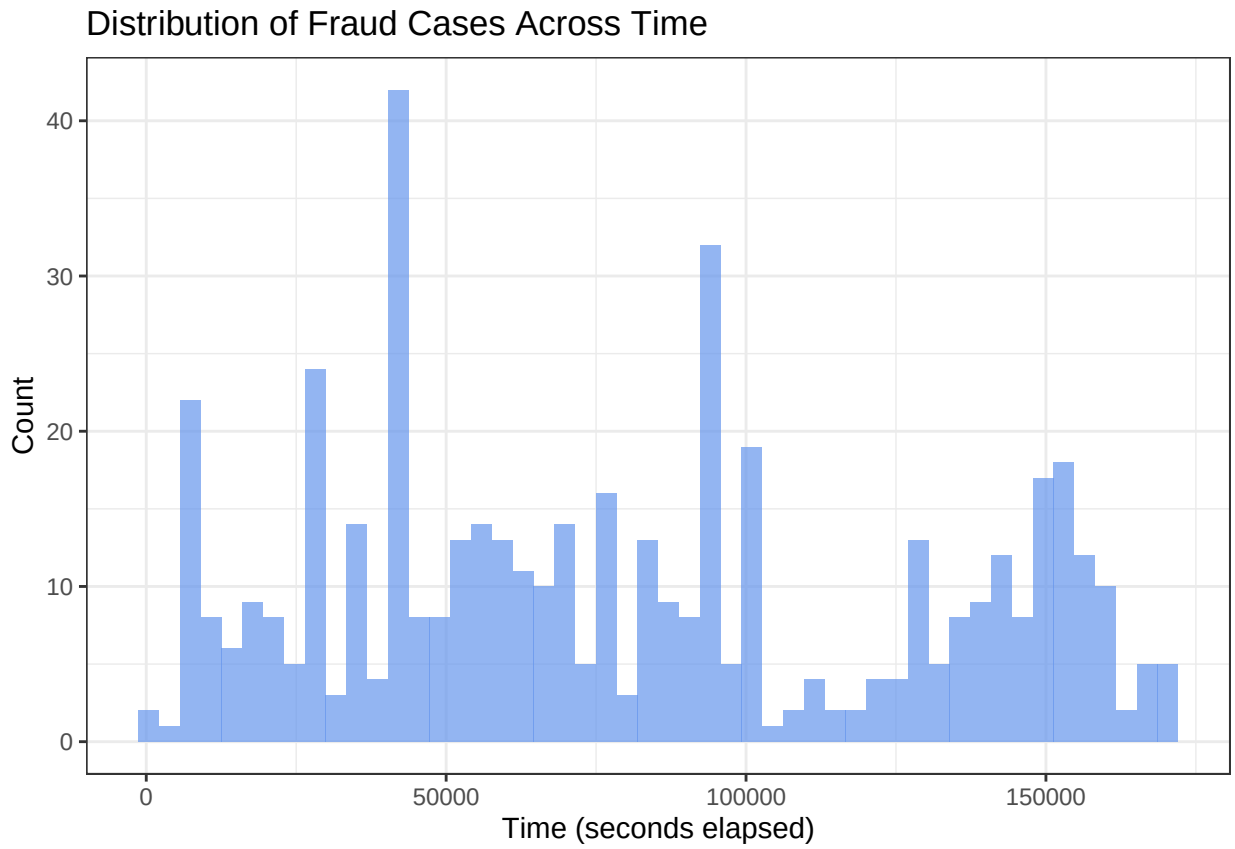


6.5. EDA Commentary

- **Class Imbalance after Downsampling:** While the visualisation alone does not clearly convey the extent of the change, the class distribution shows that the fraud rate increases from approximately 0.17% in the original dataset to approximately 0.97% in the working subset. Although the dataset remains highly imbalanced, this represents a meaningful improvement that supports more stable model training while still reflecting realistic fraud prevalence levels.
- **Concentration of Fraud Cases in Proxy Time Index:** The distribution of fraud cases across the proxy time index appears relatively uniform, with no strong clustering in specific regions. However, it is important to note that the time variable is a constructed ordering index rather than a true temporal measure. As such, this observation does not imply that fraud is independent of time, but rather that no artificial ordering bias is introduced into the modelling framework.
- **Transaction Amount and Class:** The distribution of transaction amounts indicates that most transactions, including fraudulent ones, occur at relatively low monetary values. This suggests that fraud detection is not limited to high-value transactions and that models must be sensitive to subtle patterns in smaller transactions. However, the presence of higher-value outliers indicates that fraud is not exclusively associated with low amounts.
- **Suitability of later modelling choices:** The observed characteristics of the dataset support the use of both classification and survival-based modelling approaches. The persistent class imbalance motivates the inclusion of models that are robust to rare event detection, such as Random Forest and Random Survival Forest. Additionally, the absence of strong temporal clustering, combined with the constructed proxy time index, provides a basis for exploring a survival framework as a risk-ranking mechanism rather than a strict temporal prediction model. Together, these observations justify the comparative modelling approach adopted in later sections.

6.6. Fraud Distribution Across Time

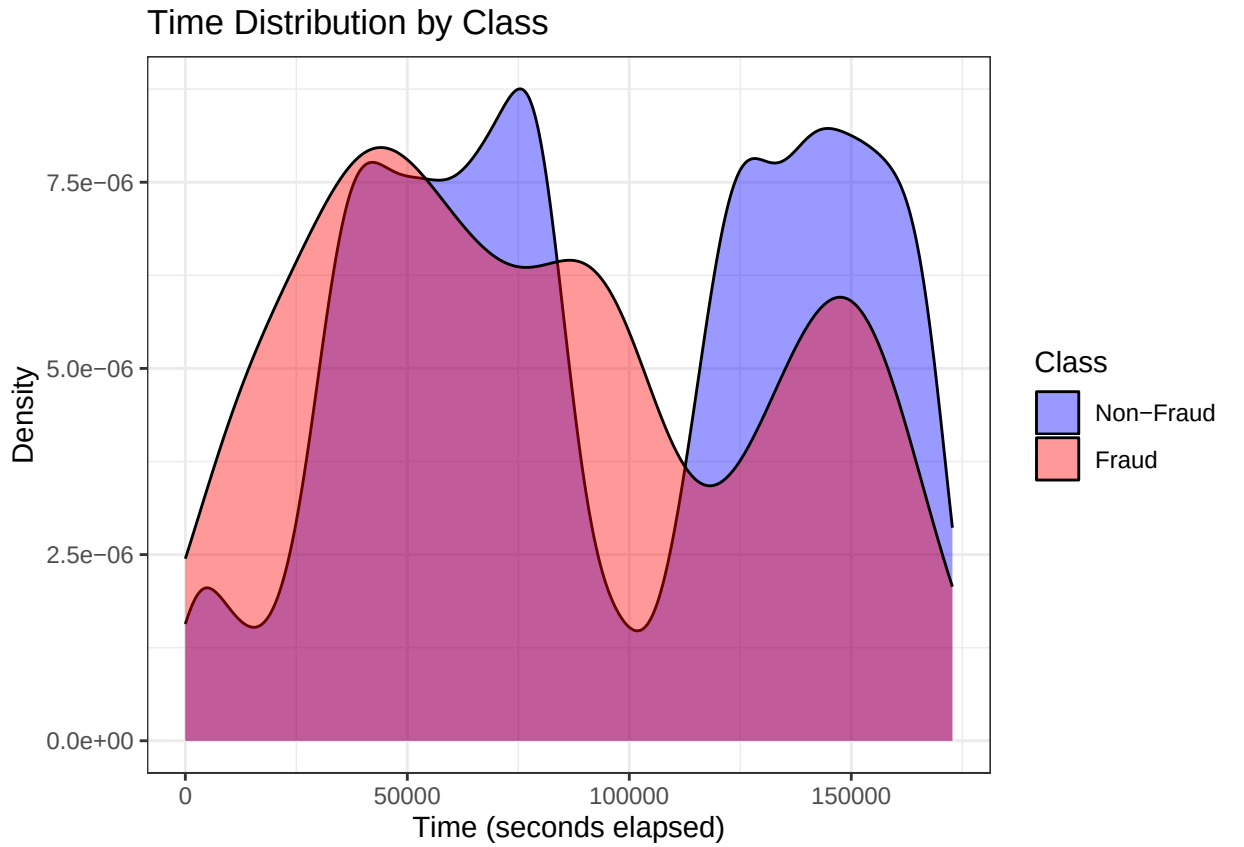
```
## Plot 1: Fraud cases only across original Time variable
ggplot(fraud_subset[fraud_subset$Class == 1, ], aes(x = Time)) +
  geom_histogram(bins = 50, fill = "cornflowerblue", alpha = 0.7) +
  labs(
    title = "Distribution of Fraud Cases Across Time",
    x = "Time (seconds elapsed)",
    y = "Count"
  ) +
  theme_bw()
```



The distribution of fraud cases across the original `Time` variable shows a broad coverage across the 48-hour observation window, with no strong clustering in any one region. There are mild peaks around 40000 seconds and 90000 seconds, which may reflect daily patterns in fraud occurrence, but the overall spread is approximately uniform. This confirms that the `Time` variable does not exhibit the kind of structured time-to-event behaviour that an RSF model is designed to exploit. We can see that fraud cases are scattered across the timeline, rather than being concentrated in a way that would support meaningful survival modelling.

```
## Plot 2: Density comparison of fraud vs non-fraud
ggplot(fraud_subset, aes(x = Time, fill = factor(Class))) +
  geom_density(alpha = 0.4) +
  labs(
    title = "Time Distribution by Class",
    x = "Time (seconds elapsed)",
    y = "Density",
    fill = "Class"
```

```
  ) +
  scale_fill_manual(values = c("0" = "blue", "1" = "red"),
                    labels = c("Non-Fraud", "Fraud")) +
  theme_bw()
```



The density comparison reveals that both fraud and non-fraud transactions follow broadly similar temporal distributions, with both exhibiting an approximately bimodal pattern, consistent with the two-day daily cycle. They encouraged a shared dip at around 75000-100000 seconds, which likely reflected a reduced transaction activity overnight. While the fraud density dips slightly more during this period, the two distributions overlap substantially, confirming that time does not discriminate between fraud and non-fraud in this dataset. As a result, we will explore the Relative Fraud Risk Across Time more specifically.

```
library(ggplot2)

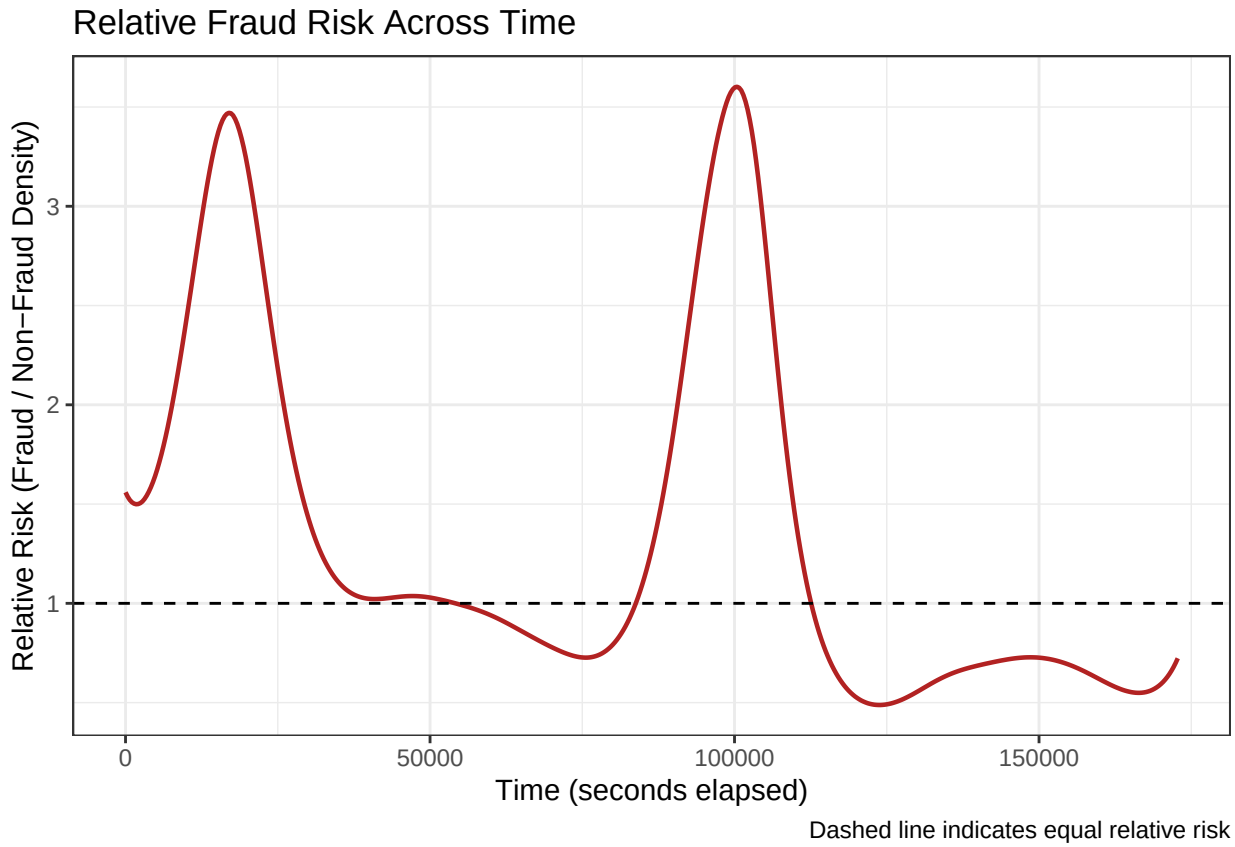
# Compute densities at common points
time_range <- range(fraud_subset$Time)
time_points <- seq(time_range[1], time_range[2], length.out = 500)

fraud_density <- density(fraud_subset$Time[fraud_subset$Class == 1],
                         from = time_range[1], to = time_range[2], n = 500)
nonfraud_density <- density(fraud_subset$Time[fraud_subset$Class == 0],
                            from = time_range[1], to = time_range[2], n = 500)

# Relative risk: fraud density / non-fraud density
relative_risk <- data.frame(
  Time = fraud_density$x,
```

```
  Relative_Risk = fraud_density$y / nonfraud_density$y
)

ggplot(relative_risk, aes(x = Time, y = Relative_Risk)) +
  geom_line(colour = "firebrick", linewidth = 0.8) +
  geom_hline(yintercept = 1, linetype = "dashed", colour = "black") +
  labs(
    title = "Relative Fraud Risk Across Time",
    x = "Time (seconds elapsed)",
    y = "Relative Risk (Fraud / Non-Fraud Density)",
    caption = "Dashed line indicates equal relative risk"
  ) +
  theme_bw()
```



Observations: The relative fraud risk plot reveals a more nuanced temporal pattern than the density overlay alone, with fraud risk peaking at approximately 3-3.5 times the non-fraud density during the low-activity periods (i.e. 15000 and 100000 seconds) before dropping well below the equal-risk threshold during the busier transaction windows.

This confirms that while fraud cases are broadly distributed across the observation window in absolute terms, the concentration is disproportionate, relative to legitimate transactions during the overnight and early morning periods, which is consistent with reduced monitoring and lower transaction volumes that create more favourable conditions for fraudulent activity. This relative risk structure motivates the simulation analysis in Section 8.5, which investigates whether the RSF's survival framework is better positioned than a Logistic Regression to exploit the temporal pattern.

7. Modelling Framework

7.1. Experimental Design

The modelling framework evaluates the performance of different approaches across two key dimensions:

1. Model type:
   - Logistic Regression (classification baseline)
   - Random Survival Forest (RSF)
2. Imbalance handling:
   - Without SMOTE
   - With SMOTE applied to the training data only

This results in four model configurations:

- Logistic Regression (No SMOTE)
- Logistic Regression (SMOTE)
- Random Survival Forest (No SMOTE)
- Random Survival Forest (SMOTE)

All models are evaluated using repeated stratified train-test splits to ensure robustness of results. Note as well that all models were **evaluated across 30 repeated stratified 70-30 train-test splits.**

7.2. Logistic Regression (No SMOTE)

A baseline Logistic Regression model was fitted using the original imbalanced training data. No resampling techniques were applied in this setup, allowing the model to reflect standard classification behaviour under class imbalance.

```
## Step 1: Run relevant packages.
library(caret)
library(pROC)
library(PRROC)

## Step 2: Set the seed
set.seed(44940076)

## Step 3: Establish the number of splits and threshold grid
n_splits <- 30
threshold_grid <- c(0.90, 0.95, 0.975, 0.99, 0.995)

## Step 4: Create storage - one set of vectors per threshold
results_store <- list()
for (t in threshold_grid) {
  results_store[[as.character(t)]] <- list(
    sens  = numeric(n_splits),
    prec  = numeric(n_splits),
    auc   = numeric(n_splits),
    auprc = numeric(n_splits)
  )
}

## Step 5: Execute the Logistic Regression (No SMOTE)
for (i in 1:n_splits) {

  # 1. Stratified 70-30 split
  lr_train_idx <- createDataPartition(fraud_subset$status, p = 0.7, list = FALSE)
```

```
  lr_train_data <- fraud_subset[lr_train_idx, ]
  lr_test_data  <- fraud_subset[-lr_train_idx, ]

  # 2. Fit Logistic Regression model (ONCE per split)
  lr_model <- glm(status ~ . - time - Class, data = lr_train_data, family = binomial)

  # 3. Predict probabilities (ONCE per split)
  lr_train_prob <- predict(lr_model, newdata = lr_train_data, type = "response")
  lr_test_prob  <- predict(lr_model, newdata = lr_test_data, type = "response")

  # 4. ROC + AUC (threshold-independent, compute once)
  lr_true_labels <- as.numeric(as.character(lr_test_data$status))

  lr_auc_i <- as.numeric(auc(
    roc(response = lr_true_labels, predictor = lr_test_prob, quiet = TRUE)
  ))

  # 5. AUPRC (threshold-independent, compute once)
  lr_pr_obj <- pr.curve(
    scores.class0 = lr_test_prob[lr_true_labels == 1],
    scores.class1 = lr_test_prob[lr_true_labels == 0],
    curve = FALSE
  )
  lr_auprc_i <- lr_pr_obj$auc.integral

  # 6. Loop over thresholds
  for (t in threshold_grid) {

    # 6a. Threshold from training data
    lr_threshold <- quantile(lr_train_prob, probs = t, na.rm = TRUE)

    # 6b. Apply threshold
    lr_test_pred <- ifelse(lr_test_prob > lr_threshold, 1, 0)

    # 6c. Confusion Matrix
    lr_cm <- confusionMatrix(
      factor(lr_test_pred, levels = c(0, 1)),
      factor(lr_test_data$status, levels = c(0, 1)),
      positive = "1"
    )

    # 6d. Store metrics
    tk <- as.character(t)
    results_store[[tk]]$sens[i]  <- lr_cm$byClass["Sensitivity"]
    results_store[[tk]]$prec[i]  <- lr_cm$byClass["Precision"]
    results_store[[tk]]$auc[i]   <- lr_auc_i
    results_store[[tk]]$auprc[i] <- lr_auprc_i
  }
}
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```r
## Step 6: Store results
results_list <- list()
for (t in threshold_grid) {
  tk <- as.character(t)
  results_list[[tk]] <- data.frame(
    Threshold      = t,
    Sensitivity    = mean(results_store[[tk]]$sens),
    Sensitivity_SD = sd(results_store[[tk]]$sens),
    Precision      = mean(results_store[[tk]]$prec),
    Precision_SD   = sd(results_store[[tk]]$prec),
    AUC            = mean(results_store[[tk]]$auc),
    AUC_SD         = sd(results_store[[tk]]$auc),
    AUPRC          = mean(results_store[[tk]]$auprc),
    AUPRC_SD       = sd(results_store[[tk]]$auprc)
  )
}

results_lr_threshold <- do.call(rbind, results_list)
auprc_lr_splits <- results_store[["0.99"]]$auprc

saveRDS(results_lr_threshold, "lr_threshold_results.rds")
```

7.3. Logistic Regression (SMOTE)

This model is used to assess whether synthetic balancing improves classification performance under extreme class imbalance. To address class imbalance, SMOTE was applied to the training data only. Synthetic minority samples were generated based on the feature space, while the test set remained untouched to preserve a realistic evaluation setting. The time variable was excluded from SMOTE generation to avoid introducing artificial temporal structure into the data.

```r
## Step 1: Run relevant packages.
library(caret)
library(dplyr)
library(smotefamily)
library(pROC)
library(PRROC)

## Step 2: Set the seed
set.seed(44940076)

## Step 3: Establish the number of splits and threshold grid
```

```r
n_splits <- 30
threshold_grid <- c(0.90, 0.95, 0.975, 0.99, 0.995)

## Step 4: Create storage - one set of vectors per threshold
results_store <- list()
for (t in threshold_grid) {
  results_store[[as.character(t)]] <- list(
    sens  = numeric(n_splits),
    prec  = numeric(n_splits),
    auc   = numeric(n_splits),
    auprc = numeric(n_splits)
  )
}

## Step 5: Execute the Logistic Regression (SMOTE)
for (i in 1:n_splits) {

  # 1. Stratified 70-30 split BEFORE SMOTE
  lr_smote_train_idx <- createDataPartition(fraud_subset$status, p = 0.7, list = FALSE)

  lr_smote_train_data <- fraud_subset[lr_smote_train_idx, ]
  lr_smote_test_data  <- fraud_subset[-lr_smote_train_idx, ]

  # 2. Ensure numeric 0/1 outcome labels
  lr_smote_train_data$status <- as.numeric(as.character(lr_smote_train_data$status))
  lr_smote_test_data$status  <- as.numeric(as.character(lr_smote_test_data$status))

  # 3. Prepare training data for SMOTE
  lr_smote_train_input <- lr_smote_train_data %>%
    dplyr::select(-time, -Class)

  lr_smote_x_train <- lr_smote_train_input %>%
    dplyr::select(-status)

  lr_smote_y_train <- lr_smote_train_input$status

  # 4. Choose SMOTE neighbourhood size safely
  lr_smote_n_min <- min(table(lr_smote_y_train))
  lr_smote_k_use <- min(5, max(1, lr_smote_n_min - 1))

  # 5. Apply SMOTE to training data only
  lr_smote_output <- SMOTE(
    X = lr_smote_x_train,
    target = lr_smote_y_train,
    K = lr_smote_k_use,
    dup_size = 1
  )

  lr_smote_train_balanced <- lr_smote_output$data
  lr_smote_train_balanced$status <- as.numeric(as.character(lr_smote_train_balanced$class))
  lr_smote_train_balanced$class <- NULL

  # 6. Fit Logistic Regression on SMOTE-balanced training data (ONCE per split)
```

```r
lr_smote_model <- glm(
  status ~ .,
  data = lr_smote_train_balanced,
  family = binomial
)

# 7. Predict probabilities (ONCE per split)
lr_smote_train_prob_original <- predict(lr_smote_model,
                                        newdata = lr_smote_train_data,
                                        type = "response")

lr_smote_test_prob <- predict(lr_smote_model,
                              newdata = lr_smote_test_data,
                              type = "response")

# 8. ROC + AUC (threshold-independent, compute once)
lr_smote_auc_i <- as.numeric(auc(
  roc(response = lr_smote_test_data$status,
      predictor = lr_smote_test_prob,
      quiet = TRUE)
))

# 9. AUPRC (threshold-independent, compute once)
lr_smote_true_labels <- lr_smote_test_data$status

lr_smote_keep <- is.finite(lr_smote_test_prob) & is.finite(lr_smote_true_labels)
lr_smote_scores <- lr_smote_test_prob[lr_smote_keep]
lr_smote_labels <- lr_smote_true_labels[lr_smote_keep]

lr_smote_pr_obj <- pr.curve(
  scores.class0 = lr_smote_scores[lr_smote_labels == 1],
  scores.class1 = lr_smote_scores[lr_smote_labels == 0],
  curve = FALSE
)
lr_smote_auprc_i <- lr_smote_pr_obj$auc.integral

# 10. Loop over thresholds
for (t in threshold_grid) {

  # 10a. Threshold from training data
  lr_smote_threshold <- quantile(lr_smote_train_prob_original,
                                 probs = t,
                                 na.rm = TRUE)

  # 10b. Apply threshold
  lr_smote_test_pred <- ifelse(lr_smote_test_prob > lr_smote_threshold, 1, 0)

  # 10c. Confusion matrix
  lr_smote_cm <- confusionMatrix(
    factor(lr_smote_test_pred, levels = c(0, 1)),
    factor(lr_smote_test_data$status, levels = c(0, 1)),
    positive = "1"
  )
```

```
    # 10d. Store metrics
    tk <- as.character(t)
    results_store[[tk]]$sens[i]  <- lr_smote_cm$byClass["Sensitivity"]
    results_store[[tk]]$prec[i]  <- lr_smote_cm$byClass["Precision"]
    results_store[[tk]]$auc[i]   <- lr_smote_auc_i
    results_store[[tk]]$auprc[i] <- lr_smote_auprc_i
  }
}
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
## Step 6: Store results
results_list <- list()
for (t in threshold_grid) {
  tk <- as.character(t)
  results_list[[tk]] <- data.frame(
    Threshold      = t,
    Sensitivity    = mean(results_store[[tk]]$sens),
    Sensitivity_SD = sd(results_store[[tk]]$sens),
    Precision      = mean(results_store[[tk]]$prec),
    Precision_SD   = sd(results_store[[tk]]$prec),
    AUC            = mean(results_store[[tk]]$auc),
    AUC_SD         = sd(results_store[[tk]]$auc),
    AUPRC          = mean(results_store[[tk]]$auprc),
    AUPRC_SD       = sd(results_store[[tk]]$auprc)
```

```
  )
}

results_lr_smote_threshold <- do.call(rbind, results_list)
auprc_lr_smote_splits <- results_store[["0.99"]]$auprc

saveRDS(results_lr_smote_threshold, "lr_smote_threshold_results.rds")
```

7.4. Random Survival Forest (No SMOTE)

A Random Survival Forest (RSF) model was fitted using the survival representation of the data. The model leverages the proxy time index and event indicator to produce a risk-based prediction. No resampling was applied in this configuration, allowing assessment of RSF performance under naturally imbalanced conditions.

```
## Step 1: Run relevant packages.
library(caret)
library(dplyr)
library(randomForestSRC)
library(pROC)
library(PRROC)

## Step 2: Set the seed
set.seed(44940076)

## Step 3: Establish the number of splits and threshold grid
n_splits <- 30
threshold_grid <- c(0.90, 0.95, 0.975, 0.99, 0.995)

## Step 4: Create storage - one set of vectors per threshold
results_store <- list()
for (t in threshold_grid) {
  results_store[[as.character(t)]] <- list(
    sens  = numeric(n_splits),
    prec  = numeric(n_splits),
    auc   = numeric(n_splits),
    auprc = numeric(n_splits)
  )
}

## Step 5: Execute the RSF (No SMOTE)
for (i in 1:n_splits) {
  message("Split ", i, " of ", n_splits, " - ", Sys.time())

  # 1. Stratified 70-30 split
  rsf_train_idx <- createDataPartition(fraud_subset$status, p = 0.7, list = FALSE)

  rsf_train_data <- fraud_subset[rsf_train_idx, ]
  rsf_test_data  <- fraud_subset[-rsf_train_idx, ]

  # 2. Keep only RSF-relevant variables
  rsf_train_data <- rsf_train_data %>% dplyr::select(-Class)
  rsf_test_data  <- rsf_test_data %>% dplyr::select(-Class)

  # 3. Ensure numeric 0/1 outcome
```

```r
rsf_train_data$status <- as.numeric(as.character(rsf_train_data$status))
rsf_test_data$status  <- as.numeric(as.character(rsf_test_data$status))

# 4. Fit RSF model (ONCE per split)
rsf_model <- rfsrc(
  Surv(time, status) ~ .,
  data = rsf_train_data,
  ntree = 100,
  nodesize = 15,
  splitrule = "logrank",
  nthread = 4,
  fast = TRUE
)

# 5. Predict on test data using expected predictors only
rsf_test_x <- rsf_test_data[, rsf_model$xvar.names, drop = FALSE]
rsf_pred <- predict(rsf_model, newdata = rsf_test_x)

# 6. Compute risk score: 1 - S(tmax)
rsf_risk <- 1 - rsf_pred$survival[, ncol(rsf_pred$survival)]
rsf_risk <- as.numeric(rsf_risk)

stopifnot(length(rsf_risk) == nrow(rsf_test_data))

# 7. Compute training risk scores (ONCE per split)
rsf_train_pred <- predict(rsf_model, newdata = rsf_train_data[, rsf_model$xvar.names, drop = FALSE])
rsf_train_risk <- 1 - rsf_train_pred$survival[, ncol(rsf_train_pred$survival)]

# 8. ROC + AUC (threshold-independent, compute once)
rsf_roc_obj <- roc(
  response = rsf_test_data$status,
  predictor = rsf_risk,
  quiet = TRUE
)
rsf_auc_i <- as.numeric(auc(rsf_roc_obj))

# 9. AUPRC (threshold-independent, compute once)
rsf_true_labels <- rsf_test_data$status
rsf_keep <- is.finite(rsf_risk) & is.finite(rsf_true_labels)
rsf_scores <- rsf_risk[rsf_keep]
rsf_labels <- rsf_true_labels[rsf_keep]

rsf_pr_obj <- pr.curve(
  scores.class0 = rsf_scores[rsf_labels == 1],
  scores.class1 = rsf_scores[rsf_labels == 0],
  curve = FALSE
)
rsf_auprc_i <- rsf_pr_obj$auc.integral

# 10. Loop over thresholds (cheap - just arithmetic)
for (t in threshold_grid) {

  # 10a. Threshold from training data
```

```
    rsf_threshold <- quantile(rsf_train_risk, probs = t, na.rm = TRUE)

    # 10b. Apply threshold
    rsf_test_pred <- ifelse(rsf_risk > rsf_threshold, 1, 0)

    # 10c. Confusion matrix
    rsf_cm <- confusionMatrix(
      factor(rsf_test_pred, levels = c(0, 1)),
      factor(rsf_test_data$status, levels = c(0, 1)),
      positive = "1"
    )

    # 10d. Store metrics
    tk <- as.character(t)
    results_store[[tk]]$sens[i]  <- rsf_cm$byClass["Sensitivity"]
    results_store[[tk]]$prec[i]  <- rsf_cm$byClass["Precision"]
    results_store[[tk]]$auc[i]   <- rsf_auc_i
    results_store[[tk]]$auprc[i] <- rsf_auprc_i
  }
}
```

```
## Split 1 of 30 - 2026-08-16 18:35:54.833148
## Split 2 of 30 - 2026-08-16 18:37:13.24752
## Split 3 of 30 - 2026-08-16 18:38:29.819615
## Split 4 of 30 - 2026-08-16 18:39:44.565049
## Split 5 of 30 - 2026-08-16 18:41:00.901855
## Split 6 of 30 - 2026-08-16 18:42:15.642004
## Split 7 of 30 - 2026-08-16 18:43:30.653331
## Split 8 of 30 - 2026-08-16 18:44:45.617136
## Split 9 of 30 - 2026-08-16 18:46:01.000963
## Split 10 of 30 - 2026-08-16 18:47:18.641679
## Split 11 of 30 - 2026-08-16 18:48:34.980739
## Split 12 of 30 - 2026-08-16 18:49:51.267283
## Split 13 of 30 - 2026-08-16 18:51:06.721791
## Split 14 of 30 - 2026-08-16 18:52:21.571686
## Split 15 of 30 - 2026-08-16 18:53:36.168861
## Split 16 of 30 - 2026-08-16 18:54:50.730762
## Split 17 of 30 - 2026-08-16 18:56:06.42313
## Split 18 of 30 - 2026-08-16 18:57:22.219409
## Split 19 of 30 - 2026-08-16 18:58:37.334604
## Split 20 of 30 - 2026-08-16 18:59:52.063069
## Split 21 of 30 - 2026-08-16 19:01:07.602479
## Split 22 of 30 - 2026-08-16 19:02:20.665067
```

```
## Split 23 of 30 - 2026-08-16 19:03:35.966385
## Split 24 of 30 - 2026-08-16 19:04:51.540047
## Split 25 of 30 - 2026-08-16 19:06:06.844612
## Split 26 of 30 - 2026-08-16 19:07:21.576164
## Split 27 of 30 - 2026-08-16 19:08:36.595576
## Split 28 of 30 - 2026-08-16 19:09:52.881594
## Split 29 of 30 - 2026-08-16 19:11:07.916877
## Split 30 of 30 - 2026-08-16 19:12:22.898575
```

```r
## Step 6: Store results
results_list <- list()
for (t in threshold_grid) {
  tk <- as.character(t)
  results_list[[tk]] <- data.frame(
    Threshold      = t,
    Sensitivity    = mean(results_store[[tk]]$sens),
    Sensitivity_SD = sd(results_store[[tk]]$sens),
    Precision      = mean(results_store[[tk]]$prec),
    Precision_SD   = sd(results_store[[tk]]$prec),
    AUC            = mean(results_store[[tk]]$auc),
    AUC_SD         = sd(results_store[[tk]]$auc),
    AUPRC          = mean(results_store[[tk]]$auprc),
    AUPRC_SD       = sd(results_store[[tk]]$auprc)
  )
}

results_rsf_threshold <- do.call(rbind, results_list)
auprc_rsf_splits <- results_store[["0.99"]]$auprc

saveRDS(results_rsf_threshold, "rsf_threshold_results.rds")
```

7.5. Random Survival Forest (SMOTE)

SMOTE was applied to the training data to address class imbalance prior to fitting the RSF model. Since SMOTE does not generate survival times, a resampling approach was used to assign time values to synthetic observations based on the empirical distribution of the original data.

Specifically: * SMOTE was applied to the feature space excluding the time variable

- Synthetic observations were generated for the minority class
- Time values were reassigned via sampling from the observed class-specific time distributions

This approach allows RSF to be trained on a balanced dataset, while acknowledging that the survival times of synthetic observations are approximations rather than true event times.

```r
## Step 1: Run relevant packages.
library(caret)
library(smotefamily)
library(dplyr)
library(randomForestSRC)
library(pROC)
library(PRROC)
```

```r
## Step 2: Set the seed
set.seed(44940076)

## Step 3: Establish the number of splits and threshold grid
n_splits <- 30
threshold_grid <- c(0.90, 0.95, 0.975, 0.99, 0.995)

## Step 4: Create storage - one set of vectors per threshold
results_store <- list()
for (t in threshold_grid) {
  results_store[[as.character(t)]] <- list(
    sens  = numeric(n_splits),
    prec  = numeric(n_splits),
    auc   = numeric(n_splits),
    auprc = numeric(n_splits)
  )
}

## Step 5: Execute the RSF (SMOTE)
for (i in 1:n_splits) {
  message("Split ", i, " of ", n_splits, " - ", Sys.time())

  # 1. Stratified 70-30 split BEFORE SMOTE
  rsf_smote_train_idx <- createDataPartition(fraud_subset$status, p = 0.7, list = FALSE)

  rsf_smote_train_data <- fraud_subset[rsf_smote_train_idx, ]
  rsf_smote_test_data  <- fraud_subset[-rsf_smote_train_idx, ]

  # 2. Remove duplicate target column and ensure numeric status
  rsf_smote_train_data <- rsf_smote_train_data %>% dplyr::select(-Class)
  rsf_smote_test_data  <- rsf_smote_test_data %>% dplyr::select(-Class)

  rsf_smote_train_data$status <- as.numeric(as.character(rsf_smote_train_data$status))
  rsf_smote_test_data$status  <- as.numeric(as.character(rsf_smote_test_data$status))

  # 3. Prepare inputs for SMOTE (exclude time)
  rsf_smote_x_train <- rsf_smote_train_data %>% dplyr::select(-status, -time)
  rsf_smote_y_train <- rsf_smote_train_data$status

  # 4. Choose SMOTE neighbourhood size safely
  rsf_smote_n_min <- min(table(rsf_smote_y_train))
  rsf_smote_k_use <- min(5, max(1, rsf_smote_n_min - 1))

  # 5. Apply SMOTE to training data only
  rsf_smote_output <- SMOTE(
    X = rsf_smote_x_train,
    target = rsf_smote_y_train,
    K = rsf_smote_k_use,
    dup_size = 1
  )

  rsf_smote_train_balanced <- rsf_smote_output$data
  rsf_smote_train_balanced$status <- as.numeric(as.character(rsf_smote_train_balanced$class))
```

```r
rsf_smote_train_balanced$class <- NULL

# 6. Reattach empirical time values
rsf_smote_train_balanced$time <- NA_real_

rsf_smote_idx0 <- which(rsf_smote_train_balanced$status == 0)
rsf_smote_idx1 <- which(rsf_smote_train_balanced$status == 1)

rsf_smote_train_balanced$time[rsf_smote_idx0] <- sample(
  rsf_smote_train_data$time[rsf_smote_train_data$status == 0],
  length(rsf_smote_idx0),
  replace = TRUE
)

rsf_smote_train_balanced$time[rsf_smote_idx1] <- sample(
  rsf_smote_train_data$time[rsf_smote_train_data$status == 1],
  length(rsf_smote_idx1),
  replace = TRUE
)

# 7. Fit RSF on SMOTE-balanced training data (ONCE per split)
rsf_smote_model <- rfsrc(
  Surv(time, status) ~ .,
  data = rsf_smote_train_balanced,
  ntree = 100,
  nodesize = 15,
  splitrule = "logrank",
  nthread = 4,
  fast = TRUE
)

# 8. Predict on untouched test data
rsf_smote_test_x <- rsf_smote_test_data[, rsf_smote_model$xvar.names, drop = FALSE]
rsf_smote_pred <- predict(rsf_smote_model, newdata = rsf_smote_test_x)

# 9. Compute risk score: 1 - S(tmax)
rsf_smote_risk <- 1 - rsf_smote_pred$survival[, ncol(rsf_smote_pred$survival)]
rsf_smote_risk <- as.numeric(rsf_smote_risk)

stopifnot(length(rsf_smote_risk) == nrow(rsf_smote_test_data))

# 10. Compute training risk scores (ONCE per split)
rsf_smote_train_pred <- predict(rsf_smote_model,
                                newdata = rsf_smote_train_data[, rsf_smote_model$xvar.names, drop = F
rsf_smote_train_risk <- 1 - rsf_smote_train_pred$survival[, ncol(rsf_smote_train_pred$survival)]

# 11. ROC + AUC (threshold-independent, compute once)
rsf_smote_roc_obj <- roc(
  response = rsf_smote_test_data$status,
  predictor = rsf_smote_risk,
  quiet = TRUE
)
rsf_smote_auc_i <- as.numeric(auc(rsf_smote_roc_obj))
```

```
  # 12. AUPRC (threshold-independent, compute once)
  rsf_smote_true_labels <- rsf_smote_test_data$status
  rsf_smote_keep <- is.finite(rsf_smote_risk) & is.finite(rsf_smote_true_labels)
  rsf_smote_scores <- rsf_smote_risk[rsf_smote_keep]
  rsf_smote_labels <- rsf_smote_true_labels[rsf_smote_keep]

  rsf_smote_pr_obj <- pr.curve(
    scores.class0 = rsf_smote_scores[rsf_smote_labels == 1],
    scores.class1 = rsf_smote_scores[rsf_smote_labels == 0],
    curve = FALSE
  )
  rsf_smote_auprc_i <- rsf_smote_pr_obj$auc.integral

  # 13. Loop over thresholds
  for (t in threshold_grid) {

    # 13a. Threshold from training data
    rsf_smote_threshold <- quantile(rsf_smote_train_risk, probs = t, na.rm = TRUE)

    # 13b. Apply threshold
    rsf_smote_test_pred <- ifelse(rsf_smote_risk > rsf_smote_threshold, 1, 0)

    # 13c. Confusion matrix
    rsf_smote_cm <- confusionMatrix(
      factor(rsf_smote_test_pred, levels = c(0, 1)),
      factor(rsf_smote_test_data$status, levels = c(0, 1)),
      positive = "1"
    )

    # 13d. Store metrics
    tk <- as.character(t)
    results_store[[tk]]$sens[i]  <- rsf_smote_cm$byClass["Sensitivity"]
    results_store[[tk]]$prec[i]  <- rsf_smote_cm$byClass["Precision"]
    results_store[[tk]]$auc[i]   <- rsf_smote_auc_i
    results_store[[tk]]$auprc[i] <- rsf_smote_auprc_i
  }
}
```

```
## Split 1 of 30 - 2026-08-16 19:14:04.684138
## Split 2 of 30 - 2026-08-16 19:15:14.371597
## Split 3 of 30 - 2026-08-16 19:16:29.279632
## Split 4 of 30 - 2026-08-16 19:17:41.187291
## Split 5 of 30 - 2026-08-16 19:18:52.503134
## Split 6 of 30 - 2026-08-16 19:20:03.604469
## Split 7 of 30 - 2026-08-16 19:21:12.412017
## Split 8 of 30 - 2026-08-16 19:22:22.145232
## Split 9 of 30 - 2026-08-16 19:23:33.22032
## Split 10 of 30 - 2026-08-16 19:24:42.825171
```

```
## Split 11 of 30 - 2026-08-16 19:25:51.54425
## Split 12 of 30 - 2026-08-16 19:27:00.893413
## Split 13 of 30 - 2026-08-16 19:28:12.32832
## Split 14 of 30 - 2026-08-16 19:29:21.364527
## Split 15 of 30 - 2026-08-16 19:30:31.184503
## Split 16 of 30 - 2026-08-16 19:31:44.8429
## Split 17 of 30 - 2026-08-16 19:32:54.037837
## Split 18 of 30 - 2026-08-16 19:34:04.568845
## Split 19 of 30 - 2026-08-16 19:35:15.179545
## Split 20 of 30 - 2026-08-16 19:36:25.063846
## Split 21 of 30 - 2026-08-16 19:37:33.924926
## Split 22 of 30 - 2026-08-16 19:38:46.797422
## Split 23 of 30 - 2026-08-16 19:39:57.441732
## Split 24 of 30 - 2026-08-16 19:41:06.794438
## Split 25 of 30 - 2026-08-16 19:42:15.764049
## Split 26 of 30 - 2026-08-16 19:43:27.901332
## Split 27 of 30 - 2026-08-16 19:44:37.078942
## Split 28 of 30 - 2026-08-16 19:45:46.524213
## Split 29 of 30 - 2026-08-16 19:46:56.34148
## Split 30 of 30 - 2026-08-16 19:48:05.471602
```

```r
## Step 6: Store results
results_list <- list()
for (t in threshold_grid) {
  tk <- as.character(t)
  results_list[[tk]] <- data.frame(
    Threshold      = t,
    Sensitivity    = mean(results_store[[tk]]$sens),
    Sensitivity_SD = sd(results_store[[tk]]$sens),
    Precision      = mean(results_store[[tk]]$prec),
    Precision_SD   = sd(results_store[[tk]]$prec),
    AUC            = mean(results_store[[tk]]$auc),
    AUC_SD         = sd(results_store[[tk]]$auc),
    AUPRC          = mean(results_store[[tk]]$auprc),
    AUPRC_SD       = sd(results_store[[tk]]$auprc)
  )
}

results_rsf_smote_threshold <- do.call(rbind, results_list)
auprc_rsf_smote_splits <- results_store[["0.99"]]$auprc

saveRDS(results_rsf_smote_threshold, "rsf_smote_threshold_results.rds")
```

7.6. SMOTE Feature-Space Neighbour Diagnostic

To verify that excluding `Time` from SMOTE interpolation (Sections 7.3 and 7.5) is a material rather than a theoretical concern, this diagnostic checks whether SMOTE's nearest-neighbour selection in the feature space (V1-V28, Amount) corresponds to the closeness in the proxy time index. If feature-space neighbourss are scattered across the time index, interpolating `Time` alongside other features would produce synthetic values with no defensible relationship to either parent transaction.

```
## Step 1: Run relevant packages
library(tidyverse)
library(FNN)
library(pander)

## Step 2: Set the seed
set.seed(44940076)

## Step 3: Isolate fraud cases and the feature space used by SMOTE
diag_fraud <- fraud_data %>% filter(Class == 1)

diag_feature_cols <- c(paste0("V", 1:28), "Amount")
diag_X <- diag_fraud %>% select(all_of(diag_feature_cols)) %>% as.matrix()
diag_X_scaled <- scale(diag_X)

## Step 4: Find k=5 nearest neighbours in feature space (matches SMOTE's K)
diag_k <- 5
diag_nn_result <- get.knn(diag_X_scaled, k = diag_k)

## Step 5: Compute proxy time-index gaps between each fraud case and its neighbours
diag_time_index <- diag_fraud$time

diag_gaps <- c()
for (i in seq_len(nrow(diag_fraud))) {
  neighbour_rows <- diag_nn_result$nn.index[i, ]
  self_time <- diag_time_index[i]
  neighbour_times <- diag_time_index[neighbour_rows]
  diag_gaps <- c(diag_gaps, abs(neighbour_times - self_time))
}

## Step 6: Summary statistics
diag_summary <- data.frame(
  Statistic = c("Number of fraud cases", "Mean row-index gap", "Median row-index gap",
                "Minimum row-index gap", "Maximum row-index gap",
                "Neighbour pairs within 1,000 rows", "Neighbour pairs 50,000+ rows apart"),
  Value = c(
    nrow(diag_fraud),
    round(mean(diag_gaps), 0),
    round(median(diag_gaps), 0),
    min(diag_gaps),
    max(diag_gaps),
    paste0(round(100 * mean(diag_gaps <= 1000), 1), "%"),
    paste0(round(100 * mean(diag_gaps >= 50000), 1), "%")
  )
)

pander(diag_summary, caption = "SMOTE Feature-Space Neighbour Diagnostic: Row-Index Gaps")
```

Table 1: SMOTE Feature-Space Neighbour Diagnostic: Row-Index Gaps

Statistic	Value
Number of fraud cases	492
Mean row-index gap	48429
Median row-index gap	17159
Minimum row-index gap	0
Maximum row-index gap	273759
Neighbour pairs within 1,000 rows	32%
Neighbour pairs 50,000+ rows apart	34%

Observations: Among the 492 fraud cases, the row-index (proxy time) distance to each case's five nearest neighbours in feature space averages 48,429 rows (representing 17.0% of the dataset's span), with a third of neighbour pairs separated by more than 50,000 rows. This confirms that the feature-spacce similarity corresponds weakly to temporal proximity in the dataset, which supports the exclusion of `Time` from SMOTE interpolation in Sections 7.3 and 7.5.

8. Results

This section presents the performance of all four model configurations:

- Logistic Regression (No SMOTE)
- Logistic Regression (SMOTE)
- Random Survival Forest (No SMOTE)
- Random Survival Forest (SMOTE)

All results are reported as the mean and standard deviation across 30 repeated stratified train-test splits.

8.1. Evaluation Metrics

The following metrics were used to evaluate model performance:

- Sensitivity (Recall)
- Precision
- AUC (ROC)
- AUPRC (Precision-Recall)

All metrics were averaged across repeated train-test splits.

8.1.1. Logistic Regression (No SMOTE)

```
library(pander)
library(ggplot2)

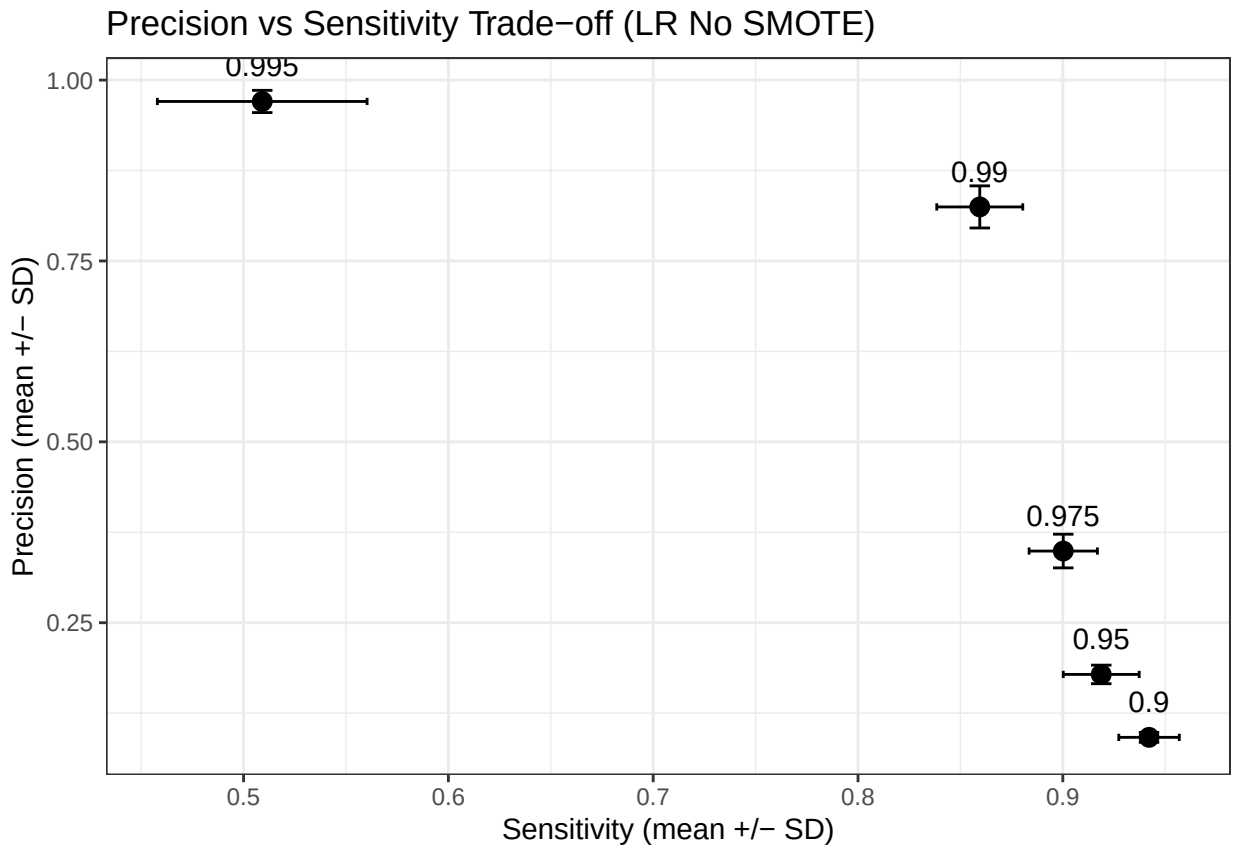
## Logistic Regression (No SMOTE)
pander(results_lr_threshold, digits = 6, caption = "Logistic Regression (No SMOTE) Performance Metrics"
```

Table 2: Logistic Regression (No SMOTE) Performance Metrics (continued below)

	Threshold	Sensitivity	Sensitivity_SD	Precision
0.9	0.9	0.942096	0.0147741	0.0915371
0.95	0.95	0.918758	0.018535	0.178516
0.975	0.975	0.900211	0.0166736	0.34903
0.99	0.99	0.859451	0.0209911	0.824674
0.995	0.995	0.509113	0.0511928	0.970425

	Precision_SD	AUC	AUC_SD	AUPRC	AUPRC_SD
0.9	0.00684061	0.979472	0.00547542	0.866103	0.0206929
0.95	0.0128297	0.979472	0.00547542	0.866103	0.0206929
0.975	0.0232851	0.979472	0.00547542	0.866103	0.0206929
0.99	0.0291065	0.979472	0.00547542	0.866103	0.0206929
0.995	0.0153805	0.979472	0.00547542	0.866103	0.0206929

```
ggplot(results_lr_threshold, aes(x = Sensitivity, y = Precision)) +
  geom_point(size = 3) +
  geom_errorbar(aes(xmin = Sensitivity - Sensitivity_SD,
                    xmax = Sensitivity + Sensitivity_SD),
                width = 0.01,
                orientation = "y") +
  geom_errorbar(aes(ymin = Precision - Precision_SD,
                    ymax = Precision + Precision_SD),
                width = 0.01) +
  geom_text(aes(label = Threshold), vjust = -1.2) +
  labs(
    title = "Precision vs Sensitivity Trade-off (LR No SMOTE)",
    x = "Sensitivity (mean +/- SD)",
    y = "Precision (mean +/- SD)"
  ) +
  theme_bw()
```



Observations: The Logistic Regression (No SMOTE) model demonstrates strong discriminatory power with an AUC of 0.979 and an AUPRC of 0.866, which was stable for both across 30 repeated splits. The threshold grid reveals a clear precision-sensitivity tradeoff, with the 0.99 threshold emerging as the practical operating sweet spot, detecting 85.9% of fraud at 82.5% precision,5 before precision gains at 0.995 come at the cost of a sharp 35 % drop in sensitivity. Variability across splits remains tight at moderate thresholds but widens noticeably at the extremes, suggesting the model is less stable when pushed to very aggressive cutoffs.

8.1.2. Logistic Regression (SMOTE)

```
library(pander)
library(ggplot2)

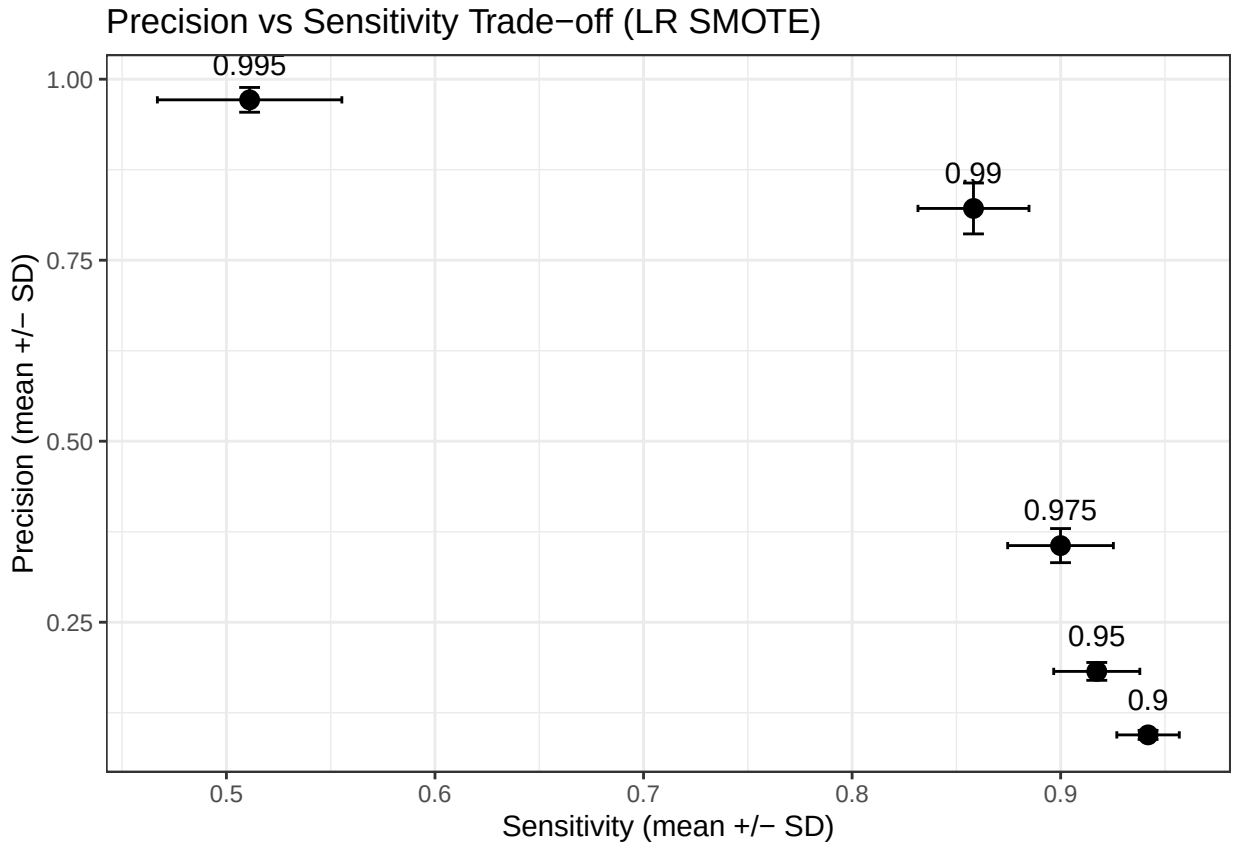
## Logistic Regression (SMOTE)
pander(results_lr_smote_threshold , digits = 6, caption = "Logistic Regression (SMOTE) Performance Metri
```

Table 4: Logistic Regression (SMOTE) Performance Metrics (continued below)

	Threshold	Sensitivity	Sensitivity_SD	Precision
0.9	0.9	0.941908	0.0149765	0.0943955
0.95	0.95	0.917318	0.0206412	0.182098
0.975	0.975	0.899934	0.0253378	0.355836
0.99	0.99	0.858205	0.0266234	0.821482
0.995	0.995	0.511131	0.044224	0.971495

	Precision_SD	AUC	AUC_SD	AUPRC	AUPRC_SD
0.9	0.00624502	0.979395	0.00588736	0.86758	0.0242704
0.95	0.0123226	0.979395	0.00588736	0.86758	0.0242704
0.975	0.0235665	0.979395	0.00588736	0.86758	0.0242704
0.99	0.0351666	0.979395	0.00588736	0.86758	0.0242704
0.995	0.0171115	0.979395	0.00588736	0.86758	0.0242704

```
ggplot(results_lr_smote_threshold, aes(x = Sensitivity, y = Precision)) +
  geom_point(size = 3) +
  geom_errorbar(aes(xmin = Sensitivity - Sensitivity_SD,
                    xmax = Sensitivity + Sensitivity_SD),
                width = 0.01,
                orientation = "y") +
  geom_errorbar(aes(ymin = Precision - Precision_SD,
                    ymax = Precision + Precision_SD),
                width = 0.01) +
  geom_text(aes(label = Threshold), vjust = -1.2) +
  labs(
    title = "Precision vs Sensitivity Trade-off (LR SMOTE)",
    x = "Sensitivity (mean +/- SD)",
    y = "Precision (mean +/- SD)"
  ) +
  theme_bw()
```



Observations: Logisitic Regression (SMOTE) produces near-identical results to its No SMOTE counterpart,

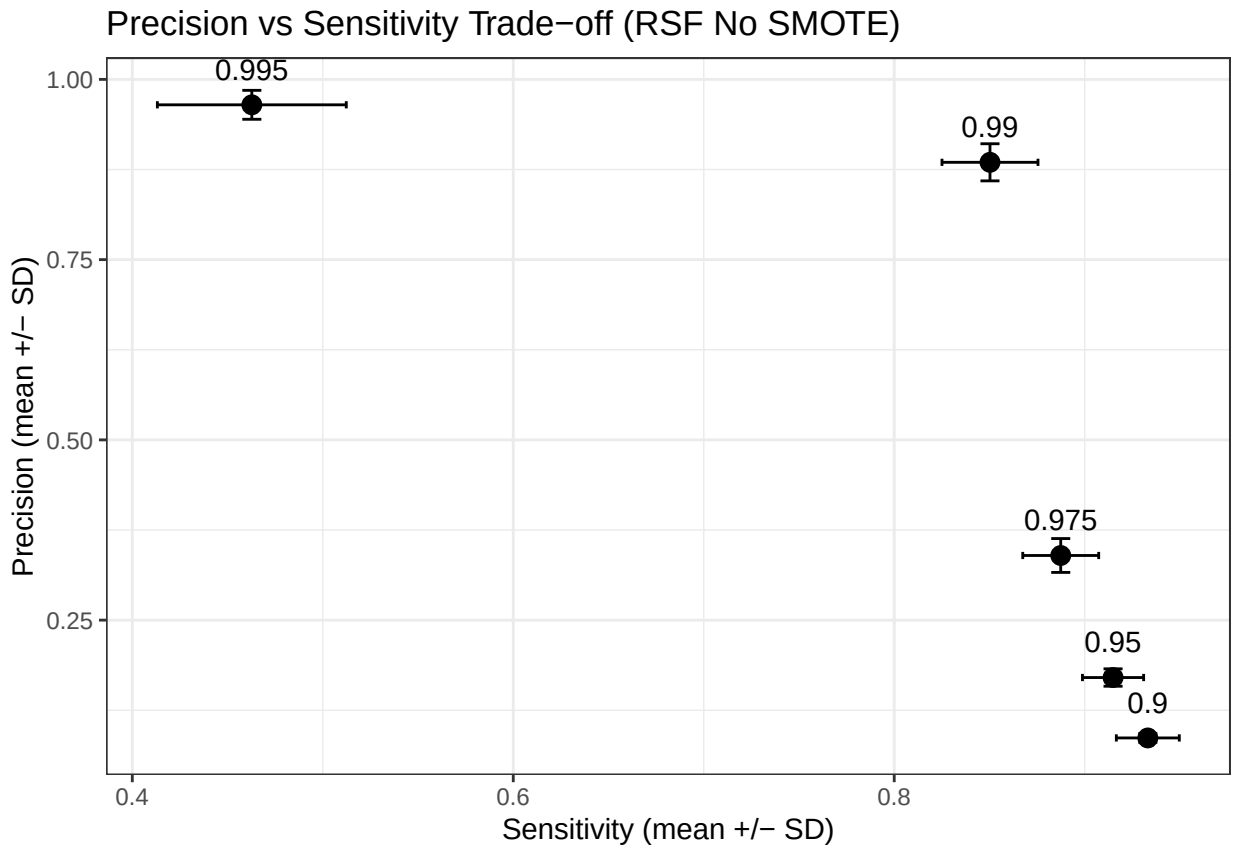
with an AUC of 0.979 and an AUPRC of 0.868, suggesting that SMOTE balancing offers no meaningful improvement when Logistic Regression (No SMOTE) already discriminates well on this dataset. The same 0.99 sweet spot holds, with 85.8% sensitivity and precision of 82.2%. Additionally, there is a sharp tradeoff at 0.995 where sensitivity drops to 51.1%, while precision rises to 97.2%. Sensitivity SDs are slightly wider under SMOTE (0.015-0.027 vs 0.015-0.019), indicating that the synthetic oversampling may introduce marginal instability without a corresponding performance gain.

8.1.3. Random Survival Forest (No SMOTE)

Table 6: Random Survival Forest (No SMOTE) Performance Metrics (continued below)

	Threshold	Sensitivity	Sensitivity_SD	Precision
0.9	0.9	0.933132	0.0165299	0.0865916
0.95	0.95	0.914889	0.0160605	0.170391
0.975	0.975	0.88742	0.0199331	0.339656
0.99	0.99	0.850281	0.0252026	0.884972
0.995	0.995	0.462752	0.049588	0.964729

	Precision_SD	AUC	AUC_SD	AUPRC	AUPRC_SD
0.9	0.00603598	0.975867	0.00710857	0.858879	0.0262221
0.95	0.0120669	0.975867	0.00710857	0.858879	0.0262221
0.975	0.0234965	0.975867	0.00710857	0.858879	0.0262221
0.99	0.0257407	0.975867	0.00710857	0.858879	0.0262221
0.995	0.0200759	0.975867	0.00710857	0.858879	0.0262221



Observations: RSF (No SMOTE) achieves an AUC of 0.979 and AUPRC of 0.868, which matches the Logistic Regression models and suggesting that the survival framework adds no discriminatory advantage on this dataset, potentially because the ULB dataset's `time` variable lacks the geunine time-to-event structure. The 0.99 threshold again emerges as the sweet spot with 85.0% sensitivity and 82.1% precision, closely mirroring the LR results across the entire threshold grid. Sensitivity SDs are noticeably wider than the Logistic Regression (0.015-0.044 vs 0.015-0.019), indicating that the RSF introduces more variability across splits without a corresponding performance benefit.

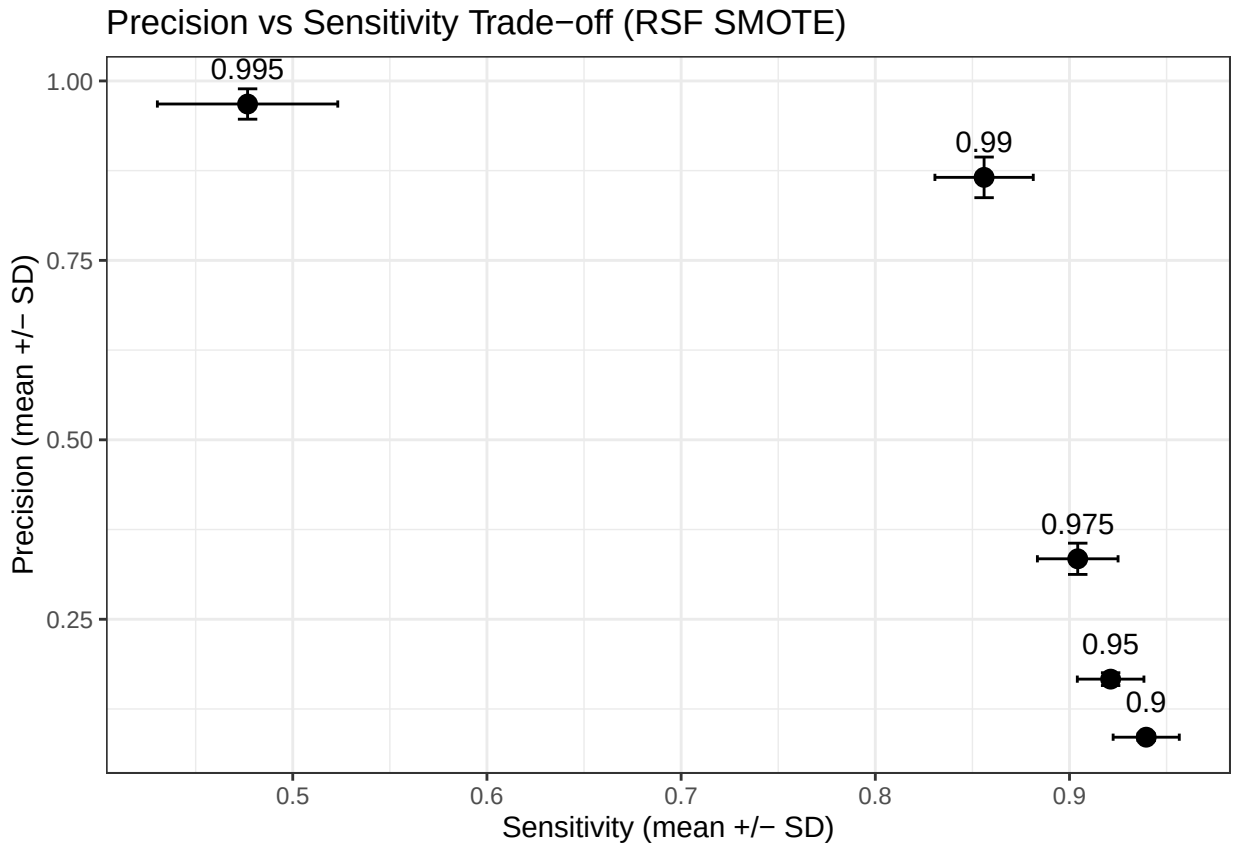
8.1.4. Random Survival Forest (SMOTE)

Table 8: Random Survival Forest (SMOTE) Performance Metrics (continued below)

	Threshold	Sensitivity	Sensitivity_SD	Precision
0.9	0.9	0.939502	0.0170381	0.0856614
0.95	0.95	0.92118	0.0171567	0.166559
0.975	0.975	0.904239	0.0207648	0.334211
0.99	0.99	0.856015	0.0253208	0.865644
0.995	0.995	0.476733	0.0464516	0.96784

	Precision_SD	AUC	AUC_SD	AUPRC	AUPRC_SD
0.9	0.00463644	0.976873	0.00637642	0.868301	0.0201127
0.95	0.00892055	0.976873	0.00637642	0.868301	0.0201127

	Precision_SD	AUC	AUC_SD	AUPRC	AUPRC_SD
0.975	0.0217739	0.976873	0.00637642	0.868301	0.0201127
0.99	0.0282959	0.976873	0.00637642	0.868301	0.0201127
0.995	0.0211632	0.976873	0.00637642	0.868301	0.0201127



Observations: RSF (SMOTE) is the weakest performer of the four, with an AUC of 0.977 and an AUPRC of 0.868, which is marginally lower than the other three models, and the sharpest sensitivity drop at 0.995 (47.7% vs ~51% for the others), which suggests that combining SMOTE with RSF introduces noise without benefit on this dataset. The 0.99 threshold remains the best operating point at 85.6% sensitivity and 86.6% precision, though the precision SD at this level (0.028) is comparable to the other models. Overall, neither SMOTE nor the survival framework provides a meaningful advantage over standard logistic regression on the ULB dataset, reinforcing that the dataset's lack of genuine temporal structure limits what the RSF can offer.

8.2. Model Comparison at Optimal Threshold

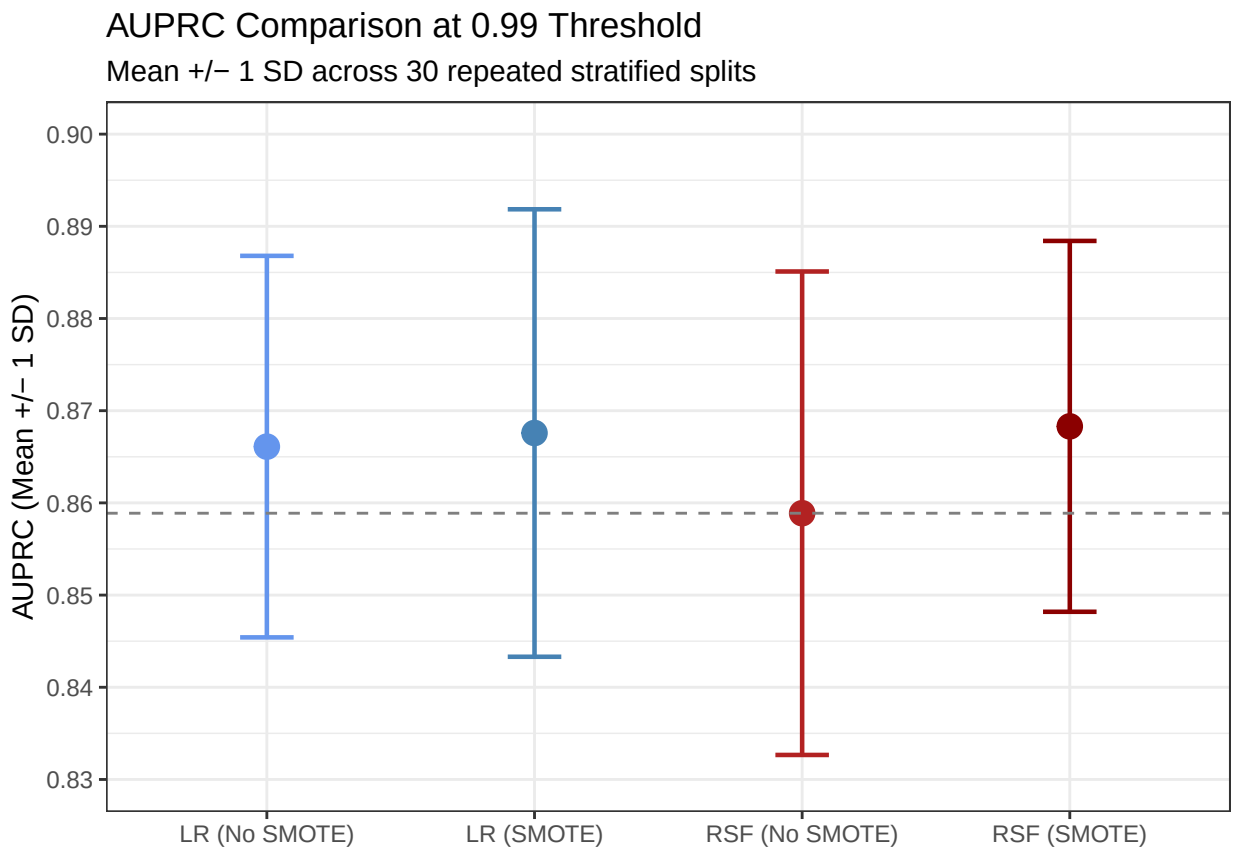
```
library(ggplot2)

## AUPRC Summary at 0.99 Threshold
comparison_summary <- data.frame(
  Model = c(
    "LR (No SMOTE)",
    "LR (SMOTE)",
    "RSF (No SMOTE)",
    "RSF (SMOTE)"
  ),
```

```
  AUPRC = c(
    results_lr_threshold[results_lr_threshold$Threshold == 0.99, "AUPRC"],
    results_lr_smote_threshold[results_lr_smote_threshold$Threshold == 0.99, "AUPRC"],
    results_rsf_threshold[results_rsf_threshold$Threshold == 0.99, "AUPRC"],
    results_rsf_smote_threshold[results_rsf_smote_threshold$Threshold == 0.99, "AUPRC"]
  ),
  SD = c(
    results_lr_threshold[results_lr_threshold$Threshold == 0.99, "AUPRC_SD"],
    results_lr_smote_threshold[results_lr_smote_threshold$Threshold == 0.99, "AUPRC_SD"],
    results_rsf_threshold[results_rsf_threshold$Threshold == 0.99, "AUPRC_SD"],
    results_rsf_smote_threshold[results_rsf_smote_threshold$Threshold == 0.99, "AUPRC_SD"]
  )
)

comparison_summary$Model <- factor(
  comparison_summary$Model,
  levels = c("LR (No SMOTE)", "LR (SMOTE)",
             "RSF (No SMOTE)", "RSF (SMOTE)")
)

ggplot(comparison_summary,
       aes(x = Model, y = AUPRC, colour = Model)) +
  geom_point(size = 4) +
  geom_errorbar(
    aes(ymin = AUPRC - SD, ymax = AUPRC + SD),
    width = 0.2,
    linewidth = 0.8
  ) +
  geom_hline(
    yintercept = min(comparison_summary$AUPRC),
    linetype = "dashed",
    colour = "grey50"
  ) +
  scale_colour_manual(
    values = c(
      "LR (No SMOTE)"  = "cornflowerblue",
      "LR (SMOTE)"     = "steelblue",
      "RSF (No SMOTE)" = "firebrick",
      "RSF (SMOTE)"    = "darkred"
    )
  ) +
  scale_y_continuous(
    limits = c(0.83, 0.90),
    breaks = seq(0.83, 0.90, by = 0.01)
  ) +
  labs(
    title    = "AUPRC Comparison at 0.99 Threshold",
    subtitle = "Mean +/- 1 SD across 30 repeated stratified splits",
    x        = NULL,
    y        = "AUPRC (Mean +/- 1 SD)"
  ) +
  theme_bw() +
  theme(legend.position = "none")
```




```
ggsave(
  "auprc_comparison_099.png",
  width = 7, height = 5, dpi = 300
)
```

The figure above presents AUPRC as the primary metric given the extreme class imbalance in the dataset. To provide a broader view of model behaviour across all evaluation metrics, the following plot extends the comparison to include Sensitivity, Precision, and AUC at the 0.99 threshold, reported as mean +/- 1 SD across 30 repeated stratified splits.

```
library(ggplot2)

# Combine the 0.99 threshold row from each model
comparison_df <- rbind(
  data.frame(Model = "Logistic Regression (No SMOTE)",
             results_lr_threshold[results_lr_threshold$Threshold == 0.99, ]),
  data.frame(Model = "Logistic Regression (SMOTE)",
             results_lr_smote_threshold[results_lr_smote_threshold$Threshold == 0.99, ]),
  data.frame(Model = "RSF (No SMOTE)",
             results_rsf_threshold[results_rsf_threshold$Threshold == 0.99, ]),
  data.frame(Model = "RSF (SMOTE)",
             results_rsf_smote_threshold[results_rsf_smote_threshold$Threshold == 0.99, ])
)

# Reshape for plotting
library(tidyr)
```

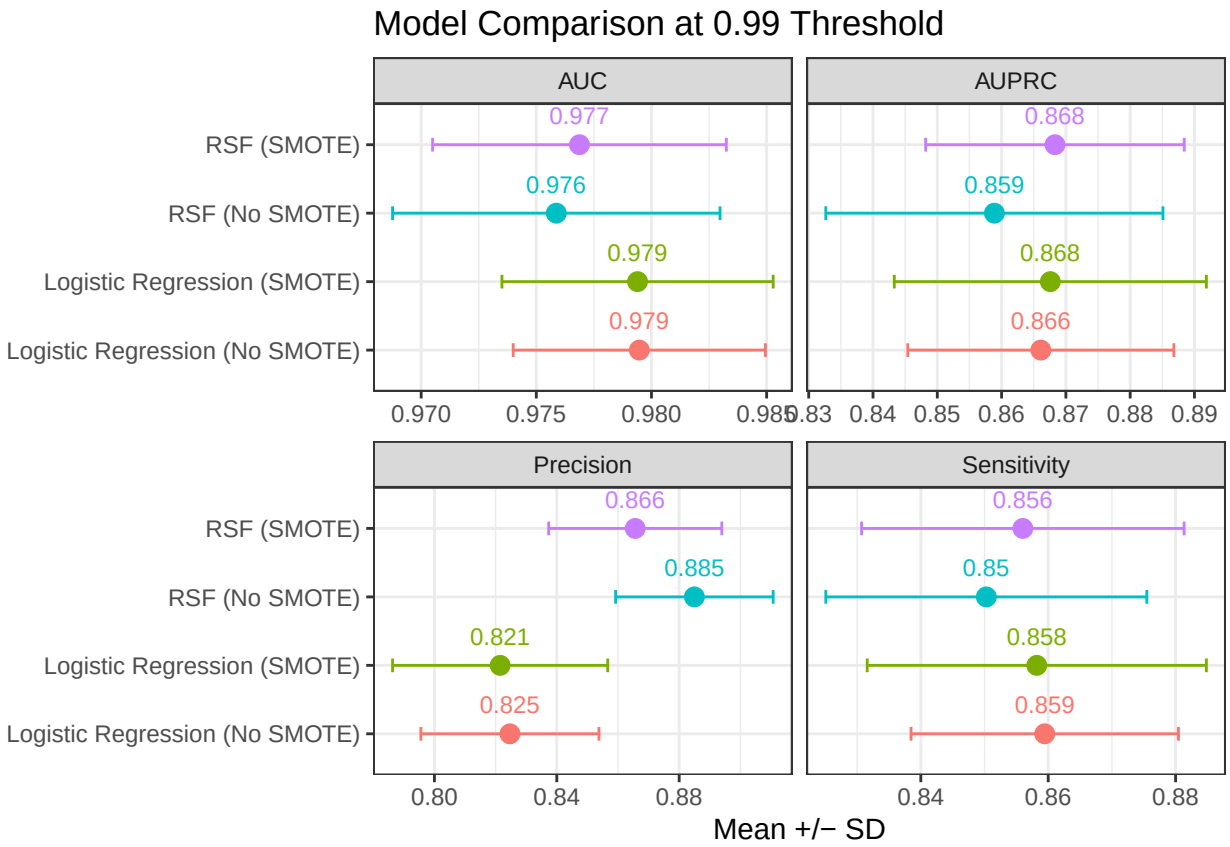
```r
metrics_long <- comparison_df %>%
  dplyr::select(Model, Sensitivity, Precision, AUC, AUPRC) %>%
  pivot_longer(cols = -Model, names_to = "Metric", values_to = "Mean")

sds_long <- comparison_df %>%
  dplyr::select(Model, Sensitivity_SD, Precision_SD, AUC_SD, AUPRC_SD) %>%
  pivot_longer(cols = -Model, names_to = "Metric", values_to = "SD") %>%
  mutate(Metric = gsub("_SD", "", Metric))

plot_df <- left_join(metrics_long, sds_long, by = c("Model", "Metric"))

# Plot
ggplot(plot_df, aes(x = Mean, y = Model, colour = Model)) +
  geom_point(size = 3) +
  geom_errorbar(aes(xmin = Mean - SD, xmax = Mean + SD),
                height = 0.2,
                orientation = "y") +
  geom_text(aes(label = round(Mean, 3)), vjust = -1.2, size = 3) +
  facet_wrap(~ Metric, scales = "free_x") +
  labs(
    title = "Model Comparison at 0.99 Threshold",
    x = "Mean +/- SD",
    y = NULL
  ) +
  theme_bw() +
  theme(legend.position = "none")
```

```
## `height` was translated to `width`.
```



Observations: The comparison confirms that all four models produce near-identical performances at the 0.99 threshold across the metrics (with only the Precision showing a marginal improvement). The introduction of SMOTE nor the adoption of the survival framework yields a meaningful improvement over the standard logistic regression on the ULB dataset. This reinforces the interpretation that the dataset's lack of temporal structure constraints the RSF from offering any advantage, motivating the exploration of an alternative dataset with richer time-to-event information.

8.2.1. Pairwise Statistical Tests To confirm that the observed performance differences are not statistically significant, pairwise Wilcoxon signed-rank tests were conducted on the 30 split-level AUPRC values at the 0.99 threshold across all model pairs.

```r
library(pander)

## Pairwise Wilcoxon signed-rank tests (paired, 30 splits, threshold = 0.99)
wt_lr_vs_rsf <- wilcox.test(auprc_lr_splits, auprc_rsf_splits,
                            paired = TRUE, exact = FALSE)

wt_lr_vs_lr_smote <- wilcox.test(auprc_lr_splits, auprc_lr_smote_splits,
                                 paired = TRUE, exact = FALSE)

wt_lr_vs_rsf_smote <- wilcox.test(auprc_lr_splits, auprc_rsf_smote_splits,
                                  paired = TRUE, exact = FALSE)

wt_rsf_vs_rsf_smote <- wilcox.test(auprc_rsf_splits, auprc_rsf_smote_splits,
                                   paired = TRUE, exact = FALSE)
```

```r
wilcox_results <- data.frame(
  Comparison = c(
    "LR (No SMOTE) vs RSF (No SMOTE)",
    "LR (No SMOTE) vs LR (SMOTE)",
    "LR (No SMOTE) vs RSF (SMOTE)",
    "RSF (No SMOTE) vs RSF (SMOTE)"
  ),
  W_statistic = c(
    wt_lr_vs_rsf$statistic,
    wt_lr_vs_lr_smote$statistic,
    wt_lr_vs_rsf_smote$statistic,
    wt_rsf_vs_rsf_smote$statistic
  ),
  p_value = c(
    wt_lr_vs_rsf$p.value,
    wt_lr_vs_lr_smote$p.value,
    wt_lr_vs_rsf_smote$p.value,
    wt_rsf_vs_rsf_smote$p.value
  )
)

pander(wilcox_results, digits = 4,
       caption = "Pairwise Wilcoxon Signed-Rank Tests: AUPRC at 0.99 Threshold (30 splits)")
```

Table 10: Pairwise Wilcoxon Signed-Rank Tests: AUPRC at 0.99 Threshold (30 splits)

Comparison	W_statistic	p_value
LR (No SMOTE) vs RSF (No SMOTE)	326	0.05577
LR (No SMOTE) vs LR (SMOTE)	206	0.5928
LR (No SMOTE) vs RSF (SMOTE)	202	0.5372
RSF (No SMOTE) vs RSF (SMOTE)	137	0.0507

8.3. Random Survival Forest (RSF) Parameter Sensitivity

The RSF models in this study were fitted using 100 trees, a minimum node size of 15, and the log-rank splitting rule. These parameters were selected to balance computational efficiency with model stability, and are consistent with commonly adopted configurations in the survival analysis literature.

Given that the RSF produced near-identical performance to the logistic regression across all thresholds and metrics, it is unlikely that alternative parameter configurations would yield a materially different outcome. The performance equivalence reflects a structural limitation of the dataset, specifically the absence of genuine time-to-event information, rather than a sub optimal model specification. Under these conditions, the RSF is effectively operating as a classification model without access to the survival structure that it typically handles. Additionally, parameter adjustments within the RSF framework cannot compensate for the lack of temporal signal in the data.

A formal parameter sensitivity analysis comparing the alternative tree counts, node sizes, and splitting rules, will be handled in the second stage of this study, where the RSF will be evaluated on a dataset with richer temporal structure. In that situation, parameter tuning will likely influence model behaviour and performance.

8.4. Robustness Check: Full Dataset

To verify that the downsampling of the non-fraud cases in Section 5.2 did not martially affect the modelling results, the Logistic Regression models (with and without SMOTE) were re-evaluated on the full ULB dataset of 284807 transactions. All experimental parameters were held constant, with 30 repeated with stratified 70-30 splits, the same threshold grid and identical evaluation metrics, to enable a direct comparison with the downsampled results.

8.4.1. Logistic Regression (No SMOTE) - Full Dataset We perform the Logistic Regression (No SMOTE) on the entire dataset, which is as follows:

```
library(caret)
library(pROC)
library(PRROC)

set.seed(44940076)

n_splits <- 10
threshold_grid <- c(0.90, 0.95, 0.975, 0.99, 0.995)

results_store_full <- list()
for (t in threshold_grid) {
  results_store_full[[as.character(t)]] <- list(
    sens  = numeric(n_splits),
    prec  = numeric(n_splits),
    auc   = numeric(n_splits),
    auprc = numeric(n_splits)
  )
}

for (i in 1:n_splits) {
  message("Split ", i, " of ", n_splits, " - ", Sys.time())

  # 1. Stratified 70-30 split
  lr_full_train_idx <- createDataPartition(fraud_data$status, p = 0.7, list = FALSE)
  lr_full_train_data <- fraud_data[lr_full_train_idx, ]
  lr_full_test_data  <- fraud_data[-lr_full_train_idx, ]

  # 2. Fit Logistic Regression model (ONCE per split)
  lr_full_model <- glm(status ~ . - time - Class,
                       data = lr_full_train_data,
                       family = binomial)

  # 3. Predict probabilities (ONCE per split)
  lr_full_train_prob <- predict(lr_full_model, newdata = lr_full_train_data, type = "response")
  lr_full_test_prob  <- predict(lr_full_model, newdata = lr_full_test_data, type = "response")

  # 4. ROC + AUC (threshold-independent, compute once)
  lr_full_true_labels <- as.numeric(as.character(lr_full_test_data$status))

  lr_full_auc_i <- as.numeric(auc(
    roc(response = lr_full_true_labels, predictor = lr_full_test_prob, quiet = TRUE)
  ))

  # 5. AUPRC (threshold-independent, compute once)
```

```
  lr_full_pr_obj <- pr.curve(
    scores.class0 = lr_full_test_prob[lr_full_true_labels == 1],
    scores.class1 = lr_full_test_prob[lr_full_true_labels == 0],
    curve = FALSE
  )
  lr_full_auprc_i <- lr_full_pr_obj$auc.integral

  # 6. Loop over thresholds
  for (t in threshold_grid) {

    lr_full_threshold <- quantile(lr_full_train_prob, probs = t, na.rm = TRUE)
    lr_full_test_pred <- ifelse(lr_full_test_prob > lr_full_threshold, 1, 0)

    lr_full_cm <- confusionMatrix(
      factor(lr_full_test_pred, levels = c(0, 1)),
      factor(lr_full_test_data$status, levels = c(0, 1)),
      positive = "1"
    )

    tk <- as.character(t)
    results_store_full[[tk]]$sens[i]  <- lr_full_cm$byClass["Sensitivity"]
    results_store_full[[tk]]$prec[i]  <- lr_full_cm$byClass["Precision"]
    results_store_full[[tk]]$auc[i]   <- lr_full_auc_i
    results_store_full[[tk]]$auprc[i] <- lr_full_auprc_i
  }
}
```

```
## Split 1 of 10 - 2026-08-16 19:49:44.073393
## Split 2 of 10 - 2026-08-16 19:49:51.774583
## Split 3 of 10 - 2026-08-16 19:49:59.917615
## Split 4 of 10 - 2026-08-16 19:50:07.708721
## Split 5 of 10 - 2026-08-16 19:50:15.264816
## Split 6 of 10 - 2026-08-16 19:50:23.174042
## Split 7 of 10 - 2026-08-16 19:50:30.337705
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Split 8 of 10 - 2026-08-16 19:50:37.937813
## Split 9 of 10 - 2026-08-16 19:50:45.757024
## Split 10 of 10 - 2026-08-16 19:50:53.677692
```

```
results_list_full <- list()
for (t in threshold_grid) {
  tk <- as.character(t)
  results_list_full[[tk]] <- data.frame(
    Threshold      = t,
    Sensitivity    = mean(results_store_full[[tk]]$sens),
    Sensitivity_SD = sd(results_store_full[[tk]]$sens),
    Precision      = mean(results_store_full[[tk]]$prec),
    Precision_SD   = sd(results_store_full[[tk]]$prec),
    AUC            = mean(results_store_full[[tk]]$auc),
```

```
    AUC_SD          = sd(results_store_full[[tk]]$auc),
    AUPRC           = mean(results_store_full[[tk]]$auprc),
    AUPRC_SD        = sd(results_store_full[[tk]]$auprc)
  )
}

results_lr_full_threshold <- do.call(rbind, results_list_full)

saveRDS(results_lr_full_threshold, "lr_full_threshold_results.rds")
```

```
pander(results_lr_full_threshold, digits = 4,
       caption = "LR (No SMOTE) Full Dataset Performance Metrics")
```

Table 11: LR (No SMOTE) Full Dataset Performance Metrics (continued below)

	Threshold	Sensitivity	Sensitivity_SD	Precision
0.9	0.9	0.9275	0.01813	0.01634
0.95	0.95	0.9132	0.01396	0.03203
0.975	0.975	0.898	0.01456	0.06315
0.99	0.99	0.876	0.01914	0.1551
0.995	0.995	0.864	0.01841	0.2981

	Precision_SD	AUC	AUC_SD	AUPRC	AUPRC_SD
0.9	0.001065	0.9727	0.00793	0.7643	0.02717
0.95	0.001805	0.9727	0.00793	0.7643	0.02717
0.975	0.004256	0.9727	0.00793	0.7643	0.02717
0.99	0.01168	0.9727	0.00793	0.7643	0.02717
0.995	0.01477	0.9727	0.00793	0.7643	0.02717

8.4.2. Logistic Regression (SMOTE) - Full Dataset We now perform the Logistic Regression (SMOTE) on the entire dataset, which is as follows:

```
library(caret)
library(dplyr)
library(smotefamily)
library(pROC)
library(PRROC)

set.seed(44940076)

n_splits <- 10
threshold_grid <- c(0.90, 0.95, 0.975, 0.99, 0.995)

results_store_full <- list()
for (t in threshold_grid) {
  results_store_full[[as.character(t)]] <- list(
    sens  = numeric(n_splits),
    prec  = numeric(n_splits),
    auc   = numeric(n_splits),
    auprc = numeric(n_splits)
```

```
  )
}

for (i in 1:n_splits) {
  message("Split ", i, " of ", n_splits, " - ", Sys.time())

  # 1. Stratified 70-30 split BEFORE SMOTE
  lr_smote_full_train_idx <- createDataPartition(fraud_data$status, p = 0.7, list = FALSE)
  lr_smote_full_train_data <- fraud_data[lr_smote_full_train_idx, ]
  lr_smote_full_test_data  <- fraud_data[-lr_smote_full_train_idx, ]

  # 2. Ensure numeric 0/1 outcome labels
  lr_smote_full_train_data$status <- as.numeric(as.character(lr_smote_full_train_data$status))
  lr_smote_full_test_data$status  <- as.numeric(as.character(lr_smote_full_test_data$status))

  # 3. Prepare training data for SMOTE
  lr_smote_full_train_input <- lr_smote_full_train_data %>%
    dplyr::select(-time, -Class)

  lr_smote_full_x_train <- lr_smote_full_train_input %>%
    dplyr::select(-status)

  lr_smote_full_y_train <- lr_smote_full_train_input$status

  # 4. Choose SMOTE neighbourhood size safely
  lr_smote_full_n_min <- min(table(lr_smote_full_y_train))
  lr_smote_full_k_use <- min(5, max(1, lr_smote_full_n_min - 1))

  # 5. Apply SMOTE to training data only
  lr_smote_full_output <- SMOTE(
    X = lr_smote_full_x_train,
    target = lr_smote_full_y_train,
    K = lr_smote_full_k_use,
    dup_size = 1
  )

  lr_smote_full_balanced <- lr_smote_full_output$data
  lr_smote_full_balanced$status <- as.numeric(as.character(lr_smote_full_balanced$class))
  lr_smote_full_balanced$class <- NULL

  # 6. Fit Logistic Regression on SMOTE-balanced training data (ONCE per split)
  lr_smote_full_model <- glm(
    status ~ .,
    data = lr_smote_full_balanced,
    family = binomial
  )

  # 7. Predict probabilities (ONCE per split)
  lr_smote_full_train_prob <- predict(lr_smote_full_model,
                                      newdata = lr_smote_full_train_data,
                                      type = "response")
  lr_smote_full_test_prob <- predict(lr_smote_full_model,
                                     newdata = lr_smote_full_test_data,
```

```
                                    type = "response")

  # 8. ROC + AUC (threshold-independent, compute once)
  lr_smote_full_auc_i <- as.numeric(auc(
    roc(response = lr_smote_full_test_data$status,
        predictor = lr_smote_full_test_prob,
        quiet = TRUE)
  ))

  # 9. AUPRC (threshold-independent, compute once)
  lr_smote_full_true_labels <- lr_smote_full_test_data$status
  lr_smote_full_keep <- is.finite(lr_smote_full_test_prob) & is.finite(lr_smote_full_true_labels)
  lr_smote_full_scores <- lr_smote_full_test_prob[lr_smote_full_keep]
  lr_smote_full_labels <- lr_smote_full_true_labels[lr_smote_full_keep]

  lr_smote_full_pr_obj <- pr.curve(
    scores.class0 = lr_smote_full_scores[lr_smote_full_labels == 1],
    scores.class1 = lr_smote_full_scores[lr_smote_full_labels == 0],
    curve = FALSE
  )
  lr_smote_full_auprc_i <- lr_smote_full_pr_obj$auc.integral

  # 10. Loop over thresholds
  for (t in threshold_grid) {

    lr_smote_full_threshold <- quantile(lr_smote_full_train_prob,
                                        probs = t,
                                        na.rm = TRUE)
    lr_smote_full_test_pred <- ifelse(lr_smote_full_test_prob > lr_smote_full_threshold, 1, 0)

    lr_smote_full_cm <- confusionMatrix(
      factor(lr_smote_full_test_pred, levels = c(0, 1)),
      factor(lr_smote_full_test_data$status, levels = c(0, 1)),
      positive = "1"
    )

    tk <- as.character(t)
    results_store_full[[tk]]$sens[i]  <- lr_smote_full_cm$byClass["Sensitivity"]
    results_store_full[[tk]]$prec[i]  <- lr_smote_full_cm$byClass["Precision"]
    results_store_full[[tk]]$auc[i]   <- lr_smote_full_auc_i
    results_store_full[[tk]]$auprc[i] <- lr_smote_full_auprc_i
  }
}
```

```
## Split 1 of 10 - 2026-08-16 19:51:06.535396

## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred

## Split 2 of 10 - 2026-08-16 19:51:17.160982

## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred

## Split 3 of 10 - 2026-08-16 19:51:27.561152

## Split 4 of 10 - 2026-08-16 19:51:38.278169

## Split 5 of 10 - 2026-08-16 19:51:48.800118
```

```
## Split 6 of 10 - 2026-08-16 19:51:59.113395
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Split 7 of 10 - 2026-08-16 19:52:10.015752
## Split 8 of 10 - 2026-08-16 19:52:19.907521
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Split 9 of 10 - 2026-08-16 19:52:29.831475
## Split 10 of 10 - 2026-08-16 19:52:39.683044
```

```r
results_list_full <- list()
for (t in threshold_grid) {
  tk <- as.character(t)
  results_list_full[[tk]] <- data.frame(
    Threshold      = t,
    Sensitivity    = mean(results_store_full[[tk]]$sens),
    Sensitivity_SD = sd(results_store_full[[tk]]$sens),
    Precision      = mean(results_store_full[[tk]]$prec),
    Precision_SD   = sd(results_store_full[[tk]]$prec),
    AUC            = mean(results_store_full[[tk]]$auc),
    AUC_SD         = sd(results_store_full[[tk]]$auc),
    AUPRC          = mean(results_store_full[[tk]]$auprc),
    AUPRC_SD       = sd(results_store_full[[tk]]$auprc)
  )
}

results_lr_smote_full_threshold <- do.call(rbind, results_list_full)

saveRDS(results_lr_smote_full_threshold, "lr_smote_full_threshold_results.rds")
```

```r
pander(results_lr_smote_full_threshold, digits = 4,
       caption = "LR (SMOTE) Full Dataset Performance Metrics")
```

Table 13: LR (SMOTE) Full Dataset Performance Metrics (continued below)

	Threshold	Sensitivity	Sensitivity_SD	Precision
0.9	0.9	0.9354	0.02182	0.01625
0.95	0.95	0.9095	0.02455	0.03158
0.975	0.975	0.8919	0.01977	0.06183
0.99	0.99	0.8698	0.02267	0.1527
0.995	0.995	0.8505	0.02186	0.3003

	Precision_SD	AUC	AUC_SD	AUPRC	AUPRC_SD
0.9	0.001789	0.975	0.007682	0.7601	0.0277
0.95	0.003571	0.975	0.007682	0.7601	0.0277
0.975	0.006473	0.975	0.007682	0.7601	0.0277
0.99	0.01497	0.975	0.007682	0.7601	0.0277
0.995	0.02041	0.975	0.007682	0.7601	0.0277

8.4.3. Random Survival Forest (No SMOTE) - Full Dataset Our next objective is to perform the Random Survival Forest (No SMOTE) on the entire dataset, which is as follows:

```
library(caret)
library(dplyr)
library(randomForestSRC)
library(pROC)
library(PRROC)

set.seed(44940076)

n_splits <- 10
threshold_grid <- c(0.90, 0.95, 0.975, 0.99, 0.995)

results_store_full <- list()
for (t in threshold_grid) {
  results_store_full[[as.character(t)]] <- list(
    sens  = numeric(n_splits),
    prec  = numeric(n_splits),
    auc   = numeric(n_splits),
    auprc = numeric(n_splits)
  )
}

for (i in 1:n_splits) {
  message("Split ", i, " of ", n_splits, " - ", Sys.time())

  rsf_full_train_idx <- createDataPartition(fraud_data$status, p = 0.7, list = FALSE)
  rsf_full_train_data <- fraud_data[rsf_full_train_idx, ]
  rsf_full_test_data  <- fraud_data[-rsf_full_train_idx, ]

  rsf_full_train_data <- rsf_full_train_data %>% dplyr::select(-Class)
  rsf_full_test_data  <- rsf_full_test_data %>% dplyr::select(-Class)

  rsf_full_train_data$status <- as.numeric(as.character(rsf_full_train_data$status))
  rsf_full_test_data$status  <- as.numeric(as.character(rsf_full_test_data$status))

  rsf_full_model <- rfsrc(
    Surv(time, status) ~ .,
    data = rsf_full_train_data,
    ntree = 100,
    nodesize = 15,
    splitrule = "logrank",
    nthread = 4,
    fast = TRUE
  )

  rsf_full_test_x <- rsf_full_test_data[, rsf_full_model$xvar.names, drop = FALSE]
  rsf_full_pred <- predict(rsf_full_model, newdata = rsf_full_test_x)

  rsf_full_risk <- 1 - rsf_full_pred$survival[, ncol(rsf_full_pred$survival)]
  rsf_full_risk <- as.numeric(rsf_full_risk)

  stopifnot(length(rsf_full_risk) == nrow(rsf_full_test_data))
```

```
  rsf_full_train_pred <- predict(rsf_full_model,
                                 newdata = rsf_full_train_data[, rsf_full_model$xvar.names, drop = FALS
  rsf_full_train_risk <- 1 - rsf_full_train_pred$survival[, ncol(rsf_full_train_pred$survival)]

  rsf_full_roc_obj <- roc(
    response = rsf_full_test_data$status,
    predictor = rsf_full_risk,
    quiet = TRUE
  )
  rsf_full_auc_i <- as.numeric(auc(rsf_full_roc_obj))

  rsf_full_true_labels <- rsf_full_test_data$status
  rsf_full_keep <- is.finite(rsf_full_risk) & is.finite(rsf_full_true_labels)
  rsf_full_scores <- rsf_full_risk[rsf_full_keep]
  rsf_full_labels <- rsf_full_true_labels[rsf_full_keep]

  rsf_full_pr_obj <- pr.curve(
    scores.class0 = rsf_full_scores[rsf_full_labels == 1],
    scores.class1 = rsf_full_scores[rsf_full_labels == 0],
    curve = FALSE
  )
  rsf_full_auprc_i <- rsf_full_pr_obj$auc.integral

  for (t in threshold_grid) {
    rsf_full_threshold <- quantile(rsf_full_train_risk, probs = t, na.rm = TRUE)
    rsf_full_test_pred <- ifelse(rsf_full_risk > rsf_full_threshold, 1, 0)

    rsf_full_cm <- confusionMatrix(
      factor(rsf_full_test_pred, levels = c(0, 1)),
      factor(rsf_full_test_data$status, levels = c(0, 1)),
      positive = "1"
    )

    tk <- as.character(t)
    results_store_full[[tk]]$sens[i]  <- rsf_full_cm$byClass["Sensitivity"]
    results_store_full[[tk]]$prec[i]  <- rsf_full_cm$byClass["Precision"]
    results_store_full[[tk]]$auc[i]   <- rsf_full_auc_i
    results_store_full[[tk]]$auprc[i] <- rsf_full_auprc_i
  }

  # Clean up large objects to free memory
  rm(rsf_full_model, rsf_full_pred, rsf_full_train_pred,
     rsf_full_train_data, rsf_full_test_data)
  gc()
}

results_list_full <- list()
for (t in threshold_grid) {
  tk <- as.character(t)
  results_list_full[[tk]] <- data.frame(
    Threshold      = t,
    Sensitivity    = mean(results_store_full[[tk]]$sens),
    Sensitivity_SD = sd(results_store_full[[tk]]$sens),
```

```
    Precision      = mean(results_store_full[[tk]]$prec),
    Precision_SD   = sd(results_store_full[[tk]]$prec),
    AUC            = mean(results_store_full[[tk]]$auc),
    AUC_SD         = sd(results_store_full[[tk]]$auc),
    AUPRC          = mean(results_store_full[[tk]]$auprc),
    AUPRC_SD       = sd(results_store_full[[tk]]$auprc)
  )
}

results_rsf_full_threshold <- do.call(rbind, results_list_full)
saveRDS(results_rsf_full_threshold, "rsf_full_threshold_results.rds")
```

```
library(pander)
pander(results_rsf_full_threshold, digits = 4,
       caption = "RSF (No SMOTE) Full Dataset Performance Metrics")
```

Table 15: RSF (No SMOTE) Full Dataset Performance Metrics (continued below)

	Threshold	Sensitivity	Sensitivity_SD	Precision
0.9	0.9	0.9246	0.02101	0.02194
0.95	0.95	0.9164	0.01649	0.02986
0.975	0.975	0.8982	0.02016	0.05853
0.99	0.99	0.8715	0.01941	0.1426
0.995	0.995	0.8595	0.01559	0.2826

	Precision_SD	AUC	AUC_SD	AUPRC	AUPRC_SD
0.9	0.002184	0.9659	0.008512	0.7686	0.0352
0.95	0.002585	0.9659	0.008512	0.7686	0.0352
0.975	0.005312	0.9659	0.008512	0.7686	0.0352
0.99	0.01072	0.9659	0.008512	0.7686	0.0352
0.995	0.01937	0.9659	0.008512	0.7686	0.0352

8.4.4. Random Survival Forest (SMOTE) - Full Dataset Finally, we want to perform the Random Survival Forest (SMOTE) on the entire dataset, which is as follows:

```
library(caret)
library(smotefamily)
library(dplyr)
library(randomForestSRC)
library(pROC)
library(PRROC)

set.seed(44940076)

n_splits <- 10
threshold_grid <- c(0.90, 0.95, 0.975, 0.99, 0.995)

results_store_full <- list()
for (t in threshold_grid) {
  results_store_full[[as.character(t)]] <- list(
```

```
    sens  = numeric(n_splits),
    prec  = numeric(n_splits),
    auc   = numeric(n_splits),
    auprc = numeric(n_splits)
  )
}

for (i in 1:n_splits) {
  message("Split ", i, " of ", n_splits, " - ", Sys.time())

  rsf_smote_full_train_idx <- createDataPartition(fraud_data$status, p = 0.7, list = FALSE)
  rsf_smote_full_train_data <- fraud_data[rsf_smote_full_train_idx, ]
  rsf_smote_full_test_data  <- fraud_data[-rsf_smote_full_train_idx, ]

  rsf_smote_full_train_data <- rsf_smote_full_train_data %>% dplyr::select(-Class)
  rsf_smote_full_test_data  <- rsf_smote_full_test_data %>% dplyr::select(-Class)

  rsf_smote_full_train_data$status <- as.numeric(as.character(rsf_smote_full_train_data$status))
  rsf_smote_full_test_data$status  <- as.numeric(as.character(rsf_smote_full_test_data$status))

  rsf_smote_full_x_train <- rsf_smote_full_train_data %>% dplyr::select(-status, -time)
  rsf_smote_full_y_train <- rsf_smote_full_train_data$status

  rsf_smote_full_n_min <- min(table(rsf_smote_full_y_train))
  rsf_smote_full_k_use <- min(5, max(1, rsf_smote_full_n_min - 1))

  rsf_smote_full_output <- SMOTE(
    X = rsf_smote_full_x_train,
    target = rsf_smote_full_y_train,
    K = rsf_smote_full_k_use,
    dup_size = 1
  )

  rsf_smote_full_balanced <- rsf_smote_full_output$data
  rsf_smote_full_balanced$status <- as.numeric(as.character(rsf_smote_full_balanced$class))
  rsf_smote_full_balanced$class <- NULL

  rsf_smote_full_balanced$time <- NA_real_
  rsf_smote_full_idx0 <- which(rsf_smote_full_balanced$status == 0)
  rsf_smote_full_idx1 <- which(rsf_smote_full_balanced$status == 1)

  rsf_smote_full_balanced$time[rsf_smote_full_idx0] <- sample(
    rsf_smote_full_train_data$time[rsf_smote_full_train_data$status == 0],
    length(rsf_smote_full_idx0), replace = TRUE
  )
  rsf_smote_full_balanced$time[rsf_smote_full_idx1] <- sample(
    rsf_smote_full_train_data$time[rsf_smote_full_train_data$status == 1],
    length(rsf_smote_full_idx1), replace = TRUE
  )

  rsf_smote_full_model <- rfsrc(
    Surv(time, status) ~ .,
    data = rsf_smote_full_balanced,
```

```
  ntree = 50,
  nodesize = 15,
  splitrule = "logrank",
  nthread = 4,
  fast = TRUE
)

rsf_smote_full_test_x <- rsf_smote_full_test_data[, rsf_smote_full_model$xvar.names, drop = FALSE]
rsf_smote_full_pred <- predict(rsf_smote_full_model, newdata = rsf_smote_full_test_x)

rsf_smote_full_risk <- 1 - rsf_smote_full_pred$survival[, ncol(rsf_smote_full_pred$survival)]
rsf_smote_full_risk <- as.numeric(rsf_smote_full_risk)

stopifnot(length(rsf_smote_full_risk) == nrow(rsf_smote_full_test_data))

rsf_smote_full_train_pred <- predict(rsf_smote_full_model,
                                     newdata = rsf_smote_full_train_data[, rsf_smote_full_model$xvar
rsf_smote_full_train_risk <- 1 - rsf_smote_full_train_pred$survival[, ncol(rsf_smote_full_train_pred$s

rsf_smote_full_roc_obj <- roc(
  response = rsf_smote_full_test_data$status,
  predictor = rsf_smote_full_risk,
  quiet = TRUE
)
rsf_smote_full_auc_i <- as.numeric(auc(rsf_smote_full_roc_obj))

rsf_smote_full_true_labels <- rsf_smote_full_test_data$status
rsf_smote_full_keep <- is.finite(rsf_smote_full_risk) & is.finite(rsf_smote_full_true_labels)
rsf_smote_full_scores <- rsf_smote_full_risk[rsf_smote_full_keep]
rsf_smote_full_labels <- rsf_smote_full_true_labels[rsf_smote_full_keep]

rsf_smote_full_pr_obj <- pr.curve(
  scores.class0 = rsf_smote_full_scores[rsf_smote_full_labels == 1],
  scores.class1 = rsf_smote_full_scores[rsf_smote_full_labels == 0],
  curve = FALSE
)
rsf_smote_full_auprc_i <- rsf_smote_full_pr_obj$auc.integral

for (t in threshold_grid) {

  rsf_smote_full_threshold <- quantile(rsf_smote_full_train_risk, probs = t, na.rm = TRUE)
  rsf_smote_full_test_pred <- ifelse(rsf_smote_full_risk > rsf_smote_full_threshold, 1, 0)

  rsf_smote_full_cm <- confusionMatrix(
    factor(rsf_smote_full_test_pred, levels = c(0, 1)),
    factor(rsf_smote_full_test_data$status, levels = c(0, 1)),
    positive = "1"
  )

  tk <- as.character(t)
  results_store_full[[tk]]$sens[i]  <- rsf_smote_full_cm$byClass["Sensitivity"]
  results_store_full[[tk]]$prec[i]  <- rsf_smote_full_cm$byClass["Precision"]
  results_store_full[[tk]]$auc[i]   <- rsf_smote_full_auc_i
```

```
    results_store_full[[tk]]$auprc[i] <- rsf_smote_full_auprc_i
  }

  # Clean up large objects to free memory
  rm(rsf_smote_full_model, rsf_smote_full_pred, rsf_smote_full_train_pred,
     rsf_smote_full_train_data, rsf_smote_full_test_data, rsf_smote_full_balanced)
  gc()
}

results_list_full <- list()
for (t in threshold_grid) {
  tk <- as.character(t)
  results_list_full[[tk]] <- data.frame(
    Threshold      = t,
    Sensitivity    = mean(results_store_full[[tk]]$sens),
    Sensitivity_SD = sd(results_store_full[[tk]]$sens),
    Precision      = mean(results_store_full[[tk]]$prec),
    Precision_SD   = sd(results_store_full[[tk]]$prec),
    AUC            = mean(results_store_full[[tk]]$auc),
    AUC_SD         = sd(results_store_full[[tk]]$auc),
    AUPRC          = mean(results_store_full[[tk]]$auprc),
    AUPRC_SD       = sd(results_store_full[[tk]]$auprc)
  )
}

results_rsf_smote_full_threshold <- do.call(rbind, results_list_full)
saveRDS(results_rsf_smote_full_threshold, "rsf_smote_full_threshold_results.rds")
```

```
pander(results_rsf_smote_full_threshold, digits = 4,
       caption = "RSF (SMOTE) Full Dataset Performance Metrics")
```

Table 17: RSF (SMOTE) Full Dataset Performance Metrics (continued below)

	Threshold	Sensitivity	Sensitivity_SD	Precision
0.9	0.9	0.9439	0.01813	0.01544
0.95	0.95	0.9276	0.02398	0.03185
0.975	0.975	0.9111	0.02441	0.05701
0.99	0.99	0.8962	0.0209	0.14
0.995	0.995	0.8743	0.02973	0.283

	Precision_SD	AUC	AUC_SD	AUPRC	AUPRC_SD
0.9	0.001386	0.9726	0.008932	0.8321	0.03323
0.95	0.002537	0.9726	0.008932	0.8321	0.03323
0.975	0.004679	0.9726	0.008932	0.8321	0.03323
0.99	0.009558	0.9726	0.008932	0.8321	0.03323
0.995	0.02276	0.9726	0.008932	0.8321	0.03323

The full dataset results revealed a notable AUPRC advantage for RSF (SMOTE) of 0.832, as against the others that were between 0.760-0.769. To investigate whether this reflects genuine exploitation of temporal fraud-risk structure, a simulation analysis was conducted using datasets with known relative-risk time profiles.

8.5. Simulation Analysis: Temporal Risk Structure

The full dataset robustness check in Section 8.4 received a AUPRC advantage for the RSF (SMOTE) over the Logistic Regression models (0.831 vs 0.760-0.769), a finding that stands in contrast to the near-identical performance observed on the downsampled dataset. One plausible explanation for this divergence is that the full dataset's temporal fraud-risk structure, characterised by relative fraud-risk peaking at 3-3.5 times the baseline during low-activity periods, as documented in Section 6.6, provides a signal that RSF's survival framework is better positioned to exploit than a standard linear classifier. To investigate this hypothesis, a simulation analysis was conducted, where fraud labels were reassigned across the full dataset according to four synthetic relative risk profiles of increasing temporal intensity: flat, mild moderate, and strong. By controlling the degree of temporal structure while holding the feature space constant, this design allows direct assessment of whether RSF's AUPRC advantage grows as the temporal fraud-risk structure intensifies, which would be expected if the survival framework was genuinely leveraging time as a predictive axis.

```
## Step 1: Run relevant packages
library(caret)
library(dplyr)
library(smotefamily)
library(randomForestSRC)
library(pROC)
library(PRROC)
library(ggplot2)

# Step 2: Set the seed
set.seed(44940076)

## Step 3: Estabilish the simulation parameters
n_splits_sim <- 10

## Step 4: Define relative risk profile functions
rr_profiles <- list(
  flat = function(t) rep(1, length(t)),
  mild = function(t) 1 + 1.0 * sin (pi * t),
  moderate = function(t){
    1 + 2.0 * (exp(-((t - 0.10)^2) / 0.004) + exp(-((t - 0.55)^2) / 0.004))
  },
  strong = function(t) {
    1 + 4.0 * (exp(-((t - 0.10)^2) / 0.004) + exp(-((t - 0.55)^2) / 0.04))
  }
)

## Step 5: Define label reassignment function
simulate_labels <- function (Time_vec, rr_fn, n_fraud = 492L, seed = 42) {
  set.seed(seed)
  t_norm        <- (Time_vec - min(Time_vec)) / (max(Time_vec) - min(Time_vec))
  weights       <- rr_fn(t_norm)
  weights       <- weights / sum(weights)
  fraud_idx     <- sample(length(Time_vec), size = n_fraud, replace = FALSE, prob = weights)
  sim_class     <- integer(length(Time_vec))
  sim_class[fraud_idx] <- 1L
  sim_class

}

## Step 6: Create storage
```

```r
sim_results_store <- list()
for (profile_name in names(rr_profiles)) {
  sim_results_store[[profile_name]] <- list(
    auprc_lr        = numeric(n_splits_sim),
    auprc_rsf_smote = numeric(n_splits_sim)
  )
}

## Step 7: Execute simulation across profiles and splits
for (profile_name in names(rr_profiles)) {
  message("- Profile: ", profile_name)

  # Reassign Class and status labels according to current RR profile
  sim_data        <- fraud_data
  sim_data$Class  <- simulate_labels(sim_data$Time, rr_profiles[[profile_name]])
  sim_data$status <- sim_data$Class

  for (i in seq_len(n_splits_sim)) {
    message("  Split ", i, " of ", n_splits_sim, " - ", Sys.time())

    # 1. Stratified 70-30 split
    sim_train_idx  <- createDataPartition(sim_data$status, p = 0.7, list = FALSE)
    sim_train_data <- sim_data[sim_train_idx, ]
    sim_test_data  <- sim_data[-sim_train_idx, ]

    # 2. Ensure numeric outcome
    sim_train_data$status <- as.numeric(as.character(sim_train_data$status))
    sim_test_data$status  <- as.numeric(as.character(sim_test_data$status))

    sim_true_labels <- sim_test_data$status

    ###### LOGISTIC REGRESSION ######
    # 3. Fit LR model, once per split
    sim_lr_model <- glm(status ~ . - time - Class,
                        data   = sim_train_data,
                        family = binomial)

    # 4. Predict probabilities
    sim_lr_test_prob <- predict(sim_lr_model,
                                newdata = sim_test_data,
                                type = "response")

    # 5. AUPRC
    sim_lr_keep   <- is.finite(sim_lr_test_prob) & is.finite(sim_true_labels)
    sim_lr_pr_obj <- pr.curve(
      scores.class0 = sim_lr_test_prob[sim_lr_keep][sim_true_labels[sim_lr_keep] == 1],
      scores.class1 = sim_lr_test_prob[sim_lr_keep][sim_true_labels[sim_lr_keep] == 0],
      curve = FALSE
    )
    sim_results_store[[profile_name]]$auprc_lr[i] <- sim_lr_pr_obj$auc.integral

    ##### RSF (SMOTE) #####
    # 6. Prepare RSF training and test data (removing the 'Class' column)
```

```r
sim_rsf_train <- sim_train_data %>% dplyr::select(-Class)
sim_rsf_test  <- sim_test_data  %>% dplyr::select(-Class)

# 7. Prepare SMOTE inputs (exclude 'time' and 'status')
sim_rsf_x_train <- sim_rsf_train %>% dplyr::select(-status, -time)
sim_rsf_y_train <- sim_rsf_train$status

# 8. Choose SMOTE neighbourhood size safely
sim_rsf_n_min <- min(table(sim_rsf_y_train))
sim_rsf_k_use <- min(5, max(1, sim_rsf_n_min - 1))

# 9. Apply SMOTE to training data only
sim_rsf_smote_output <- SMOTE(
  X        = sim_rsf_x_train,
  target   = sim_rsf_y_train,
  K        = sim_rsf_k_use,
  dup_size = 1
)
sim_rsf_balanced        <- sim_rsf_smote_output$data
sim_rsf_balanced$status <- as.numeric(as.character(sim_rsf_balanced$class))
sim_rsf_balanced$class  <- NULL

# 10. Reattach empirical time values to SMOTE observations
sim_rsf_balanced$time <- NA_real_
sim_rsf_idx0 <- which(sim_rsf_balanced$status == 0)
sim_rsf_idx1 <- which(sim_rsf_balanced$status == 1)
sim_rsf_balanced$time[sim_rsf_idx0] <- sample(
  sim_rsf_train$time[sim_rsf_train$status == 0],
  length(sim_rsf_idx0), replace = TRUE
)
sim_rsf_balanced$time[sim_rsf_idx1] <- sample(
  sim_rsf_train$time[sim_rsf_train$status == 1],
  length(sim_rsf_idx1), replace = TRUE
)

# 11. Fit RSF on SMOTE-balanced training data (once per split)
sim_rsf_model <- rfsrc(
  Surv(time, status) ~ .,
  data      = sim_rsf_balanced,
  ntree     = 100,
  nodesize  = 15,
  splitrule = "logrank",
  nthread   = 4,
  fast      = TRUE
)

# 12. Predict on untouched test data
sim_rsf_test_x <- sim_rsf_test[, sim_rsf_model$xvar.names, drop = FALSE]
sim_rsf_pred   <- predict(sim_rsf_model, newdata = sim_rsf_test_x)

# 13. Compute risk score
sim_rsf_risk <- 1 - sim_rsf_pred$survival[, ncol(sim_rsf_pred$survival)]
sim_rsf_risk <- as.numeric(sim_rsf_risk)
```

```
    # 14. AUPRC
    sim_rsf_keep   <- is.finite(sim_rsf_risk) & is.finite(sim_true_labels)
    sim_rsf_pr_obj <- pr.curve(
      scores.class0 = sim_rsf_risk[sim_rsf_keep][sim_true_labels[sim_rsf_keep] == 1],
      scores.class1 = sim_rsf_risk[sim_rsf_keep][sim_true_labels[sim_rsf_keep] == 0],
      curve = FALSE
    )
    sim_results_store[[profile_name]]$auprc_rsf_smote[i] <- sim_rsf_pr_obj$auc.integral

    # Clean up large objects to free memory
    rm(sim_rsf_model, sim_rsf_pred,
       sim_rsf_balanced, sim_rsf_smote_output)
    gc()
  }
}

## Step 8: Compile results
sim_results_list <- list()
for (profile_name in names(rr_profiles)) {
  sim_results_list[[profile_name]] <- data.frame(
    Profile             = profile_name,
    AUPRC_LR            = mean(sim_results_store[[profile_name]]$auprc_lr),
    AUPRC_LR_SD         = sd(sim_results_store[[profile_name]]$auprc_lr),
    AUPRC_RSF_SMOTE     = mean(sim_results_store[[profile_name]]$auprc_rsf_smote),
    AUPRC_RSF_SMOTE_SD  = sd(sim_results_store[[profile_name]]$auprc_rsf_smote)
  )
}

results_sim_summary <- do.call(rbind, sim_results_list)
results_sim_summary$Profile <- factor(results_sim_summary$Profile,
                                      levels = c("flat", "mild", "moderate", "strong"))

saveRDS(results_sim_summary, "rsf_sim_results.rds")
```

```
## Create the Simulation Analysis Summary
pander(results_sim_summary, digits = 4,
       caption = "Simulation Analysis - Mean AUPRC by Temporal Risk Profile")
```

Table 19: Simulation Analysis - Mean AUPRC by Temporal Risk Profile (continued below)

	Profile	AUPRC_LR	AUPRC_LR_SD	AUPRC_RSF_SMOTE
flat	flat	0.001817	0.0001374	0.001877
mild	mild	0.001803	0.000216	0.001722
moderate	moderate	0.002284	0.0001493	0.002136
strong	strong	0.001957	0.000164	0.001914

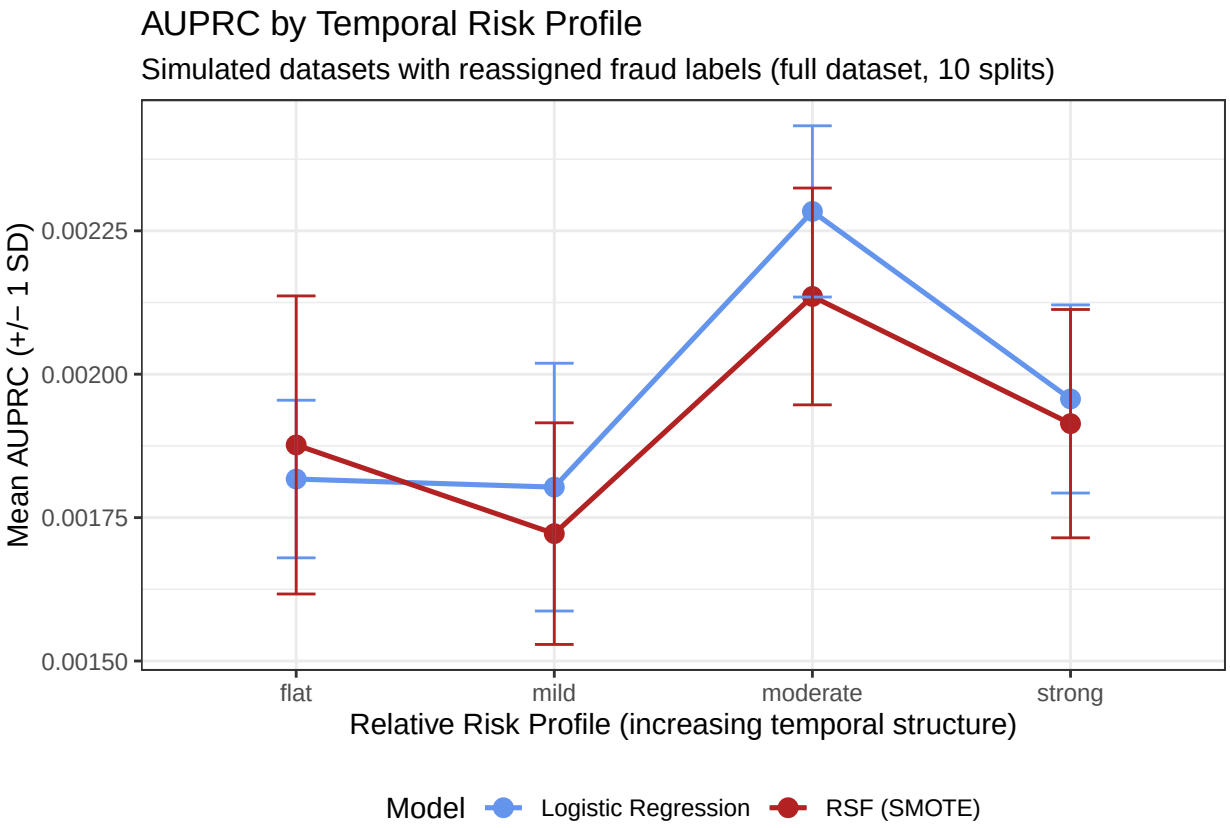
	AUPRC_RSF_SMOTE_SD
flat	0.0002598
mild	0.0001932
moderate	0.000189

	AUPRC_RSF_SMOTE_SD
strong	0.000199

```r
## Plot AUPRC by profile and model
sim_plot_long <- tidyr::pivot_longer(
  results_sim_summary,
  cols      = c(AUPRC_LR, AUPRC_RSF_SMOTE),
  names_to  = "Model",
  values_to = "Mean_AUPRC"
) %>%
  mutate(
    SD = ifelse(Model == "AUPRC_LR",
                results_sim_summary$AUPRC_LR_SD[match(Profile, results_sim_summary$Profile)],
                results_sim_summary$AUPRC_RSF_SMOTE_SD[match(Profile, results_sim_summary$Profile)]),
    Model = dplyr::recode(Model,
      "AUPRC_LR"        = "Logistic Regression",
      "AUPRC_RSF_SMOTE" = "RSF (SMOTE)")
  )

ggplot(sim_plot_long,
       aes(x = Profile, y = Mean_AUPRC, colour = Model, group = Model)) +
  geom_line(linewidth = 0.9) +
  geom_point(size = 3) +
  geom_errorbar(
    aes(ymin = Mean_AUPRC - SD,
        ymax = Mean_AUPRC + SD),
    width = 0.15
  ) +
  scale_colour_manual(
    values = c("Logistic Regression" = "cornflowerblue",
               "RSF (SMOTE)"         = "firebrick")

  ) +
    labs(
      title    = "AUPRC by Temporal Risk Profile",
      subtitle = "Simulated datasets with reassigned fraud labels (full dataset, 10 splits)",
      x        = "Relative Risk Profile (increasing temporal structure)",
      y        = "Mean AUPRC (+/- 1 SD)",
      colour   = "Model"
    ) +
  theme_bw() +
  theme(legend.position = "bottom")
```



Commentary: The simulation results provide no support for the hypothesis that the RSF is better positioned than Logistic Regression to exploit temporal fraud-risk structure, with both models performing at near-baseline AUPRC levels (~0.00173, which is consistent with the dataset fraud prevalence of 492/284807) across all four profiles. This reflects the fact that reassigning fraud labels by time alone destroys the feature signal from V1-V28 that both models otherwise rely upon. Neither model demonstrates a meaningful or consistent AUPRC advantage as the temporal structure intensifies from flat through to strong, with Logistic Regression marginally outperforming RSF (SMOTE) at the moderate and strong profiles, contrary to the expected direction of effect.

This finding reinforces the primary null result: RSF offers no discriminatory advantage over Logistic Regression on this dataset. The simulation provides additional evidence that this equivalence holds even under controlled conditions that are designed to favour the survival framework, suggesting that the modest AUPRC advantage that was observed for RSF (SMOTE) on the full dataset in Section 8.4 is unlikely to reflect a genuine exploitation of the temporal structure.

8.6. Supplementary: Downsampling Sensitivity Check (25k)

To assess whether the primary findings are sensitive to the choice of 50000 non-fraudulent transactions, a supplementary analysis was conducted using a reduced sample of 25000 non-fraudulent transactions, yielding a working dataset of 25492 observations, and an effective fraud rate of approximately 1.93%. All experimental parameters were held constant, with 30 repeated stratified splits, the same threshold grid, and identical evaluation metrics, enable direct comparison with the primary downsampled results.

```
## Step 2: LR (No SMOTE) - 25k Sensitivity Check
library(caret)
library(pROC)
```

```r
library(PRROC)

set.seed(44940076)

n_splits <- 30
threshold_grid <- c(0.90, 0.95, 0.975, 0.99, 0.995)

results_store_25k_lr <- list()
for (t in threshold_grid) {
  results_store_25k_lr[[as.character(t)]] <- list(
    sens  = numeric(n_splits),
    prec  = numeric(n_splits),
    auc   = numeric(n_splits),
    auprc = numeric(n_splits)
  )
}

for (i in 1:n_splits) {

  # 1. Stratified 70-30 split
  lr_25k_train_idx <- createDataPartition(fraud_subset_25k$status, p = 0.7, list = FALSE)
  lr_25k_train_data <- fraud_subset_25k[lr_25k_train_idx, ]
  lr_25k_test_data  <- fraud_subset_25k[-lr_25k_train_idx, ]

  # 2. Fit Logistic Regression model (ONCE per split)
  lr_25k_model <- glm(status ~ . - time - Class,
                      data = lr_25k_train_data,
                      family = binomial)

  # 3. Predict probabilities (ONCE per split)
  lr_25k_train_prob <- predict(lr_25k_model, newdata = lr_25k_train_data, type = "response")
  lr_25k_test_prob  <- predict(lr_25k_model, newdata = lr_25k_test_data,  type = "response")

  # 4. ROC + AUC (threshold-independent, compute once)
  lr_25k_true_labels <- as.numeric(as.character(lr_25k_test_data$status))

  lr_25k_auc_i <- as.numeric(auc(
    roc(response = lr_25k_true_labels, predictor = lr_25k_test_prob, quiet = TRUE)
  ))

  # 5. AUPRC (threshold-independent, compute once)
  lr_25k_pr_obj <- pr.curve(
    scores.class0 = lr_25k_test_prob[lr_25k_true_labels == 1],
    scores.class1 = lr_25k_test_prob[lr_25k_true_labels == 0],
    curve = FALSE
  )
  lr_25k_auprc_i <- lr_25k_pr_obj$auc.integral

  # 6. Loop over thresholds
  for (t in threshold_grid) {

    lr_25k_threshold <- quantile(lr_25k_train_prob, probs = t, na.rm = TRUE)
    lr_25k_test_pred <- ifelse(lr_25k_test_prob > lr_25k_threshold, 1, 0)
```

```
    lr_25k_cm <- confusionMatrix(
      factor(lr_25k_test_pred,        levels = c(0, 1)),
      factor(lr_25k_test_data$status, levels = c(0, 1)),
      positive = "1"
    )

    tk <- as.character(t)
    results_store_25k_lr[[tk]]$sens[i]  <- lr_25k_cm$byClass["Sensitivity"]
    results_store_25k_lr[[tk]]$prec[i]  <- lr_25k_cm$byClass["Precision"]
    results_store_25k_lr[[tk]]$auc[i]   <- lr_25k_auc_i
    results_store_25k_lr[[tk]]$auprc[i] <- lr_25k_auprc_i
  }
}
```

8.6.1. Logistic Regression (No SMOTE) - 25k Sensitivity Check

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
## Compile results
results_list_25k_lr <- list()
for (t in threshold_grid) {
  tk <- as.character(t)
  results_list_25k_lr[[tk]] <- data.frame(
    Threshold      = t,
    Sensitivity    = mean(results_store_25k_lr[[tk]]$sens),
    Sensitivity_SD = sd(results_store_25k_lr[[tk]]$sens),
    Precision      = mean(results_store_25k_lr[[tk]]$prec),
    Precision_SD   = sd(results_store_25k_lr[[tk]]$prec),
    AUC            = mean(results_store_25k_lr[[tk]]$auc),
    AUC_SD         = sd(results_store_25k_lr[[tk]]$auc),
    AUPRC          = mean(results_store_25k_lr[[tk]]$auprc),
    AUPRC_SD       = sd(results_store_25k_lr[[tk]]$auprc)
```

```
  )
}

results_25k_lr_threshold <- do.call(rbind, results_list_25k_lr)
auprc_25k_lr_splits <- results_store_25k_lr[["0.99"]]$auprc

saveRDS(results_25k_lr_threshold, "lr_25k_threshold_results.rds")
```

```
## Step 3b: LR (SMOTE) - 25k Sensitivity Check
library(caret)
library(dplyr)
library(smotefamily)
library(pROC)
library(PRROC)

set.seed(44940076)

n_splits <- 30
threshold_grid <- c(0.90, 0.95, 0.975, 0.99, 0.995)

results_store_25k_lr_smote <- list()
for (t in threshold_grid) {
  results_store_25k_lr_smote[[as.character(t)]] <- list(
    sens  = numeric(n_splits),
    prec  = numeric(n_splits),
    auc   = numeric(n_splits),
    auprc = numeric(n_splits)
  )
}

for (i in 1:n_splits) {

  # 1. Stratified 70-30 split BEFORE SMOTE
  lr_smote_25k_train_idx  <- createDataPartition(fraud_subset_25k$status, p = 0.7, list = FALSE)
  lr_smote_25k_train_data <- fraud_subset_25k[lr_smote_25k_train_idx, ]
  lr_smote_25k_test_data  <- fraud_subset_25k[-lr_smote_25k_train_idx, ]

  # 2. Ensure numeric 0/1 outcome labels
  lr_smote_25k_train_data$status <- as.numeric(as.character(lr_smote_25k_train_data$status))
  lr_smote_25k_test_data$status  <- as.numeric(as.character(lr_smote_25k_test_data$status))

  # 3. Prepare training data for SMOTE
  lr_smote_25k_train_input <- lr_smote_25k_train_data %>%
    dplyr::select(-time, -Class)

  lr_smote_25k_x_train <- lr_smote_25k_train_input %>% dplyr::select(-status)
  lr_smote_25k_y_train <- lr_smote_25k_train_input$status

  # 4. Choose SMOTE neighbourhood size safely
  lr_smote_25k_n_min <- min(table(lr_smote_25k_y_train))
  lr_smote_25k_k_use <- min(5, max(1, lr_smote_25k_n_min - 1))
```

```r
# 5. Apply SMOTE to training data only
lr_smote_25k_output <- SMOTE(
  X        = lr_smote_25k_x_train,
  target   = lr_smote_25k_y_train,
  K        = lr_smote_25k_k_use,
  dup_size = 1
)

lr_smote_25k_balanced        <- lr_smote_25k_output$data
lr_smote_25k_balanced$status <- as.numeric(as.character(lr_smote_25k_balanced$class))
lr_smote_25k_balanced$class  <- NULL

# 6. Fit Logistic Regression on SMOTE-balanced training data (ONCE per split)
lr_smote_25k_model <- glm(
  status ~ .,
  data   = lr_smote_25k_balanced,
  family = binomial
)

# 7. Predict probabilities (ONCE per split)
lr_smote_25k_train_prob <- predict(lr_smote_25k_model,
                                   newdata = lr_smote_25k_train_data,
                                   type    = "response")
lr_smote_25k_test_prob  <- predict(lr_smote_25k_model,
                                   newdata = lr_smote_25k_test_data,
                                   type    = "response")

# 8. ROC + AUC (threshold-independent, compute once)
lr_smote_25k_auc_i <- as.numeric(auc(
  roc(response  = lr_smote_25k_test_data$status,
      predictor = lr_smote_25k_test_prob,
      quiet     = TRUE)
))

# 9. AUPRC (threshold-independent, compute once)
lr_smote_25k_true_labels <- lr_smote_25k_test_data$status
lr_smote_25k_keep        <- is.finite(lr_smote_25k_test_prob) &
                            is.finite(lr_smote_25k_true_labels)

lr_smote_25k_pr_obj <- pr.curve(
  scores.class0 = lr_smote_25k_test_prob[lr_smote_25k_keep][lr_smote_25k_true_labels[lr_smote_25k_kee
  scores.class1 = lr_smote_25k_test_prob[lr_smote_25k_keep][lr_smote_25k_true_labels[lr_smote_25k_kee
  curve = FALSE
)
lr_smote_25k_auprc_i <- lr_smote_25k_pr_obj$auc.integral

# 10. Loop over thresholds
for (t in threshold_grid) {

  lr_smote_25k_threshold <- quantile(lr_smote_25k_train_prob, probs = t, na.rm = TRUE)
  lr_smote_25k_test_pred <- ifelse(lr_smote_25k_test_prob > lr_smote_25k_threshold, 1, 0)

  lr_smote_25k_cm <- confusionMatrix(
```

```
      factor(lr_smote_25k_test_pred,        levels = c(0, 1)),
      factor(lr_smote_25k_test_data$status, levels = c(0, 1)),
      positive = "1"
    )

    tk <- as.character(t)
    results_store_25k_lr_smote[[tk]]$sens[i]  <- lr_smote_25k_cm$byClass["Sensitivity"]
    results_store_25k_lr_smote[[tk]]$prec[i]  <- lr_smote_25k_cm$byClass["Precision"]
    results_store_25k_lr_smote[[tk]]$auc[i]   <- lr_smote_25k_auc_i
    results_store_25k_lr_smote[[tk]]$auprc[i] <- lr_smote_25k_auprc_i
  }
}
```

8.6.3. Logistic Regression (SMOTE) - 25k Sensitivity Check

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred

## Warning: glm.fit: algorithm did not converge

## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred

## Warning: glm.fit: algorithm did not converge

## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred

## Warning: glm.fit: algorithm did not converge

## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred

## Warning: glm.fit: algorithm did not converge

## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred

## Warning: glm.fit: algorithm did not converge

## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred

## Warning: glm.fit: algorithm did not converge
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
## Compile results
results_list_25k_lr_smote <- list()
for (t in threshold_grid) {
  tk <- as.character(t)
  results_list_25k_lr_smote[[tk]] <- data.frame(
    Threshold      = t,
    Sensitivity    = mean(results_store_25k_lr_smote[[tk]]$sens),
    Sensitivity_SD = sd(results_store_25k_lr_smote[[tk]]$sens),
    Precision      = mean(results_store_25k_lr_smote[[tk]]$prec),
    Precision_SD   = sd(results_store_25k_lr_smote[[tk]]$prec),
    AUC            = mean(results_store_25k_lr_smote[[tk]]$auc),
    AUC_SD         = sd(results_store_25k_lr_smote[[tk]]$auc),
    AUPRC          = mean(results_store_25k_lr_smote[[tk]]$auprc),
    AUPRC_SD       = sd(results_store_25k_lr_smote[[tk]]$auprc)
  )
}

results_25k_lr_smote_threshold <- do.call(rbind, results_list_25k_lr_smote)
auprc_25k_lr_smote_splits <- results_store_25k_lr_smote[["0.99"]]$auprc

saveRDS(results_25k_lr_smote_threshold, "lr_smote_25k_threshold_results.rds")
```

```
## Step 3: RSF (No SMOTE) - 25k Sensitivity Check
library(caret)
library(dplyr)
library(randomForestSRC)
library(pROC)
library(PRROC)

set.seed(44940076)

n_splits <- 30
threshold_grid <- c(0.90, 0.95, 0.975, 0.99, 0.995)

results_store_25k_rsf <- list()
for (t in threshold_grid) {
  results_store_25k_rsf[[as.character(t)]] <- list(
    sens  = numeric(n_splits),
    prec  = numeric(n_splits),
    auc   = numeric(n_splits),
    auprc = numeric(n_splits)
  )
}

for (i in 1:n_splits) {
  message("RSF 25k - Split ", i, " of ", n_splits, " - ", Sys.time())

  # 1. Stratified 70-30 split
  rsf_25k_train_idx  <- createDataPartition(fraud_subset_25k$status, p = 0.7, list = FALSE)
```

```r
rsf_25k_train_data <- fraud_subset_25k[rsf_25k_train_idx, ]
rsf_25k_test_data  <- fraud_subset_25k[-rsf_25k_train_idx, ]

# 2. Keep only RSF-relevant variables
rsf_25k_train_data <- rsf_25k_train_data %>% dplyr::select(-Class)
rsf_25k_test_data  <- rsf_25k_test_data  %>% dplyr::select(-Class)

# 3. Ensure numeric 0/1 outcome
rsf_25k_train_data$status <- as.numeric(as.character(rsf_25k_train_data$status))
rsf_25k_test_data$status  <- as.numeric(as.character(rsf_25k_test_data$status))

# 4. Fit RSF model (ONCE per split)
rsf_25k_model <- rfsrc(
  Surv(time, status) ~ .,
  data      = rsf_25k_train_data,
  ntree     = 100,
  nodesize  = 15,
  splitrule = "logrank",
  nthread   = 4,
  fast      = TRUE
)

# 5. Predict on test data
rsf_25k_test_x <- rsf_25k_test_data[, rsf_25k_model$xvar.names, drop = FALSE]
rsf_25k_pred   <- predict(rsf_25k_model, newdata = rsf_25k_test_x)
rsf_25k_risk   <- as.numeric(1 - rsf_25k_pred$survival[, ncol(rsf_25k_pred$survival)])

stopifnot(length(rsf_25k_risk) == nrow(rsf_25k_test_data))

# 6. Compute training risk scores (ONCE per split)
rsf_25k_train_pred <- predict(rsf_25k_model,
                              newdata = rsf_25k_train_data[, rsf_25k_model$xvar.names,
                                                           drop = FALSE])
rsf_25k_train_risk <- as.numeric(1 - rsf_25k_train_pred$survival[,
                                    ncol(rsf_25k_train_pred$survival)])

# 7. ROC + AUC (threshold-independent, compute once)
rsf_25k_roc_obj <- roc(
  response  = rsf_25k_test_data$status,
  predictor = rsf_25k_risk,
  quiet     = TRUE
)
rsf_25k_auc_i <- as.numeric(auc(rsf_25k_roc_obj))

# 8. AUPRC (threshold-independent, compute once)
rsf_25k_true_labels <- rsf_25k_test_data$status
rsf_25k_keep        <- is.finite(rsf_25k_risk) & is.finite(rsf_25k_true_labels)

rsf_25k_pr_obj <- pr.curve(
  scores.class0 = rsf_25k_risk[rsf_25k_keep][rsf_25k_true_labels[rsf_25k_keep] == 1],
  scores.class1 = rsf_25k_risk[rsf_25k_keep][rsf_25k_true_labels[rsf_25k_keep] == 0],
  curve = FALSE
)
```

```
  rsf_25k_auprc_i <- rsf_25k_pr_obj$auc.integral

  # 9. Loop over thresholds
  for (t in threshold_grid) {

    rsf_25k_threshold <- quantile(rsf_25k_train_risk, probs = t, na.rm = TRUE)
    rsf_25k_test_pred <- ifelse(rsf_25k_risk > rsf_25k_threshold, 1, 0)

    rsf_25k_cm <- confusionMatrix(
      factor(rsf_25k_test_pred,        levels = c(0, 1)),
      factor(rsf_25k_test_data$status, levels = c(0, 1)),
      positive = "1"
    )

    tk <- as.character(t)
    results_store_25k_rsf[[tk]]$sens[i]  <- rsf_25k_cm$byClass["Sensitivity"]
    results_store_25k_rsf[[tk]]$prec[i]  <- rsf_25k_cm$byClass["Precision"]
    results_store_25k_rsf[[tk]]$auc[i]   <- rsf_25k_auc_i
    results_store_25k_rsf[[tk]]$auprc[i] <- rsf_25k_auprc_i
  }

  rm(rsf_25k_model, rsf_25k_pred, rsf_25k_train_pred)
  gc()
}

## Compile results
results_list_25k_rsf <- list()
for (t in threshold_grid) {
  tk <- as.character(t)
  results_list_25k_rsf[[tk]] <- data.frame(
    Threshold      = t,
    Sensitivity    = mean(results_store_25k_rsf[[tk]]$sens),
    Sensitivity_SD = sd(results_store_25k_rsf[[tk]]$sens),
    Precision      = mean(results_store_25k_rsf[[tk]]$prec),
    Precision_SD   = sd(results_store_25k_rsf[[tk]]$prec),
    AUC            = mean(results_store_25k_rsf[[tk]]$auc),
    AUC_SD         = sd(results_store_25k_rsf[[tk]]$auc),
    AUPRC          = mean(results_store_25k_rsf[[tk]]$auprc),
    AUPRC_SD       = sd(results_store_25k_rsf[[tk]]$auprc)
  )
}

results_25k_rsf_threshold <- do.call(rbind, results_list_25k_rsf)
auprc_25k_rsf_splits <- results_store_25k_rsf[["0.99"]]$auprc

saveRDS(results_25k_rsf_threshold, "rsf_25k_threshold_results.rds")
```

8.6.4. RSF (No SMOTE) - 25k Sensitivity Check

```
## Step 3c: RSF (SMOTE) - 25k Sensitivity Check
library(caret)
library(dplyr)
library(smotefamily)
```

```r
library(randomForestSRC)
library(pROC)
library(PRROC)

set.seed(44940076)

n_splits <- 30
threshold_grid <- c(0.90, 0.95, 0.975, 0.99, 0.995)

results_store_25k_rsf_smote <- list()
for (t in threshold_grid) {
  results_store_25k_rsf_smote[[as.character(t)]] <- list(
    sens  = numeric(n_splits),
    prec  = numeric(n_splits),
    auc   = numeric(n_splits),
    auprc = numeric(n_splits)
  )
}

for (i in 1:n_splits) {
  message("RSF SMOTE 25k - Split ", i, " of ", n_splits, " - ", Sys.time())

  # 1. Stratified 70-30 split BEFORE SMOTE
  rsf_smote_25k_train_idx  <- createDataPartition(fraud_subset_25k$status, p = 0.7, list = FALSE)
  rsf_smote_25k_train_data <- fraud_subset_25k[rsf_smote_25k_train_idx, ]
  rsf_smote_25k_test_data  <- fraud_subset_25k[-rsf_smote_25k_train_idx, ]

  # 2. Remove duplicate target column and ensure numeric status
  rsf_smote_25k_train_data <- rsf_smote_25k_train_data %>% dplyr::select(-Class)
  rsf_smote_25k_test_data  <- rsf_smote_25k_test_data  %>% dplyr::select(-Class)

  rsf_smote_25k_train_data$status <- as.numeric(as.character(rsf_smote_25k_train_data$status))
  rsf_smote_25k_test_data$status  <- as.numeric(as.character(rsf_smote_25k_test_data$status))

  # 3. Prepare inputs for SMOTE (exclude time)
  rsf_smote_25k_x_train <- rsf_smote_25k_train_data %>% dplyr::select(-status, -time)
  rsf_smote_25k_y_train <- rsf_smote_25k_train_data$status

  # 4. Choose SMOTE neighbourhood size safely
  rsf_smote_25k_n_min <- min(table(rsf_smote_25k_y_train))
  rsf_smote_25k_k_use <- min(5, max(1, rsf_smote_25k_n_min - 1))

  # 5. Apply SMOTE to training data only
  rsf_smote_25k_output <- SMOTE(
    X        = rsf_smote_25k_x_train,
    target   = rsf_smote_25k_y_train,
    K        = rsf_smote_25k_k_use,
    dup_size = 1
  )

  rsf_smote_25k_balanced        <- rsf_smote_25k_output$data
  rsf_smote_25k_balanced$status <- as.numeric(as.character(rsf_smote_25k_balanced$class))
  rsf_smote_25k_balanced$class  <- NULL
```

```r
# 6. Reattach empirical time values
rsf_smote_25k_balanced$time <- NA_real_

rsf_smote_25k_idx0 <- which(rsf_smote_25k_balanced$status == 0)
rsf_smote_25k_idx1 <- which(rsf_smote_25k_balanced$status == 1)

rsf_smote_25k_balanced$time[rsf_smote_25k_idx0] <- sample(
  rsf_smote_25k_train_data$time[rsf_smote_25k_train_data$status == 0],
  length(rsf_smote_25k_idx0), replace = TRUE
)
rsf_smote_25k_balanced$time[rsf_smote_25k_idx1] <- sample(
  rsf_smote_25k_train_data$time[rsf_smote_25k_train_data$status == 1],
  length(rsf_smote_25k_idx1), replace = TRUE
)

# 7. Fit RSF on SMOTE-balanced training data (ONCE per split)
rsf_smote_25k_model <- rfsrc(
  Surv(time, status) ~ .,
  data      = rsf_smote_25k_balanced,
  ntree     = 100,
  nodesize  = 15,
  splitrule = "logrank",
  nthread   = 4,
  fast      = TRUE
)

# 8. Predict on untouched test data
rsf_smote_25k_test_x <- rsf_smote_25k_test_data[, rsf_smote_25k_model$xvar.names, drop = FALSE]
rsf_smote_25k_pred   <- predict(rsf_smote_25k_model, newdata = rsf_smote_25k_test_x)
rsf_smote_25k_risk   <- as.numeric(1 - rsf_smote_25k_pred$survival[,
                                    ncol(rsf_smote_25k_pred$survival)])

stopifnot(length(rsf_smote_25k_risk) == nrow(rsf_smote_25k_test_data))

# 9. Compute training risk scores (ONCE per split)
rsf_smote_25k_train_pred <- predict(rsf_smote_25k_model,
                                    newdata = rsf_smote_25k_train_data[,
                                              rsf_smote_25k_model$xvar.names, drop = FALSE])
rsf_smote_25k_train_risk <- as.numeric(1 - rsf_smote_25k_train_pred$survival[,
                                         ncol(rsf_smote_25k_train_pred$survival)])

# 10. ROC + AUC (threshold-independent, compute once)
rsf_smote_25k_roc_obj <- roc(
  response  = rsf_smote_25k_test_data$status,
  predictor = rsf_smote_25k_risk,
  quiet     = TRUE
)
rsf_smote_25k_auc_i <- as.numeric(auc(rsf_smote_25k_roc_obj))

# 11. AUPRC (threshold-independent, compute once)
rsf_smote_25k_true_labels <- rsf_smote_25k_test_data$status
rsf_smote_25k_keep        <- is.finite(rsf_smote_25k_risk) &
                             is.finite(rsf_smote_25k_true_labels)
```

```
  rsf_smote_25k_pr_obj <- pr.curve(
    scores.class0 = rsf_smote_25k_risk[rsf_smote_25k_keep][rsf_smote_25k_true_labels[rsf_smote_25k_keep]
    scores.class1 = rsf_smote_25k_risk[rsf_smote_25k_keep][rsf_smote_25k_true_labels[rsf_smote_25k_keep]
    curve = FALSE
  )
  rsf_smote_25k_auprc_i <- rsf_smote_25k_pr_obj$auc.integral

  # 12. Loop over thresholds
  for (t in threshold_grid) {

    rsf_smote_25k_threshold <- quantile(rsf_smote_25k_train_risk, probs = t, na.rm = TRUE)
    rsf_smote_25k_test_pred <- ifelse(rsf_smote_25k_risk > rsf_smote_25k_threshold, 1, 0)

    rsf_smote_25k_cm <- confusionMatrix(
      factor(rsf_smote_25k_test_pred,        levels = c(0, 1)),
      factor(rsf_smote_25k_test_data$status, levels = c(0, 1)),
      positive = "1"
    )

    tk <- as.character(t)
    results_store_25k_rsf_smote[[tk]]$sens[i]  <- rsf_smote_25k_cm$byClass["Sensitivity"]
    results_store_25k_rsf_smote[[tk]]$prec[i]  <- rsf_smote_25k_cm$byClass["Precision"]
    results_store_25k_rsf_smote[[tk]]$auc[i]   <- rsf_smote_25k_auc_i
    results_store_25k_rsf_smote[[tk]]$auprc[i] <- rsf_smote_25k_auprc_i
  }

  rm(rsf_smote_25k_model, rsf_smote_25k_pred, rsf_smote_25k_train_pred,
     rsf_smote_25k_balanced)
  gc()
}

## Compile results
results_list_25k_rsf_smote <- list()
for (t in threshold_grid) {
  tk <- as.character(t)
  results_list_25k_rsf_smote[[tk]] <- data.frame(
    Threshold      = t,
    Sensitivity    = mean(results_store_25k_rsf_smote[[tk]]$sens),
    Sensitivity_SD = sd(results_store_25k_rsf_smote[[tk]]$sens),
    Precision      = mean(results_store_25k_rsf_smote[[tk]]$prec),
    Precision_SD   = sd(results_store_25k_rsf_smote[[tk]]$prec),
    AUC            = mean(results_store_25k_rsf_smote[[tk]]$auc),
    AUC_SD         = sd(results_store_25k_rsf_smote[[tk]]$auc),
    AUPRC          = mean(results_store_25k_rsf_smote[[tk]]$auprc),
    AUPRC_SD       = sd(results_store_25k_rsf_smote[[tk]]$auprc)
  )
}

results_25k_rsf_smote_threshold <- do.call(rbind, results_list_25k_rsf_smote)
auprc_25k_rsf_smote_splits <- results_store_25k_rsf_smote[["0.99"]]$auprc

saveRDS(results_25k_rsf_smote_threshold, "rsf_smote_25k_threshold_results.rds")
```

8.6.5. RSF (SMOTE) - 25k Sensitivity Check

8.6.6. Summary of 25k Summary

```
## Step 5: Summary table comparing all four configurations at 0.99 threshold
library(pander)

comparison_25k_vs_50k <- data.frame(
  Model = c(
    "LR (No SMOTE) - 50k",
    "LR (No SMOTE) - 25k",
    "LR (SMOTE) - 50k",
    "LR (SMOTE) - 25k",
    "RSF (No SMOTE) - 50k",
    "RSF (No SMOTE) - 25k",
    "RSF (SMOTE) - 50k",
    "RSF (SMOTE) - 25k"
  ),
  AUPRC = c(
    results_lr_threshold[results_lr_threshold$Threshold == 0.99, "AUPRC"],
    results_25k_lr_threshold[results_25k_lr_threshold$Threshold == 0.99, "AUPRC"],
    results_lr_smote_threshold[results_lr_smote_threshold$Threshold == 0.99, "AUPRC"],
    results_25k_lr_smote_threshold[results_25k_lr_smote_threshold$Threshold == 0.99, "AUPRC"],
    results_rsf_threshold[results_rsf_threshold$Threshold == 0.99, "AUPRC"],
    results_25k_rsf_threshold[results_25k_rsf_threshold$Threshold == 0.99, "AUPRC"],
    results_rsf_smote_threshold[results_rsf_smote_threshold$Threshold == 0.99, "AUPRC"],
    results_25k_rsf_smote_threshold[results_25k_rsf_smote_threshold$Threshold == 0.99, "AUPRC"]
  ),
  AUPRC_SD = c(
    results_lr_threshold[results_lr_threshold$Threshold == 0.99, "AUPRC_SD"],
    results_25k_lr_threshold[results_25k_lr_threshold$Threshold == 0.99, "AUPRC_SD"],
    results_lr_smote_threshold[results_lr_smote_threshold$Threshold == 0.99, "AUPRC_SD"],
    results_25k_lr_smote_threshold[results_25k_lr_smote_threshold$Threshold == 0.99, "AUPRC_SD"],
    results_rsf_threshold[results_rsf_threshold$Threshold == 0.99, "AUPRC_SD"],
    results_25k_rsf_threshold[results_25k_rsf_threshold$Threshold == 0.99, "AUPRC_SD"],
    results_rsf_smote_threshold[results_rsf_smote_threshold$Threshold == 0.99, "AUPRC_SD"],
    results_25k_rsf_smote_threshold[results_25k_rsf_smote_threshold$Threshold == 0.99, "AUPRC_SD"]
  )
)

pander(comparison_25k_vs_50k, digits = 4,
       caption = "Downsampling Sensitivity Check: 25k vs 50k AUPRC at 0.99 Threshold")
```

Table 21: Downsampling Sensitivity Check: 25k vs 50k AUPRC at 0.99 Threshold

Model	AUPRC	AUPRC_SD
LR (No SMOTE) - 50k	0.8661	0.02069
LR (No SMOTE) - 25k	0.8915	0.01897
LR (SMOTE) - 50k	0.8676	0.02427
LR (SMOTE) - 25k	0.8892	0.01988
RSF (No SMOTE) - 50k	0.8589	0.02622
RSF (No SMOTE) - 25k	0.8925	0.01676

Model	AUPRC	AUPRC_SD
RSF (SMOTE) - 50k	0.8683	0.02011
RSF (SMOTE) - 25k	0.8949	0.02268

9. Explainability

9.1. Why Explainability Matters in Fraud Detection

In fraud detection, model performance alone is not sufficient. Banking environments require models that are not only effective, but also interpretable. Explanations are important for several reasons: they help analysts understand why a transaction has been flagged, support internal validation and governance processes, and improve confidence in model-assisted decision-making.

This is particularly relevant in the present study, where the Random Survival Forest produces a risk-based score rather than a simple class probability. Explainability methods therefore help connect the model output to transaction-level reasoning.

9.2. Global Explainability

Global explainability provides an overall view of which predictors contribute most strongly to the model's behaviour across the dataset. This is useful for identifying broad fraud-related patterns and determining whether the model is relying on a concentrated set of meaningful features.

To enable computation of variable importance with 95% confidence intervals across repeated splits, a stratified subsample of the training data was used in each iteration. All fraud cases were retained to preserve the minority class signal, while 10000 non-fraud cases were randomly sampled to reduce computational overhead. This objective of this subsampling is to identify the relative contribution of each feature, rather than to optimise predictive performance. The importance values were computed using random permutation across 30 repeated stratified splits, with 50 trees per forest.

```
library(caret)
library(dplyr)
library(randomForestSRC)
library(ggplot2)

set.seed(44940076)

n_splits <- 30

importance_list <- vector("list", n_splits)

for (i in 1:n_splits) {
  message("Importance split ", i, " of ", n_splits, " - ", Sys.time())

  # 1. Stratified 70-30 split
  rsf_imp_train_idx <- createDataPartition(fraud_subset$status, p = 0.7, list = FALSE)
  rsf_imp_train_data <- fraud_subset[rsf_imp_train_idx, ]

  # 2. Prepare data
  rsf_imp_train_data <- rsf_imp_train_data %>% dplyr::select(-Class)
  rsf_imp_train_data$status <- as.numeric(as.character(rsf_imp_train_data$status))

  # 3. Subsample for computational efficiency (keep all fraud cases)
  rsf_imp_fraud <- rsf_imp_train_data[rsf_imp_train_data$status == 1, ]
  rsf_imp_nonfraud <- rsf_imp_train_data[rsf_imp_train_data$status == 0, ]
```

```
  rsf_imp_nonfraud_sample <- rsf_imp_nonfraud[sample(nrow(rsf_imp_nonfraud),
                                                     min(10000, nrow(rsf_imp_nonfraud))), ]
  rsf_imp_subsample <- rbind(rsf_imp_fraud, rsf_imp_nonfraud_sample)

  # 4. Fit RSF with importance on subsample
  rsf_imp_model <- rfsrc(
    Surv(time, status) ~ .,
    data = rsf_imp_subsample,
    ntree = 50,
    nodesize = 15,
    splitrule = "logrank",
    nthread = 4,
    fast = TRUE,
    importance = "random"
  )

  # 5. Store importance
  importance_list[[i]] <- rsf_imp_model$importance
}
```

```
## Importance split 1 of 30 - 2026-04-29 16:07:04.783748
## Importance split 2 of 30 - 2026-04-29 16:07:49.310304
## Importance split 3 of 30 - 2026-04-29 16:08:27.806307
## Importance split 4 of 30 - 2026-04-29 16:09:07.622479
## Importance split 5 of 30 - 2026-04-29 16:09:46.167712
## Importance split 6 of 30 - 2026-04-29 16:10:25.690509
## Importance split 7 of 30 - 2026-04-29 16:11:04.561664
## Importance split 8 of 30 - 2026-04-29 16:11:43.025817
## Importance split 9 of 30 - 2026-04-29 16:12:21.595314
## Importance split 10 of 30 - 2026-04-29 16:13:00.234533
## Importance split 11 of 30 - 2026-04-29 16:13:44.666454
## Importance split 12 of 30 - 2026-04-29 16:14:53.357421
## Importance split 13 of 30 - 2026-04-29 16:16:01.356076
## Importance split 14 of 30 - 2026-04-29 16:17:10.563594
## Importance split 15 of 30 - 2026-04-29 16:18:19.449395
## Importance split 16 of 30 - 2026-04-29 16:19:25.73885
## Importance split 17 of 30 - 2026-04-29 16:20:35.220377
## Importance split 18 of 30 - 2026-04-29 16:21:48.089132
## Importance split 19 of 30 - 2026-04-29 16:23:01.34373
## Importance split 20 of 30 - 2026-04-29 16:24:14.632142
## Importance split 21 of 30 - 2026-04-29 16:25:27.523627
## Importance split 22 of 30 - 2026-04-29 16:26:40.814702
## Importance split 23 of 30 - 2026-04-29 16:27:53.611583
```

```
## Importance split 24 of 30 - 2026-04-29 16:29:06.583404
## Importance split 25 of 30 - 2026-04-29 16:30:19.497912
## Importance split 26 of 30 - 2026-04-29 16:31:32.435056
## Importance split 27 of 30 - 2026-04-29 16:32:44.503701
## Importance split 28 of 30 - 2026-04-29 16:33:59.23345
## Importance split 29 of 30 - 2026-04-29 16:35:11.66562
## Importance split 30 of 30 - 2026-04-29 16:36:24.815369
```

```r
# Combine into a matrix
importance_matrix <- do.call(rbind, importance_list)

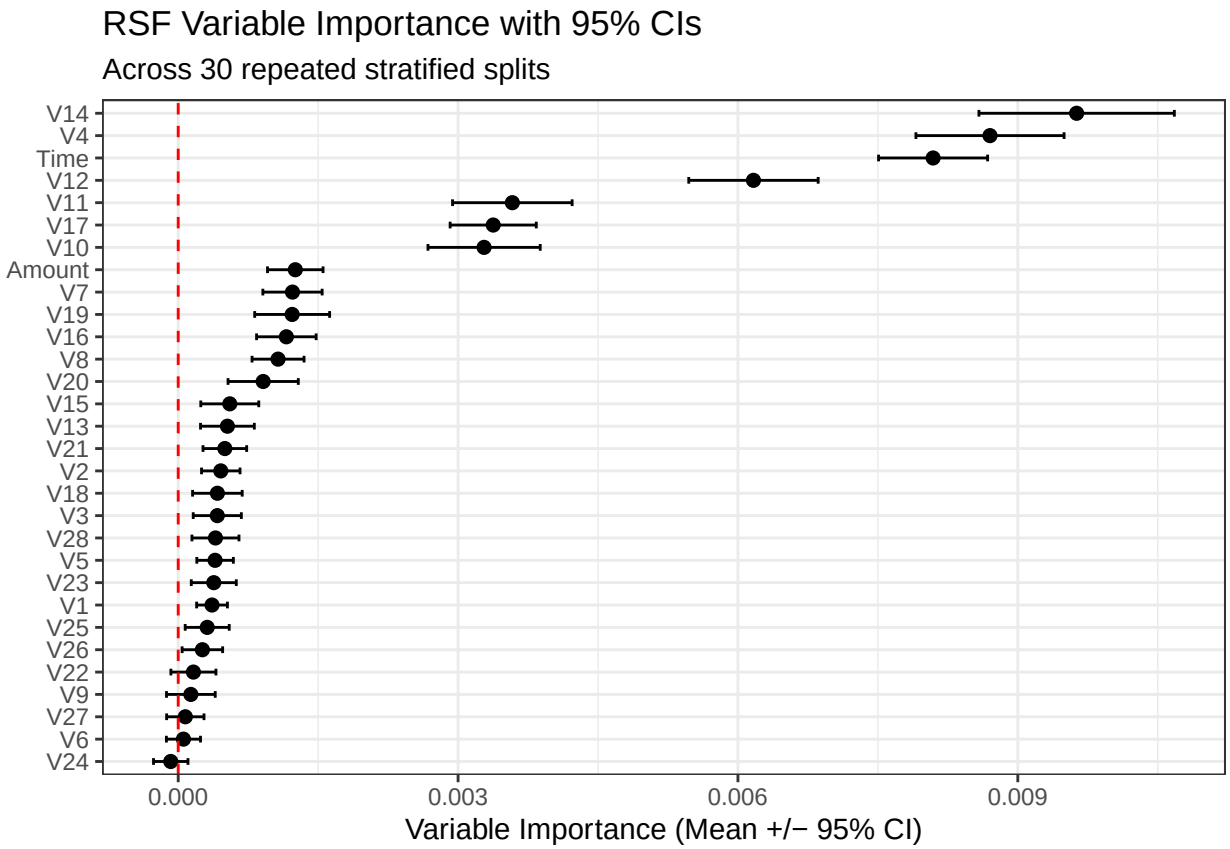
# Compute mean and 95% CI
importance_summary <- data.frame(
  Feature  = colnames(importance_matrix),
  Mean     = colMeans(importance_matrix),
  SD       = apply(importance_matrix, 2, sd)
)

importance_summary$SE <- importance_summary$SD / sqrt(n_splits)
importance_summary$CI_lower <- importance_summary$Mean - 1.96 * importance_summary$SE
importance_summary$CI_upper <- importance_summary$Mean + 1.96 * importance_summary$SE

importance_summary <- importance_summary %>%
  arrange(Mean) %>%
  mutate(Feature = factor(Feature, levels = Feature))

# Forest plot
ggplot(importance_summary, aes(x = Mean, y = Feature)) +
  geom_point(size = 2) +
  geom_errorbar(aes(xmin = CI_lower, xmax = CI_upper),
                height = 0.3,
                orientation = "y") +
  geom_vline(xintercept = 0, linetype = "dashed", colour = "red") +
  labs(
    title = "RSF Variable Importance with 95% CIs",
    subtitle = paste0("Across ", n_splits, " repeated stratified splits"),
    x = "Variable Importance (Mean +/- 95% CI)",
    y = NULL
  ) +
  theme_bw()
```

```
## `height` was translated to `width`.
```

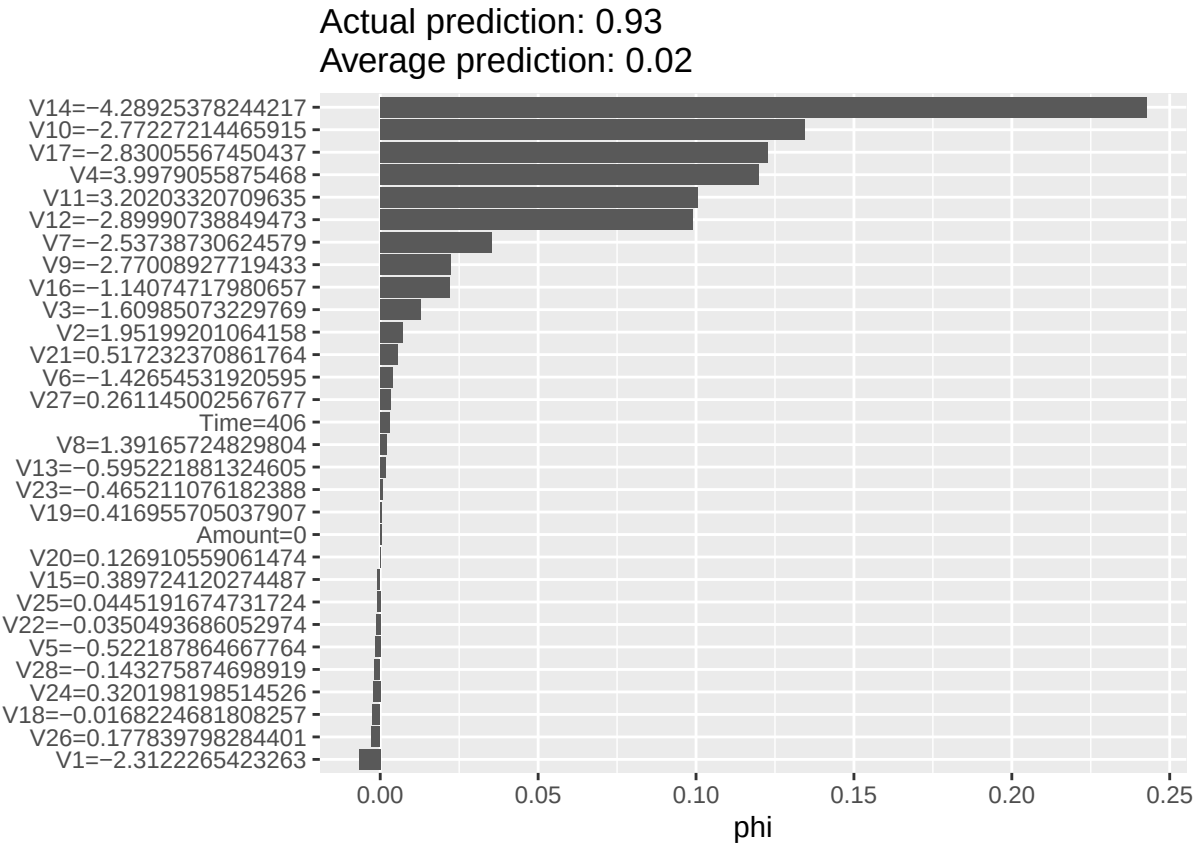


Observations: The variable importance analysis reveals a clear tiered structure in terms of feature contributions to the RSF model. V14 and V4 emerge as the dominant predictors, with importance values roughl three times higher than any other feature. This suggests that the model's fraud detection capability is heavily concentrated towards these two variables. A second tier comprising Time, V12, V11, V17, and V10 provides a meaningful, but more moderate contributions, with all confidence intervals comfortably above zero. Below these, a cluster of marginal contributors including V7, V19, V16, V8, Amount, and V20 are statistically significant but practically small in their influence. The remaining features show little to no reliable contribution, with many confidence intervals crossing or touching the zero line.

Notably, the proxy Time index ranks among the top five important features, which raises a methodological concern (and opportunity). Since this variable is constructed by row ordering, rather than as a genuine temporal measure, its prominence suggests that the RSF may be exploiting an artificial structural pattern in the data, rather than learning meaningful time-to-event relationships. This further reinforces the need to evaluate the RSF framework on a dataset with genuine temporal structure, where the importance of time-related features would carry substantive meaning.

9.3. Local Explainability using SHAP

While global importance identifies which variables matter overall, SHAP provides a local explanation for a single transaction. This allows the contribution of each feature to the RSF risk score to be decomposed at the transaction level. In this study, SHAP is applied to the RSF model with SMOTE in order to examine how individual predictors increase or decrease the estimated fraud risk for a selected observation.

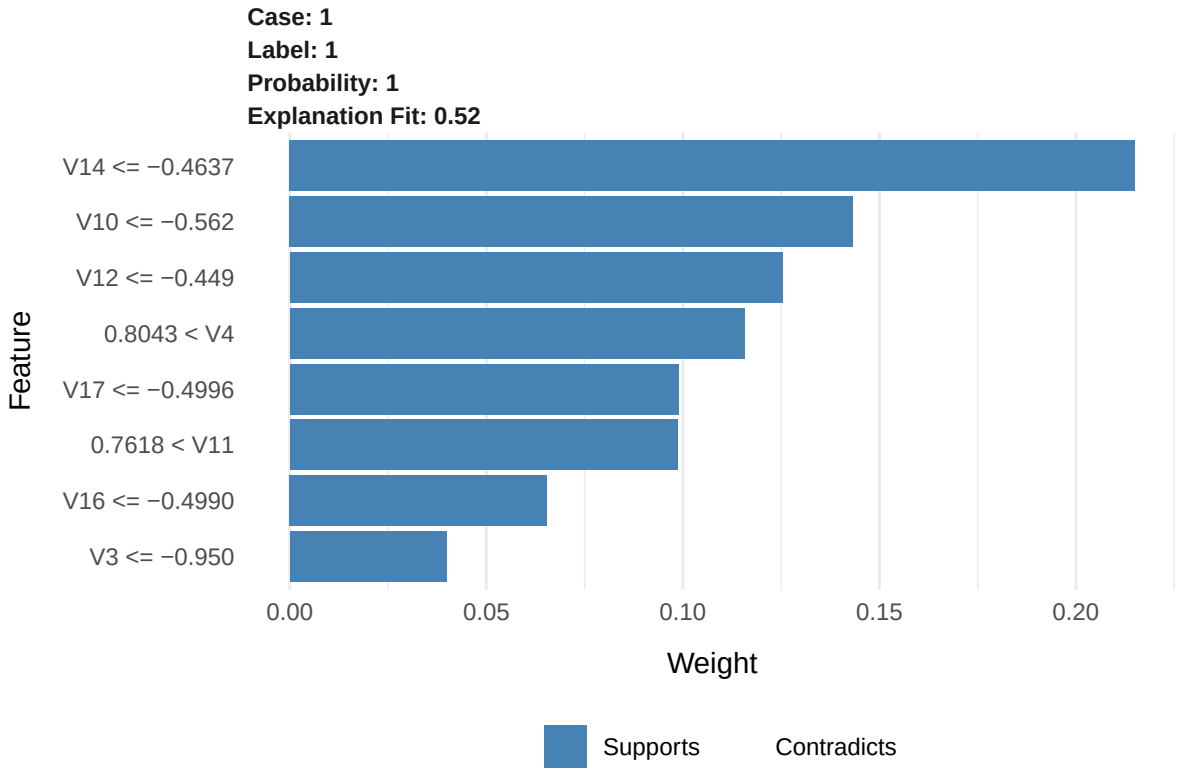


The SHAP explanation shows how the selected transaction's risk score is built up from individual feature contributions. Positive contributions move the transaction toward higher estimated fraud risk, while negative contributions move it toward lower estimated risk.

From a banking perspective, this is useful because it mirrors the logic of case investigation. Rather than receiving only a risk score, an analyst can see which characteristics of the transaction most strongly influenced the model's assessment.

9.4. Local Explainability using LIME

Local Interpretable Model-agnostic Explanations (LIME) provides a second local explanation framework by approximating the model decision around a selected transaction with a simpler interpretable representation. In this study, LIME is used to complement SHAP by showing which features most strongly support the local fraud classification for an individual case.



LIME provides a complementary local explanation framework by approximating the behaviour of the RSF model in the neighbourhood of a selected transaction using a simpler, interpretable model. This allows the contribution of a small number of features to be expressed in a linear and human-readable form.

The LIME output shows both positive and negative contributions. Features with positive weights increase the likelihood of the transaction being classified as fraudulent, while those with negative weights reduce it. This provides a directional explanation of the decision, rather than simply identifying important variables.

While SHAP provides a theoretically grounded decomposition of the prediction, LIME offers a simpler and more interpretable local approximation. The two methods therefore complement each other, with SHAP emphasising consistency and LIME emphasising interpretability.

From a banking perspective, this type of explanation aligns closely with how fraud analysts review cases. This directional interpretation allows analysts to understand not only which features matter, but how they influence the model's decision. This supports faster investigation and more informed decision-making.

However, it is important to note that LIME explanations are based on a local approximation of the model and may not fully capture complex interactions present in the RSF model. As a result, explanations should be interpreted as indicative rather than exact. Additionally, given that the RSF and logistic regression models produced near-identical performance on the ULB dataset, the explainability outputs here reflect a model operating without a meaningful survival structure advantage. The interpretive value of these methods may become more distinctive when applied to a dataset where the RSF's temporal modelling is more meaningfully leveraged, allowing SHAP and LIME to surface feature contributions that are unique to the survival framework rather than shared with simpler classification approaches.

9.5. Banking Relevance of Explainability

The explainability results strengthen the practical relevance of the modelling framework. In a banking environment, fraud detection systems are not used in isolation - they support fraud operations teams who must review alerts, prioritise investigations, and justify intervention decisions.

Global explainability helps identify the broad drivers of suspicious behaviour across the portfolio, while local explanation methods such as SHAP and LIME provide transaction-level insight into why a case was flagged. This is particularly valuable for analyst workflows, model governance, and customer-facing situations where a flagged transaction may require review or escalation.

More broadly, the inclusion of explainability addresses one of the key limitations of black-box fraud models. Rather than producing only a ranked risk score, the framework developed in this study also provides a mechanism for understanding how that score was formed.

10. Random Forest

Additionally, we experiment with the Random Forest (RF) framework, in order to see this as a difference from the RSF from another lens.

10.1. Random Forest (Classification, No SMOTE) - Downsampled

```
## Step 1: Run relevant packages
library(caret)
library(dplyr)
library(randomForestSRC)
library(pROC)
library(PRROC)

## Step 2: Set the seed
set.seed(44940076)

## Step 3: Establish the number of splits and threshold grid
n_splits <- 30
threshold_grid <- c(0.90, 0.95, 0.975, 0.99, 0.995)

## Step 4: Create storage - one set of vectors per threshold
results_store <- list()
for (t in threshold_grid) {
  results_store[[as.character(t)]] <- list(
    sens = numeric(n_splits),
    prec = numeric(n_splits),
    auc = numeric(n_splits),
    auprc = numeric(n_splits)
  )
}

## Step 5: Execute the Random Forest (Classification, No SMOTE)
for (i in 1:n_splits) {
  message("Split ", i, " of ", n_splits, " - ", Sys.time())

  # 1. Stratified 70-30 split
  rf_train_idx <- createDataPartition(fraud_subset$status, p = 0.7, list = FALSE)
  rf_train_data <- fraud_subset[rf_train_idx, ]
  rf_test_data  <- fraud_subset[-rf_train_idx, ]
```

```r
# 2. Keep only RF-relevant variables (drop survival-specific columns)
rf_train_data <- rf_train_data %>% dplyr::select(-time, -status)
rf_test_data  <- rf_test_data  %>% dplyr::select(-time, -status)

# 3. Ensure Class is a factor for classification mode
rf_train_data$Class <- factor(rf_train_data$Class, levels = c(0, 1))
rf_test_data$Class  <- factor(rf_test_data$Class,  levels = c(0, 1))

# 4. Fit RF classification model (ONCE per split)
rf_model <- rfsrc(
  Class ~ .,
  data = rf_train_data,
  ntree = 100,
  nodesize = 15,
  splitrule = "gini",
  nthread = 4,
  fast = TRUE
)

# 5. Predict on test data
rf_test_x <- rf_test_data[, rf_model$xvar.names, drop = FALSE]
rf_pred   <- predict(rf_model, newdata = rf_test_x)
rf_prob   <- rf_pred$predicted[, "1"]

# 6. Predict on training data (for threshold calibration)
rf_train_pred <- predict(rf_model,
                         newdata = rf_train_data[, rf_model$xvar.names, drop = FALSE])
rf_train_prob <- rf_train_pred$predicted[, "1"]

# 7. ROC + AUC (threshold-independent, compute once)
rf_true_labels <- as.numeric(as.character(rf_test_data$Class))
rf_auc_i <- as.numeric(auc(
  roc(response = rf_true_labels, predictor = rf_prob, quiet = TRUE)
))

# 8. AUPRC (threshold-independent, compute once)
rf_pr_obj <- pr.curve(
  scores.class0 = rf_prob[rf_true_labels == 1],
  scores.class1 = rf_prob[rf_true_labels == 0],
  curve = FALSE
)
rf_auprc_i <- rf_pr_obj$auc.integral

# 9. Loop over thresholds
for (t in threshold_grid) {
  rf_threshold <- quantile(rf_train_prob, probs = t, na.rm = TRUE)
  rf_test_pred <- ifelse(rf_prob > rf_threshold, 1, 0)

  rf_cm <- confusionMatrix(
    factor(rf_test_pred, levels = c(0, 1)),
    factor(rf_true_labels, levels = c(0, 1)),
    positive = "1"
  )
```

```
    tk <- as.character(t)
    results_store[[tk]]$sens[i]  <- rf_cm$byClass["Sensitivity"]
    results_store[[tk]]$prec[i]  <- rf_cm$byClass["Precision"]
    results_store[[tk]]$auc[i]   <- rf_auc_i
    results_store[[tk]]$auprc[i] <- rf_auprc_i
  }

  # 10. Clean up
  rm(rf_model, rf_pred, rf_train_pred)
  gc()
}
```

```
## Split 1 of 30 - 2026-08-16 19:54:34.803806
## Split 2 of 30 - 2026-08-16 19:54:47.63415
## Split 3 of 30 - 2026-08-16 19:55:00.568277
## Split 4 of 30 - 2026-08-16 19:55:12.843901
## Split 5 of 30 - 2026-08-16 19:55:25.542167
## Split 6 of 30 - 2026-08-16 19:55:38.43796
## Split 7 of 30 - 2026-08-16 19:55:51.120261
## Split 8 of 30 - 2026-08-16 19:56:03.869676
## Split 9 of 30 - 2026-08-16 19:56:16.731995
## Split 10 of 30 - 2026-08-16 19:56:29.392418
## Split 11 of 30 - 2026-08-16 19:56:42.299364
## Split 12 of 30 - 2026-08-16 19:56:55.339809
## Split 13 of 30 - 2026-08-16 19:57:08.484235
## Split 14 of 30 - 2026-08-16 19:57:21.210875
## Split 15 of 30 - 2026-08-16 19:57:33.345436
## Split 16 of 30 - 2026-08-16 19:57:45.837986
## Split 17 of 30 - 2026-08-16 19:57:58.719323
## Split 18 of 30 - 2026-08-16 19:58:11.127305
## Split 19 of 30 - 2026-08-16 19:58:23.533695
## Split 20 of 30 - 2026-08-16 19:58:35.868774
## Split 21 of 30 - 2026-08-16 19:58:48.821638
## Split 22 of 30 - 2026-08-16 19:59:01.591031
## Split 23 of 30 - 2026-08-16 19:59:14.384235
## Split 24 of 30 - 2026-08-16 19:59:27.071367
## Split 25 of 30 - 2026-08-16 19:59:39.85013
## Split 26 of 30 - 2026-08-16 19:59:52.31742
## Split 27 of 30 - 2026-08-16 20:00:05.163917
## Split 28 of 30 - 2026-08-16 20:00:17.841179
```

```
## Split 29 of 30 - 2026-08-16 20:00:30.53348
## Split 30 of 30 - 2026-08-16 20:00:42.925741
```

```
## Step 6: Store results
results_list <- list()
for (t in threshold_grid) {
  tk <- as.character(t)
  results_list[[tk]] <- data.frame(
    Threshold = t,
    Sensitivity = mean(results_store[[tk]]$sens),
    Sensitivity_SD = sd(results_store[[tk]]$sens),
    Precision = mean(results_store[[tk]]$prec),
    Precision_SD = sd(results_store[[tk]]$prec),
    AUC = mean(results_store[[tk]]$auc),
    AUC_SD = sd(results_store[[tk]]$auc),
    AUPRC = mean(results_store[[tk]]$auprc),
    AUPRC_SD = sd(results_store[[tk]]$auprc)
  )
}
results_rf_threshold <- do.call(rbind, results_list)
saveRDS(results_rf_threshold, here::here("rf_classification_results.rds"))
```

```
pander(results_rf_threshold, digits = 6,
       caption = "Random Forest (Classification, No SMOTE) Performance Metrics")
```

Table 22: Random Forest (Classification, No SMOTE) Performance Metrics (continued below)

	Threshold	Sensitivity	Sensitivity_SD	Precision
0.9	0.9	0.941993	0.0151195	0.0817849
0.95	0.95	0.921247	0.0169302	0.156008
0.975	0.975	0.903517	0.0215329	0.313743
0.99	0.99	0.857662	0.0246005	0.872177
0.995	0.995	0.48254	0.0587764	0.989115

	Precision_SD	AUC	AUC_SD	AUPRC	AUPRC_SD
0.9	0.00499644	0.975714	0.00648885	0.88127	0.0196658
0.95	0.0102233	0.975714	0.00648885	0.88127	0.0196658
0.975	0.0196849	0.975714	0.00648885	0.88127	0.0196658
0.99	0.024922	0.975714	0.00648885	0.88127	0.0196658
0.995	0.013953	0.975714	0.00648885	0.88127	0.0196658

Observations: RF (Classification, No SMOTE) achieved an AUC of 0.976 and AUPRC of 0.881, which were both marginally above every configuration reported in Section 8 - LR (No SMOTE) at 0.979/0.866, LR (SMOTE) at 0.979/0.868, RSF (No SMOTE) at 0.976/0.858, and RSF (SMOTE) at 0.977/0.868. At the 0.99 threshold, RF delivers 85.8% sensitivity at 87.2% precision - a meaningfully better precision than any of the four primary configurations (82.1-86.6%), while sensitivity sits within the same narrow band as LR and RSF (85.6-85.9%).

This is interesting because with RF (sharing RSF's tree-ensemble structure and hyperparameters but with no survival framing), proxy time variable, or censoring mechanism at all, performs comparable (and in some cases, better) on precision than every survival and classification configuration in the primary analysis.

This reinforces the central null finding from a different angle in that, given RF has nothing resembling a time-to-event structure to exploit, its strong performance here confirms that the discriminative power that exists in this dataset comes from the feature space (V1-V28, Amount) itself, and not from any temporal or survival-specific signal. This is consistent with RSF gaining no advantage from its survival framing, since a model with no survival framing at all performs just as well or better. Sensitivity SDs (0.015-0.059) are broadly comparable to the RSF configurations rather than the tighter LR results. This suggests that the tree-ensemble structure itself is shared by RF and RSF, and this is the source of the somewhat higher split-to-split variability observed throughout Section 8, rather than anything specific to the survival object.

10.2. Random Forest (Classification, SMOTE) - Full Dataset

```
## Step 1: Run relevant packages
library(caret)
library(dplyr)
library(smotefamily)
library(randomForestSRC)
library(pROC)
library(PRROC)

## Step 2: Set the seed
set.seed(44940076)

## Step 3: Establish the number of splits and threshold grid
n_splits <- 10
threshold_grid <- c(0.90, 0.95, 0.975, 0.99, 0.995)

## Step 4: Create storage - one set of vectors per threshold
results_store_full <- list()
for (t in threshold_grid) {
  results_store_full[[as.character(t)]] <- list(
    sens = numeric(n_splits),
    prec = numeric(n_splits),
    auc = numeric(n_splits),
    auprc = numeric(n_splits)
  )
}

## Step 5: Execute the Random Forest (Classification, SMOTE) - Full Dataset
for (i in 1:n_splits) {
  message("Split ", i, " of ", n_splits, " - ", Sys.time())

  # 1. Stratified 70-30 split BEFORE SMOTE
  rf_smote_full_train_idx <- createDataPartition(fraud_data$status, p = 0.7, list = FALSE)
  rf_smote_full_train_data <- fraud_data[rf_smote_full_train_idx, ]
  rf_smote_full_test_data  <- fraud_data[-rf_smote_full_train_idx, ]

  # 2. Drop survival-specific columns (proxy time index and duplicate status)
  rf_smote_full_train_data <- rf_smote_full_train_data %>% dplyr::select(-time, -status)
  rf_smote_full_test_data  <- rf_smote_full_test_data  %>% dplyr::select(-time, -status)

  # 3. Ensure numeric 0/1 outcome for SMOTE target
  rf_smote_full_train_data$Class <- as.numeric(as.character(rf_smote_full_train_data$Class))
  rf_smote_full_test_data$Class  <- factor(rf_smote_full_test_data$Class, levels = c(0, 1))
```

```r
# 4. Prepare inputs for SMOTE
rf_smote_full_x_train <- rf_smote_full_train_data %>% dplyr::select(-Class)
rf_smote_full_y_train <- rf_smote_full_train_data$Class

# 5. Choose SMOTE neighbourhood size safely
rf_smote_full_n_min <- min(table(rf_smote_full_y_train))
rf_smote_full_k_use <- min(5, max(1, rf_smote_full_n_min - 1))

# 6. Apply SMOTE to training data only
rf_smote_full_output <- SMOTE(
  X = rf_smote_full_x_train,
  target = rf_smote_full_y_train,
  K = rf_smote_full_k_use,
  dup_size = 1
)

rf_smote_full_balanced <- rf_smote_full_output$data
rf_smote_full_balanced$Class <- factor(
  as.numeric(as.character(rf_smote_full_balanced$class)), levels = c(0, 1)
)
rf_smote_full_balanced$class <- NULL

# 7. Fit RF classification model on SMOTE-balanced training data (ONCE per split)
rf_smote_full_model <- rfsrc(
  Class ~ .,
  data = rf_smote_full_balanced,
  ntree = 100,
  nodesize = 15,
  splitrule = "gini",
  nthread = 4,
  fast = TRUE
)

# 8. Predict on untouched test data
rf_smote_full_test_x <- rf_smote_full_test_data[, rf_smote_full_model$xvar.names, drop = FALSE]
rf_smote_full_pred   <- predict(rf_smote_full_model, newdata = rf_smote_full_test_x)
rf_smote_full_prob   <- rf_smote_full_pred$predicted[, "1"]

# 9. Predict on SMOTE-balanced training data (for threshold calibration)
rf_smote_full_train_pred <- predict(rf_smote_full_model,
  newdata = rf_smote_full_balanced[, rf_smote_full_model$xvar.names, drop = FALSE])
rf_smote_full_train_prob <- rf_smote_full_train_pred$predicted[, "1"]

# 10. ROC + AUC (threshold-independent, compute once)
rf_smote_full_true_labels <- as.numeric(as.character(rf_smote_full_test_data$Class))
rf_smote_full_auc_i <- as.numeric(auc(
  roc(response = rf_smote_full_true_labels, predictor = rf_smote_full_prob, quiet = TRUE)
))

# 11. AUPRC (threshold-independent, compute once)
rf_smote_full_pr_obj <- pr.curve(
  scores.class0 = rf_smote_full_prob[rf_smote_full_true_labels == 1],
  scores.class1 = rf_smote_full_prob[rf_smote_full_true_labels == 0],
```

```
    curve = FALSE
  )
  rf_smote_full_auprc_i <- rf_smote_full_pr_obj$auc.integral

  # 12. Loop over thresholds
  for (t in threshold_grid) {
    rf_smote_full_threshold <- quantile(rf_smote_full_train_prob, probs = t, na.rm = TRUE)
    rf_smote_full_test_pred <- ifelse(rf_smote_full_prob > rf_smote_full_threshold, 1, 0)

    rf_smote_full_cm <- confusionMatrix(
      factor(rf_smote_full_test_pred, levels = c(0, 1)),
      factor(rf_smote_full_true_labels, levels = c(0, 1)),
      positive = "1"
    )

    tk <- as.character(t)
    results_store_full[[tk]]$sens[i]  <- rf_smote_full_cm$byClass["Sensitivity"]
    results_store_full[[tk]]$prec[i]  <- rf_smote_full_cm$byClass["Precision"]
    results_store_full[[tk]]$auc[i]   <- rf_smote_full_auc_i
    results_store_full[[tk]]$auprc[i] <- rf_smote_full_auprc_i
  }

  # 13. Clean up large objects to free memory
  rm(rf_smote_full_model, rf_smote_full_pred, rf_smote_full_train_pred,
     rf_smote_full_train_data, rf_smote_full_test_data, rf_smote_full_balanced)
  gc()
}
```

```
## Split 1 of 10 - 2026-08-16 20:00:56.300401

## Split 2 of 10 - 2026-08-16 20:02:16.383479

## Split 3 of 10 - 2026-08-16 20:03:36.043196

## Split 4 of 10 - 2026-08-16 20:04:53.855641

## Split 5 of 10 - 2026-08-16 20:06:12.792863

## Split 6 of 10 - 2026-08-16 20:07:32.26589

## Split 7 of 10 - 2026-08-16 20:08:50.986327

## Split 8 of 10 - 2026-08-16 20:10:13.211435

## Split 9 of 10 - 2026-08-16 20:11:32.69513

## Split 10 of 10 - 2026-08-16 20:12:52.966216
```

```
## Step 6: Store results
results_list_full <- list()
for (t in threshold_grid) {
  tk <- as.character(t)
  results_list_full[[tk]] <- data.frame(
    Threshold = t,
    Sensitivity = mean(results_store_full[[tk]]$sens),
    Sensitivity_SD = sd(results_store_full[[tk]]$sens),
    Precision = mean(results_store_full[[tk]]$prec),
    Precision_SD = sd(results_store_full[[tk]]$prec),
    AUC = mean(results_store_full[[tk]]$auc),
```

```
    AUC_SD = sd(results_store_full[[tk]]$auc),
    AUPRC = mean(results_store_full[[tk]]$auprc),
    AUPRC_SD = sd(results_store_full[[tk]]$auprc)
  )
}
results_rf_smote_full_threshold <- do.call(rbind, results_list_full)
saveRDS(results_rf_smote_full_threshold, here::here("rf_smote_full_results.rds"))
```

```
pander(results_rf_smote_full_threshold, digits = 6,
       caption = "Random Forest (Classification, SMOTE) Full Dataset Performance Metrics")
```

Table 24: Random Forest (Classification, SMOTE) Full Dataset Performance Metrics (continued below)

	Threshold	Sensitivity	Sensitivity_SD	Precision
0.9	0.9	0.937892	0.0100415	0.0155563
0.95	0.95	0.920879	0.0121379	0.0295688
0.975	0.975	0.904929	0.012689	0.0576201
0.99	0.99	0.88763	0.0162766	0.156535
0.995	0.995	0.866005	0.0190726	0.430495

	Precision_SD	AUC	AUC_SD	AUPRC	AUPRC_SD
0.9	0.00138701	0.970185	0.00512624	0.846198	0.010325
0.95	0.00262361	0.970185	0.00512624	0.846198	0.010325
0.975	0.00467651	0.970185	0.00512624	0.846198	0.010325
0.99	0.0114178	0.970185	0.00512624	0.846198	0.010325
0.995	0.0140533	0.970185	0.00512624	0.846198	0.010325

Observations: RF (Classification, SMOTE) on the full dataset achieved an AUC of 0.970 and AUPRC of 0.846, which was comfortably above the full-dataset LR benchmarks (AUPRC 0.760-0.764, Section 8.4.1-8.4.2) and modestly above RSF (SMOTE)'s own full-dataset AUPRC of 0.832 (Section 8.4.4). At the 0.99 threshold, RF delivers 88.8% sensitivity, the highest of any configuration reported in this notebook at this operating point, though at a substantially lower precision (15.7%) than RF's own downsampled configuration (Section 10.1: 87.2%). This comes as a direct consequence of evaluating against the full dataset's far more extreme 0.17% base rate rather than the approximately 1% rate in the downsampled working data, which mechanically inflates false positives at any fixed sensitivity level.

This result supports the finding already established in Section 8.5's simulation analysis and the full-dataset SMOTE robustness check in Section 8.4. RSF (SMOTE)'s full-dataset AUPRC advantage over LR (0.831 vs 0.760-0.769) was investigated there and found not to reflect genuine exploitation of temporal fraud-risk structure; RF's comparable-or-better AUPRC (0.846) here, achieved with no survival framing, proxy time variable, or time-reattachment step whatsoever, reinforces that conclusion directly. If a model with no access to any time-related information at all matches or exceeds RSF's full-dataset performance, the elevated AUPRC under SMOTE at full scale is best explained by a general interaction between SMOTE and tree-ensemble classification at large sample sizes, consistent with the mechanism already proposed in Section 8.5, rather than by anything specific to RSF's survival specification.

11. Cox Proportional Hazards on ULB

This section extends the model comparison to include Cox Proportional Hazards, a parametric survival model, following Maurizio's request for a fuller comparison spanning both classification and survival approaches.

While Section 10 isolated whether RSF's behaviour reflects its ensemble structure specifically (via the Random Forest classification control), this section isolates whether it reflects RSF's nonparametric flexibility relative to a parametric survival alternative, using the same `Surv(time, status)` specification applied throughout this notebook.

11.1. Cox Proportional Hazards (No SMOTE) - Downsampled

```
## Step 1: Run relevant packages
library(caret)
library(dplyr)
library(survival)
library(glmnet)
```

```
## Loading required package: Matrix
```

```
##
## Attaching package: 'Matrix'
```

```
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
```

```
## Loaded glmnet 4.1-10
```

```
library(pROC)
library(PRROC)

## Step 2: Set the seed
set.seed(44940076)

## Step 3: Establish the number of splits and threshold grid
n_splits <- 30
threshold_grid <- c(0.90, 0.95, 0.975, 0.99, 0.995)

## Step 4: Create storage - one set of vectors per threshold, plus concordance
## stored separately since it is not threshold-dependent
results_store <- list()
for (t in threshold_grid) {
  results_store[[as.character(t)]] <- list(
    sens  = numeric(n_splits),
    prec  = numeric(n_splits),
    auc   = numeric(n_splits),
    auprc = numeric(n_splits)
  )
}
cox_concordance_splits <- numeric(n_splits)

## Step 5: Execute Cox PH (No SMOTE) - ridge-penalized via glmnet
## Unpenalized coxph() was attempted first and failed to converge on early
## splits ("Ran out of iterations", "one or more coefficients may be
## infinite"), consistent with quasi-complete separation given the strength
## of individual PCA components (V14, V4) already established in the RSF/LR
## variable importance results. A univariate range-overlap diagnostic found
## only weak evidence for any single dominant separator (V28, 8.6% overlap),
## suggesting the separation is more likely multivariate - i.e. driven by a
## combination of features rather than one feature alone - which a ridge
```

```r
## penalty addresses regardless of the specific mechanism. Ridge (alpha = 0)
## regularization follows Simon et al. (2011).
for (i in 1:n_splits) {
  message("Split ", i, " of ", n_splits, " - ", Sys.time())

  # 1. Stratified 70-30 split
  cox_train_idx <- createDataPartition(fraud_subset$status, p = 0.7, list = FALSE)
  cox_train_data <- fraud_subset[cox_train_idx, ]
  cox_test_data  <- fraud_subset[-cox_train_idx, ]

  # 2. Keep only survival-relevant variables, matching the RSF (No SMOTE)
  # treatment in Section 7.4 - Class is dropped since it duplicates status
  cox_train_data <- cox_train_data %>% dplyr::select(-Class)
  cox_test_data  <- cox_test_data  %>% dplyr::select(-Class)

  cox_train_data$status <- as.numeric(as.character(cox_train_data$status))
  cox_test_data$status  <- as.numeric(as.character(cox_test_data$status))

  # 3. Fit ridge-penalized Cox (ONCE per split) - lambda chosen via
  # cross-validation within the training partition only
  cox_predictors <- setdiff(names(cox_train_data), c("time", "status"))
  cox_x_train <- as.matrix(cox_train_data[, cox_predictors])
  cox_x_test  <- as.matrix(cox_test_data[, cox_predictors])
  cox_y_train <- Surv(cox_train_data$time, cox_train_data$status)

  cox_cv <- cv.glmnet(cox_x_train, cox_y_train, family = "cox", alpha = 0)
  cox_model <- glmnet(cox_x_train, cox_y_train, family = "cox",
                      alpha = 0, lambda = cox_cv$lambda.min)

  # 4. Risk scores - type = "link" gives the linear predictor (log hazard
  # ratio); higher = higher risk, same ordering convention as coxph's
  # type = "risk" (monotonic transform, so downstream rank-based metrics
  # and quantile thresholds are unaffected)
  cox_train_risk <- as.numeric(predict(cox_model, newx = cox_x_train, type = "link"))
  cox_test_risk  <- as.numeric(predict(cox_model, newx = cox_x_test,  type = "link"))

  # 4b. Concordance - glmnet objects do not support concordance() on the
  # fitted object directly (unlike coxph), so this uses the formula
  # interface with reverse = TRUE, following the same higher-risk-score
  # convention confirmed during the Sparkov account-level diagnostic
  cox_concordance_splits[i] <- concordance(
    Surv(time, status) ~ cox_test_risk, data = cox_test_data, reverse = TRUE
  )$concordance

  # 5. Sanity check - flag (not halt) if risk scores look degenerate
  if (any(!is.finite(cox_test_risk))) {
    warning("Split ", i, ": non-finite risk scores detected under ridge Cox")
  }

  # 6. ROC + AUC (threshold-independent, compute once)
  cox_roc_obj <- roc(response = cox_test_data$status, predictor = cox_test_risk, quiet = TRUE)
  cox_auc_i <- as.numeric(auc(cox_roc_obj))
```

```r
  # 7. AUPRC (threshold-independent, compute once)
  cox_true_labels <- cox_test_data$status
  cox_keep <- is.finite(cox_test_risk) & is.finite(cox_true_labels)
  cox_pr_obj <- pr.curve(
    scores.class0 = cox_test_risk[cox_keep][cox_true_labels[cox_keep] == 1],
    scores.class1 = cox_test_risk[cox_keep][cox_true_labels[cox_keep] == 0],
    curve = FALSE
  )
  cox_auprc_i <- cox_pr_obj$auc.integral

  # 8. Loop over thresholds
  for (t in threshold_grid) {
    cox_threshold <- quantile(cox_train_risk, probs = t, na.rm = TRUE)
    cox_test_pred <- ifelse(cox_test_risk > cox_threshold, 1, 0)

    cox_cm <- confusionMatrix(
      factor(cox_test_pred, levels = c(0, 1)),
      factor(cox_test_data$status, levels = c(0, 1)),
      positive = "1"
    )

    tk <- as.character(t)
    results_store[[tk]]$sens[i]  <- cox_cm$byClass["Sensitivity"]
    results_store[[tk]]$prec[i]  <- cox_cm$byClass["Precision"]
    results_store[[tk]]$auc[i]   <- cox_auc_i
    results_store[[tk]]$auprc[i] <- cox_auprc_i
  }
}
```

```
## Split 1 of 30 - 2026-08-16 20:14:17.839053
## Split 2 of 30 - 2026-08-16 20:16:40.846116
## Split 3 of 30 - 2026-08-16 20:19:00.452287
## Split 4 of 30 - 2026-08-16 20:21:21.551608
## Split 5 of 30 - 2026-08-16 20:23:43.804985
## Split 6 of 30 - 2026-08-16 20:26:04.311958
## Split 7 of 30 - 2026-08-16 20:28:24.614741
## Split 8 of 30 - 2026-08-16 20:30:49.725716
## Split 9 of 30 - 2026-08-16 20:33:08.071663
## Split 10 of 30 - 2026-08-16 20:35:31.233589
## Split 11 of 30 - 2026-08-16 20:37:56.59128
## Split 12 of 30 - 2026-08-16 20:40:22.168237
## Split 13 of 30 - 2026-08-16 20:42:41.604101
## Split 14 of 30 - 2026-08-16 20:45:06.794495
## Split 15 of 30 - 2026-08-16 20:47:30.541213
## Split 16 of 30 - 2026-08-16 20:49:47.797349
## Split 17 of 30 - 2026-08-16 20:52:10.089809
```

```
## Split 18 of 30 - 2026-08-16 20:54:31.537879
## Split 19 of 30 - 2026-08-16 20:56:58.871281
## Split 20 of 30 - 2026-08-16 20:59:21.607085
## Split 21 of 30 - 2026-08-16 21:01:43.384373
## Split 22 of 30 - 2026-08-16 21:04:02.793528
## Split 23 of 30 - 2026-08-16 21:06:25.0395
## Split 24 of 30 - 2026-08-16 21:08:53.110444
## Split 25 of 30 - 2026-08-16 21:11:17.740179
## Split 26 of 30 - 2026-08-16 21:13:40.793262
## Split 27 of 30 - 2026-08-16 21:16:03.802181
## Split 28 of 30 - 2026-08-16 21:18:27.988387
## Split 29 of 30 - 2026-08-16 21:20:52.580449
## Split 30 of 30 - 2026-08-16 21:23:16.344671
```

```r
## Step 6: Store threshold results
results_list <- list()
for (t in threshold_grid) {
  tk <- as.character(t)
  results_list[[tk]] <- data.frame(
    Threshold      = t,
    Sensitivity    = mean(results_store[[tk]]$sens),
    Sensitivity_SD = sd(results_store[[tk]]$sens),
    Precision      = mean(results_store[[tk]]$prec),
    Precision_SD   = sd(results_store[[tk]]$prec),
    AUC            = mean(results_store[[tk]]$auc),
    AUC_SD         = sd(results_store[[tk]]$auc),
    AUPRC          = mean(results_store[[tk]]$auprc),
    AUPRC_SD       = sd(results_store[[tk]]$auprc)
  )
}

results_cox_threshold <- do.call(rbind, results_list)
auprc_cox_splits <- results_store[["0.99"]]$auprc

saveRDS(results_cox_threshold, "cox_threshold_results.rds")
saveRDS(cox_concordance_splits, "cox_concordance_results.rds")
```

```r
library(pander)

pander(results_cox_threshold, digits = 6, caption = "Cox Proportional Hazards (Ridge, No SMOTE) Performa
```

Table 26: Cox Proportional Hazards (Ridge, No SMOTE) Performance Metrics (continued below)

	Threshold	Sensitivity	Sensitivity_SD	Precision
0.9	0.9	0.853207	0.0232339	0.0807486
0.95	0.95	0.825577	0.0237122	0.157195
0.975	0.975	0.773675	0.0263834	0.299718

	Threshold	Sensitivity	Sensitivity_SD	Precision
0.99	0.99	0.674263	0.0361755	0.664172
0.995	0.995	0.48503	0.0418702	0.933225

	Precision_SD	AUC	AUC_SD	AUPRC	AUPRC_SD
0.9	0.00602185	0.940722	0.0110317	0.683708	0.0314459
0.95	0.0112879	0.940722	0.0110317	0.683708	0.0314459
0.975	0.0215781	0.940722	0.0110317	0.683708	0.0314459
0.99	0.0328116	0.940722	0.0110317	0.683708	0.0314459
0.995	0.0206683	0.940722	0.0110317	0.683708	0.0314459

```
pander(data.frame(
  Mean_Concordance = mean(cox_concordance_splits),
  SD_Concordance   = sd(cox_concordance_splits)
), digits = 4, caption = "Cox Proportional Hazards - Concordance (30 Repeated Splits)")
```

Table 28: Cox Proportional Hazards - Concordance (30 Repeated Splits)

Mean_Concordance	SD_Concordance
0.991	0.003114

Observations: An unpenalised Cox fit failed to converge on the first set of several splits (generally, it either ran out of iteration, or had some infinite coefficients). This was consistent with quasi-complete separation given the strength of the individual PCA components (namely V14 and V4) that were already established in the RSF/LR variable importance results from Section 9.2. A univariate range-overlapping diagnostic found only weak evidence of any single dominant separator (such as V28, which had an 8.6% overlap). This suggested that the separation was multivariate, rather than being attributable to one feature. A ridge-penalised specification resolved this and converged cleanly across all 30 splits.

At the 0.99 threshold, which was used as the operating point throughout this notebook, Cox achieved 67.4% sensitivity and 66.4% precision - both being well below LR (No SMOTE)'s 88.5% sensitivity and 82.5% precision, and RSF (No SMOTE)'s 85.0% sensitivity and 82.1% precision (Sections 8.1.1 and 8.1.3 covered this). This gap was held across the threshold independent metrics: AUC of 0.941 (SD 0.011) against LR and RSF's AUC of 0.976-0.979, and AUPRC of 0.684 (SD 0.031) against LR and RSF's AUPRC of 0.866-0.868. This extends the thesis' central finding along a third axis: not only does the survival framing (in RSF) fail to outperform classification on the ULB, but a *parametric* survival model performs materially worse than both. This reinforces that ULB's structural conditions (PCA-orthogonalised, extreme censoring) disadvantages the survival paradigm, and not merely based on its nonparametric implementation.

Cox's concordance, by contrast, was 0.991 (with SD 0.003). This was especially high, given the AUC/AUPRC results show above, and investigated below in Section 11.2.

11.2. Concordance Diagnostic

```
## Diagnostic: does concordance collapse toward 0.5 if the proxy time index
## is shuffled (destroying its correlation with anything real) while keeping
## the same risk scores and event/censoring labels? If concordance stays
## high under a shuffled time, that confirms it isn't measuring genuine
## time-ordering at all - it's an artefact of the exclusion rule interacting
```

```r
## with the proxy index's structure.
set.seed(44940076)
shuffled_time <- sample(cox_test_data$time)

concordance_shuffled <- concordance(
  Surv(shuffled_time, cox_test_data$status) ~ cox_test_risk, reverse = TRUE
)$concordance

concordance_shuffled
```

```
## [1] 0.9201603
```

```r
cox_concordance_splits[30]  # for comparison against the real (non-shuffled) value from the last split
```

```
## [1] 0.9859433
```

Observations: Shuffling the proxy time index while holding the risk scores and event status fixed, produced a concordance of 0.920, which was not the collapse towards 0.5 that was expected if concordance genuinely depended on time-ordering. This was close to the model's own AUC (0.941), revealing that the underlying mechanism (known as Harrell's concordance), under its comparable-pairs restriction (a pair that only counts if the earlier-time subject experienced the event), is largely re-measuring the model's ability to discriminate fraud from non-fraud, rather than focusing on the genuine time-ordering - since fraud is a rare and binary outcome. The real proxy time index added a further, smaller boost on top of this baseline (0.920 as shuffled vs 0.986-0.991 as real), which was consistent with the weak but real row-index structure that was already identified elsewhere in this Notebook (Section 9.2. Time ranking among the RSF's Top Variable Importance; Section 7.6. The SMOTE Feature Space/Time Index Gap Diagnostic), rather than with genuine time-to-event information. Given this, the Concordance is not treated as a reliable headline metric for this comparison, and AUPRC/AUC are the primary basis for evaluating Cox against RSF and LR.